

C++ Standard Library Issues to be moved in Issaquah

Doc. no. P0165R3

Date: Revised 2016-10-17 at 04:10:06 UTC

Project: Programming Language C++

Reply to: Marshall Clow <lwgchair@gmail.com>

Ready Issues

[2062](#). Effect contradictions w/o no-throw guarantee of `std::function` swaps

Section: 20.14.12.2 [func.wrap.func], 20.14.12.2.2 [func.wrap.func.mod] **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krügler
Opened: 2011-05-28 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [func.wrap.func].

View all other [issues](#) in [func.wrap.func].

View all issues with [Tentatively Ready](#) status.

Discussion:

Howard Hinnant observed in reflector message c++std-lib-30841 that 20.14.12.2 [func.wrap.func] makes the member swap `noexcept`, even though the non-member swap is not `noexcept`.

The latter was an outcome of the discussions during the Batavia meeting and the Madrid meeting involving LWG [1349](#), which seems to indicate that the remaining `noexcept` specifier at the member swap is incorrect and should be removed.

But if we allow for a potentially throwing member swap of `std::function`, this causes another conflict with the exception specification for the following member function:

```
template<class F> function& operator=(reference_wrapper<F> f) noexcept;
```

Effects: `function(f).swap(*this);`

Note that in this example the sub-expression `function(f)` does not cause any problems, because of the nothrow-guarantee given in 20.14.12.2.1 [func.wrap.func.con] p. 10. The problem is located in the usage of the swap which could potentially throw given the general latitude.

So, either the Madrid meeting decision need to be revised (and both member and free swap of `std::function` should be `noexcept`), or this function needs to be adapted as well, e.g. by taking the exception-specification away or by changing the semantics.

One argument for "swap-may-throw" would be to allow for small-object optimization techniques where the copy of the target may throw. But given the fact that the swap function has been guaranteed to be "Throws: Nothing" from TR1 on, it seems to me that that there would still be opportunities to perform small-object optimizations just restricted to the set of target copies that cannot throw.

In my opinion member swap of `std::function` has always been intended to be no-throw, because otherwise there would be no good technical reason to specify the effects of several member functions in terms of the "construct-swap" idiom (There are three functions that are defined this way), which provides the strong exception safety in this case. I suggest to enforce that both member swap and non-member swap of `std::function` are nothrow functions as it had been guaranteed since TR1 on.

[2011 Bloomington]

Dietmar: May not be swappable in the first place.

Alisdair: This is wide contact. Then we should be taking `noexcept` off instead of putting it on. This is preferred resolution.

Pablo: This is bigger issue. Specification of assignment in terms of swap is suspect to begin with. It is over specification. How this was applied to string is a better example to work from.

Pablo: Two problems: inconsistency that should be fixed (neither should have `noexcept`), the other issues is that assignment

should not be specified in terms of swap. There are cases where assignment should succeed where swap would fail. This is easier with string as it should follow container rules.

Action Item (Alisdair): There are a few more issues found to file.

Dave: This is because of allocators? The allocator makes this not work.

Howard: There is a type erased allocator in `shared_ptr`. There is a `noexcept` allocator in `shared_ptr`.

Pablo: `shared_ptr` is a different case. There are shared semantics and the allocator does move around. A function does not have shared semantics.

Alisdair: Function objects think they have unique ownership.

Howard: In function we specify semantics with copy construction and swap.

Action Item (Pablo): Write this up better (why assignment should not be defined in terms of swap)

Howard: Not having trouble making function constructor no throw.

Dietmar: Function must allocate memory.

Howard: Does not put stuff that will throw on copy or swap in small object optimization. Put those on heap. Storing allocator, but has to be no throw copy constructable.

Pablo: Are you allowed to or required to swap or move allocators in case or swap or move.

Dave: An allocator that is type erased should be different...

Pablo: it is

Dave: Do you need to know something about allocator types? But only at construction time.

Pablo: You could have allocators that are different types.

Dave: Swap is two ended operation.

Pablo: Opinion is that both have to say propagate on swap for them to swap.

John: It is not arbitrary. If one person says no. No is no.

Howard: Find `noexcept` swap to be very useful. Would like to move in that direction and bring container design along.

Dave: If you have something where allocator must not propagate you can detect that at construction time.

...

Pablo: Need to leave this open and discuss in smaller group.

Alisdair: Tried to add `boost::any` as TR2 proposal and ran into this issue. Only the first place where we run into issues with type erased allocators. Suggest we move it to open.

Action Item: Move to open.

Action Item (Pablo works with Howard and Daniel): Address the more fundamental issue (which may be multiple issues) and write up findings.

Previous resolution [SUPERSEDED]:

This wording is relative to the FDIS.

1. Modify the header `<functional>` synopsis in 20.14 [function.objects] as indicated:

```
namespace std {
    [...]

    template<class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;

    [...]
}
```

2. Modify the class template function synopsis in 20.14.12.2 [func.wrap.func] as indicated:

```
namespace std {  
    [...]  
  
    // [func.wrap.func.alg], specialized algorithms:  
    template<class R, class... ArgTypes>  
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;  
  
    [...]  
}
```

3. Modify 20.14.12.2.7 [func.wrap.func.alg] as indicated:

```
template<class R, class... ArgTypes>  
void swap(function<R(ArgTypes...)>& f1, function<R(ArgTypes...)>& f2) noexcept;  
  
-1- Effects: f1.swap(f2);
```

[2014-02-28 (Post Issaquah), Pablo provides more information]

For cross-referencing purposes: The resolution of this issue should be harmonized with any resolution to LWG [2370](#), which addresses inappropriate `noexcept`s in some function constructors.

We have the following choices:

1. `swap()` does not throw

Discussion: This definition is desirable, and allows assignment to be implemented with the strong exception guarantee, but it does have consequences: The implementation cannot use the small-object optimization for a function-object `F` unless `F` is `NothrowMovable` (nothrow-swappable is unimportant because `F` is not swapped with another `F`). Note that many functors written before C++11 will not have move constructors decorated with `noexcept`, so this limitation could affect a lot of code.

It is not clear what other implementation restrictions might be needed. Allocators are required not to throw on move or copy. Is that sufficient?

2. `swap()` can throw

Discussion: This definition gives maximum latitude to implementation to use small-object optimization. However, the strong guarantee on assignment is difficult to achieve. Should we consider giving up on the strong guarantee? How much are we willing to pessimize code for exceptions?

3. `swap()` will not throw if both functions have `NoThrowMoveable` functors

Discussion: This definition is similar to option 2, but gives slightly stronger guarantees. Here, `swap()` can throw, but the programmer can theoretically prevent that from happening. This should be straight-forward to implement and gives the implementation a lot of latitude for optimization. However, because this is a dynamic decision, the program is not as easy to reason about. Also, the strong guarantee for assignment is compromised as in option 2.

[2016-08-02, Ville, Billy, and Billy comment and reinstantiate the original P/R]

We (Ville, Billy, and Billy) propose to require that function's `swap` is `noexcept` in all cases.

- `libstdc++` does not throw in their `swap`. It is not `noexcept` today, but the small functor optimization only engages for trivially copyable types.
- `msvc++` checks `is_nothrow_move_constructible` before engaging the small functor optimization and marks its `swap` `noexcept`
- `libc++` marks `swap` `noexcept` (though I have not looked at its implementation)

Moreover, many of the concerns that were raised by providing this guarantee are no longer applicable now that [P0302](#) has been accepted, which removes allocator support from `std::function`.

Therefore we are re-proposing the original resolution above.

[2016-08 Chicago]

Tues PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify the header `<functional>` synopsis in 20.14 [function.objects] as indicated:

```
namespace std {  
    [...]  
  
    template<class R, class... ArgTypes>  
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;  
  
    [...]  
}
```

2. Modify the class template `function` synopsis in 20.14.12.2 [func.wrap.func] as indicated:

```
namespace std {  
    [...]  
  
    // [func.wrap.func.alg], specialized algorithms:  
    template<class R, class... ArgTypes>  
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&) noexcept;  
  
    [...]  
}
```

3. Modify 20.14.12.2.7 [func.wrap.func.alg] as indicated:

```
template<class R, class... ArgTypes>  
void swap(function<R(ArgTypes...)>& f1, function<R(ArgTypes...)>& f2) noexcept;  
  
-1- Effects: As if by: f1.swap(f2);
```

2166. Heap property underspecified?

Section: 25.5.6 [alg.heap.operations] **Status:** [Tentatively Ready](#) **Submitter:** Peter Sommerlad **Opened:** 2012-07-09 **Last modified:** 2016-08-08

Priority: 3

View all other [issues](#) in [alg.heap.operations].

View all issues with [Tentatively Ready](#) status.

Discussion:

Another similar issue to the `operator<` vs `greater` in `nth_element` but not as direct occurs in 25.5.6 [alg.heap.operations]:

-1- A *heap* is a particular organization of elements in a range between two random access iterators `[a,b)`. Its two key properties are:

1. There is no element greater than `*a` in the range and
2. `*a` may be removed by `pop_heap()`, or a new element added by `push_heap()`, in $O(\log(N))$ time.

As noted by Richard Smith, it seems that the first bullet should read:

`*a` is not less than any element in the range

Even better the heap condition could be stated here directly, instead of leaving it unspecified, i.e.,

Each element at $(a+2*i+1)$ and $(a+2*i+2)$ is less than the element at $(a+i)$, if those elements exist, for $i \geq 0$.

But may be that was may be intentional to allow other heap organizations?

See also follow-up discussion of `c++std-lib-32780`.

[2016-08 Chicago]

Walter provided wording

Tues PM: Alisdair & Billy(MS) to improve the wording.

[2016-08-02 Chicago LWG]

Walter provides initial Proposed Resolution. Alisdair objects to perceived complexity of the mathematical phrasing.

Previous resolution [SUPERSEDED]:

[Note to editor: As a drive-by editorial adjustment, please replace the current enumerated list format by the numbered bullet items shown below.]

Change [alg.heap.operations]:

1 A heap is a particular organization of elements in a range between two random access iterators [a, b). Its two key properties are such that:

(1.1) -- There is no element greater than *a in the range and

For all $i \geq 0$,

$\text{comp}(a[i], a[L])$ is false whenever $L = 2*i+1 < b-a$,

and

$\text{comp}(a[i], a[R])$ is false whenever $R = 2*i+2 < b-a$.

(1.2) -- *a may be removed by `pop_heap()`, or a new element added by `push_heap()`, in $O(\log(N))$ time.

[2016-08-03 Chicago LWG]

Walter and Billy O'Neal provide revised Proposed Resolution, superseding yesterday's.

Thurs PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 25.5.6 [alg.heap.operations] as indicated:

Note to project editor: As a drive-by editorial adjustment, please replace the current enumerated list format by numbered bullet items.

-1- A *heap* is a particular organization of elements in a range between two random access iterators [a, b). Its two key properties are such that:

(1.1) — There is no element greater than *a in the range and With $N \equiv b - a$, for all i , $0 \leq i < N$, $\text{comp}(a[\lfloor \frac{i-1}{2} \rfloor], a[i])$ is false.

[Note to the project editor: In LaTeX the above insertion should be expressed as follows:

With $N = b - a$, for all i , $0 < i < N$, $\text{comp}(a[\lfloor \frac{i-1}{2} \rfloor], a[i])$ is false.]

(1.2) — *a may be removed by `pop_heap()`, or a new element added by `push_heap()`, in $O(\log(N))$ time.

2221. No formatted output operator for `nullptr`

Section: 27.7.3 [output.streams] **Status:** [Tentatively Ready](#) **Submitter:** Matt Austern **Opened:** 2012-12-07 **Last modified:** 2016-08-06

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

When I write

```
std::cout << nullptr << std::endl;
```

I get a compilation error, "ambiguous overload for 'operator<<' in 'std::cout << nullptr'". As far as I can tell, the compiler is right to

issue that error. There are inserters for `const void*`, `const char*`, `const signed char*`, and `const unsigned char*`, and none for `nullptr_t`, so the expression really is ambiguous.

Proposed wording:

The obvious library solution is to add a `nullptr_t` overload, which would be defined something like

```
template<class C, class T>
basic_ostream<C, T>& operator<<(basic_ostream<C, T>& os, nullptr_t)
{
    return os << (void*) nullptr;
}
```

We might also consider addressing this at a core level: add a special-case language rule that addresses all cases where you write `f(nullptr)` and `f` is overloaded on multiple pointer types. (Perhaps a tiebreaker saying that `void*` is preferred in such cases.)

[2016-01-18, comments from Mike and Ville collected by Walter Brown]

Mike Miller: "Changing overload resolution sounds like something that should be considered by EWG before CWG [...]"

Ville: "Agreed, such a change would be Evolutionary. Personally, I think it would also be wrong, because I don't see how `void*` is the right choice to prefer in the case of code that is currently ambiguous. Sure, it would solve this particular library issue, but it seemingly has wider repercussions. If LWG really wants to, EWG can certainly discuss this issue, but I would recommend solving it on the LWG side (which doesn't mean that the standard necessarily needs to change, I wouldn't call it far-fetched to NAD it)."

[2016-08 Chicago]

Zhihao recommends NAD:

`nullptr` is printable if being treated as `void*`, but causes UB if being treated as `char cv*`. Capturing this ambiguity at compile time and avoid a runtime UB is a good thing.

[2016-08 Chicago]

Tues PM: General agreement on providing the overload; discussion on what it should say.

Polls:

Matt's suggestion (in the issue): 2/0/6/2/2/

Unspecified output: 3/2/5/0/1

Specified output: 1/1/6/3/0

Move to Open

[2016-08 Chicago]

The group consensus is that we only output `nullptr` because it is of a fundamental type, causing problems in functions doing forwarding, and we don't want to read it back.

Fri PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606

1. Insert the signature into 27.7.3.1 [ostream], class template `basic_ostream` synopsis, as follows:

[Drafting notes: Why member? Don't want to define a new category of inserters just for this.]

```
namespace std {
    template <class charT, class traits = char_traits<charT> >
    class basic_ostream
    : virtual public basic_ios<charT, traits> {
    public:
        [...]
        basic_ostream<charT, traits>& operator<<(const void* p);
        basic_ostream<charT, traits>& operator<<(nullptr t);
        basic_ostream<charT, traits>& operator<<(
            basic_streambuf<char_type, traits>* sb);
        [...]
    };
```

2. Append the following new paragraphs to 27.7.3.6.3 [ostream.inserters]:

```
basic_ostream<charT, traits>& operator<<
(basic_streambuf<charT, traits>* sb);
```

[...]

-10- *Returns:* *this.

```
basic_ostream<charT, traits>& operator<<(nullptr t):
```

```
-??- Effects: Equivalent to return *this << s; where s is an implementation-defined NTCTS.
```

2223. shrink_to_fit effect on iterator validity

Section: 23.3.11.3 [vector.capacity] **Status:** [Tentatively Ready](#) **Submitter:** Juan Soulie **Opened:** 2012-12-17 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [vector.capacity].

View all other [issues](#) in [vector.capacity].

View all issues with [Tentatively Ready](#) status.

Discussion:

After the additions by [2033](#), it appears clear that the intended effect includes a reallocation and thus the potential effect on iterators should be explicitly added to the text in order to not contradict 23.2.1 [container.requirements.general]/11, or at the very least, explicitly state that a reallocation may happen.

Taking consistency with "reserve" into consideration, I propose:

- that the current "Remarks" are made its "Effect" instead, inserting "Reallocation happens at this point if and only if the function effectively reduces the capacity." after the note on non-bindingness.
- adding a "Remarks" paragraph, similar to that of reserve: "Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence."

BTW, while we are at it, I believe the effect on iterators should also be explicitly stated in the other instance a reallocation may happen: 23.3.11.5 [vector.modifiers]/1 — even if obvious, it only contradicts 23.2.1 [container.requirements.general]/11 implicitly.

I propose to also insert "Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence." at the appropriate location in its "Remarks".

[2012-12-19: Jonathan Wakely comments]

The described problem also affects `std::basic_string` and `std::deque`.

[2013-03-15 Issues Teleconference]

Moved to Review.

[2013-04-18, Bristol]

Daniel extends the P/R.

Rationale:

The wording in 21.3.1.4 [string.capacity] combined with 21.3.1.1 [string.require] seems to say the necessary things. We cannot impose all requirements as we do for `vector`, because we want to allow the short-string-optimization.

[2014-02-15 post-Issaquah session]

STL: I think that `shrink_to_fit` should be a no-op when called twice.

STL: Do we ever define reallocation for `deque`? Nope, all mentions of "reallocation" are in `vector`. We define what it means in `vector::reserve()`, but not for `deque`.

STL: Oh duh, they define reallocate in the PR. But I think we can do better here.

STL: Optimally, `deque` shrinking just allocates a new map of pointers, and drops empty blocks, but preserves pointers/references

to elements.

Alisdair: That's like unordered containers, invalidating only iterators.

Pablo: It doesn't make sense to reduce `capacity()` to `size()`, because `deque` doesn't have capacity!

STL: For `vector`, "effectively reduces the capacity" is unnecessary, the capacity there is observable.

STL: There is a strong reason to provide an optimal shrink to fit for `deque`, since only the library implementer can do this.

STL: The other thing I don't like the repeated definition of reallocation for `vector`, we define it once and use it in a bunch of places. At most we can lift it up to the `vector` synopsis.

STL: I'll write new wording.

[2014-10-01, STL adds discussion and provides new wording]

Compared to the previous proposed resolution:

- I'm changing `basic_string`'s wording because (1) we should guarantee that capacity won't increase, (2) we should mention that it's linear complexity, and (3) we can provide a better invalidation guarantee than 21.3.1.1 [string.require]/5. (As previously noted, we already have the strong exception guarantee.) This introduces the term "reallocation" into `basic_string`, but immediately explains what it means for iterator validity. As far as I can tell, the Small String Optimization doesn't complicate the wording here; it's a reason why an implementation might not honor the request, but if the capacity is reduced, we are definitely reallocating buffers and will invalidate everything (including when the destination is the small buffer).
- Between N3485 and N3936, `deque`'s wording was updated to avoid talking about `capacity()` which it doesn't have. Since the container's capacity is unobservable, I'm saying that invalidation is unconditional.
- In `vector`'s wording, I'm also guaranteeing that capacity won't increase, and that iterators/etc. remain valid if the capacity is unchanged.

My wording doesn't directly say that `shrink_to_fit()` should be a no-op when called twice in a row. (Indirectly, if the first call reduces `capacity()` to `size()`, the second call must preserve iterators/etc.) I considered rewording the complexity to say "linear if reallocation happens", but that's potentially problematic (what if we copy almost all `n` elements, then one throws and we have to unwind? There are no effects, so reallocation didn't happen, yet we took longer than constant time). Implementers can always do better than the stated complexity bounds.

I chose not to modify `deque`'s requirements, so implementations remain free to reallocate the elements themselves.

I didn't attempt to centralize `vector`'s reallocation wording. That can be done editorially, if someone is sufficiently motivated.

Previous resolution from Juan Soulie/Daniel [SUPERSEDED]:

This wording is relative to N3485.

1. Keep 21.3.1.4 [string.capacity] around p14 *unchanged*, because we don't speak about reallocations and we give the strong exception guarantee in 21.3.1.1 [string.require] (Invalidation specification also at that place):

```
void shrink_to_fit();
```

-14- *Remarks:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*].

2. Edit 23.3.8.3 [deque.capacity] around p7 as indicated:

```
void shrink_to_fit();
```

-5- *Requires:* `T` shall be `MoveInsertable` into `*this`.

-?- *Effects:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*] Reallocation happens at this point if and only if the function effectively reduces the capacity. If an exception is thrown other than by the move constructor of a non-`CopyInsertable` `T` there are no effects.

-6- *Complexity:* Linear in the size of the sequence.

~~-7- *Remarks:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*] If an~~

exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence.

3. Edit 23.3.11.3 [vector.capacity] around p7 as indicated:

```
void shrink_to_fit();
```

-7- *Requires:* T shall be MoveInsertable into $*this$.

-?- *Effects:* shrink_to_fit is a non-binding request to reduce capacity() to size(). [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] Reallocation happens at this point if and only if the function effectively reduces the capacity. If an exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects.

-8- *Complexity:* Linear in the size of the sequence.

-9- *Remarks:* shrink_to_fit is a non-binding request to reduce capacity() to size(). [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] If an exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence.

4. Edit 23.3.11.5 [vector.modifiers] p1 as indicated:

```
iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template <class InputIterator>
iterator insert(const_iterator position, InputIterator first, InputIterator last);
iterator insert(const_iterator position, initializer_list<T>);
template <class... Args> void emplace_back(Args&&... args);
template <class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_back(const T& x);
void push_back(T&& x);
```

-1- *Remarks:* Causes reallocation if the new size is greater than the old capacity. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. If no reallocation happens, all the iterators and references before the insertion point remain valid. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of T or by any InputIterator operation there are no effects. If an exception is thrown by the move constructor of a non-CopyInsertable T , the effects are unspecified.

[2015-02 Cologne]

GR: I'm concerned that shrink_to_fit may cause reallocation without changing the capacity. [...] It's about correctness. The statement about invalidation is useless if I cannot detect whether reallocation has happened?

AM: It seems like the logic goes the other way round: It's the capacity change that causes reallocation, so if there's no capacity change, there's no reallocation. But that's not quite how I'd like to say it... maybe this, : "If capacity does not change, no reallocation occurs."

GR: Where does it actually say that reserve() invalidates? AM: It should say that in the container requirements. VV: vector specifies in reserve that there's reallocation if and only if the capacity changes. GR: I can't find anything in the container requirements about reserve. DK: No, it's specified for every container separately. GR: It isn't specified for string.

GR: I'm noticing that the issue touches on shrink_to_fit for a bunch of containers. Anyway, I think the reserve issue [re string] is in scope for this issue. This change is touching on a lot of members.

AM: Landing this change will provide clarity for what we should do with basic_string. GR: We're already asking for changes; we should fix string as well. AM: If one of the changes is ready before the other, I'd like to land the finished part first, but if both are ready for Lenexa, I'm equally happy to fix them in one go.

DK will reword this.

Conclusion: Update wording, revisit in Lenexa.

[2016-08 Chicago]

Monday PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N3936.

1. Change 21.3.1.4 [string.capacity] p14 as depicted:

```
void shrink_to_fit();
```

-14- ~~Remarks~~**Effects**: shrink_to_fit is a non-binding request to reduce capacity() to size(). [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] **It does not increase capacity(), but may reduce capacity() by causing reallocation.**

-?- Complexity: Linear in the size of the sequence.

-?- Remarks: Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. If no reallocation happens, they remain valid.

2. Change 23.3.8.3 [deque.capacity] p5-p7 as depicted:

```
void shrink_to_fit();
```

-5- **Requires**: T shall be MoveInsertable into *this.

-?- Effects: shrink to fit is a non-binding request to reduce memory use but does not change the size of the sequence. [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] If an exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects.

-6- Complexity: Linear in the size of the sequence.

-7- Remarks: shrink_to_fit is a non-binding request to reduce memory use but does not change the size of the sequence. [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] shrink_to_fit invalidates all the references, pointers, and iterators referring to the elements in the sequence.

3. Change 23.3.11.3 [vector.capacity] p7-p9 as depicted:

```
void shrink_to_fit();
```

-7- **Requires**: T shall be MoveInsertable into *this.

-?- Effects: shrink to fit is a non-binding request to reduce capacity() to size(). [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] It does not increase capacity(), but may reduce capacity() by causing reallocation. If an exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects.

-8- Complexity: Linear in the size of the sequence.

-9- Remarks: shrink_to_fit is a non-binding request to reduce capacity() to size(). [Note: The request is non-binding to allow latitude for implementation-specific optimizations. — end note] If an exception is thrown other than by the move constructor of a non-CopyInsertable T there are no effects. Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. If no reallocation happens, they remain valid.

4. Change 23.3.11.5 [vector.modifiers] p1 as depicted:

-1- **Remarks**: Causes reallocation if the new size is greater than the old capacity. **Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence.** If no reallocation happens, all the iterators and references before the insertion point remain valid. [...]

2261. Are containers required to use their 'pointer' type internally?

Section: 23.2 [container.requirements] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2013-05-14 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [container.requirements].

View all other [issues](#) in [container.requirements].

View all issues with [Tentatively Ready](#) status.

Discussion:

Is a container `c` only supposed to refer to allocated memory (blocks of contiguous storage, nodes, etc.) through objects of type `C::pointer` rather than `C::value_type*`?

I don't see anything explicitly requiring this, so a container could immediately convert the result of `get_allocator().allocate(1)` to a built-in pointer of type `value_type*` and only deal with the built-in pointer until it needs to deallocate it again, but that removes most of the benefit of allowing allocators to use custom pointer types.

[2014-06-12, Jonathan comments]

This issue is basically the same issue as LWG [1521](#), which agrees it's an issue, to be dealt with in the future, so I request that [2261](#) not be closed as a dup unless we reopen [1521](#).

[2016-08, Zhihao comments]

The `pointer` types are not exposed in the container interface, and we consider that the memory allocation constraints "all containers defined in this clause obtain memory using an allocator" already implies the reasonable expectation. We propose the fix as non-normative.

[2016-08 Chicago]

Tues PM: General agreement on direction, Alisdair and Billy to update wording

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

[Drafting notes: if people prefer this to be normative, strip the "Note" markups.]

Modify 23.2.1 [container.requirements.general]/8 as indicated:

Unless otherwise specified, all containers defined in this clause obtain memory using an allocator (see 17.6.3.5 [allocator.requirements]). *[Note: In particular, containers and iterators do not store references to allocated elements other than through the allocator's pointer type, i.e., as objects of type `P` or `pointer_traits<P>::template rebind<unspecified>`, where `P` is `allocator_traits<allocator_type>::pointer`. — end note]*

[2394](#). `locale::name` specification unclear — what is implementation-defined?

Section: 22.3.1.3 [locale.members] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2014-06-09 **Last modified:** 2016-08-04

Priority: 3

View all other [issues](#) in [locale.members].

View all issues with [Tentatively Ready](#) status.

Discussion:

22.3.1.3 [locale.members] p5 says:

Returns: The name of `*this`, if it has one; otherwise, the string `""`. If `*this` has a name, then `locale(name().c_str())` is equivalent to `*this`. Details of the contents of the resulting string are otherwise implementation-defined.

So... what is implementation-defined here, exactly? The first sentence *completely* defines the behavior of this function in all cases.

Also, the second sentence says (effectively) that all locales with the same name are equivalent: given `L1` and `L2` that have the same name `N`, they are both equivalent to `locale(N)`, and since there is no definition of "equivalent" specific to `locale`, I assume it's the normal transitive equivalence property, which would imply that `L1` is equivalent to `L2`. I'm not sure why this central fact is in the description of `locale::name`, nor why it's written in this roundabout way.

[2016-08-03 Chicago LWG]

Walter, Nevin, and Jason provide initial Proposed Resolution.

[2016-08 - Chicago]

Thurs PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 22.3.1.3 [locale.members] as indicated:

```
basic_string<char> name() const;
```

-5- *Returns:* The name of **this*, if it has one; otherwise, the string `""`. ~~If **this* has a name, then `locale(name().c_str())` is equivalent to **this*. Details of the contents of the resulting string are otherwise implementation-defined.~~

2460. LWG issue 2408 and value categories

Section: 20.15.7.6 [meta.trans.other], 24.4.1 [iterator.traits] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2014-11-19 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [meta.trans.other].

View all other [issues](#) in [meta.trans.other].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG issue [2408](#) changes the meat of the specification of `common_type` to compute:

[...] the type, if any, of an unevaluated conditional expression (5.16) whose first operand is an arbitrary value of type `bool`, whose second operand is an `xvalue` of type `T1`, and whose third operand is an `xvalue` of type `T2`.

This has an effect on the specification that I think was unintended. It used to be the case that `common_type<T&, U&&>` would consider the type of a conditional between an lvalue of type `T` and an `xvalue` of type `U`. It's now either invalid (because there is no such thing as an `xvalue` of reference type) or considers the type of a conditional between an `xvalue` of type `T` and an `xvalue` of type `U`, depending on how you choose to read it.

Put another way, this has the effect of changing the usual definition from:

```
typedef decay_t<decltype(true ? declval<T>() : declval<U>())> type;
```

to:

```
typedef decay_t<decltype(true ? declval<remove_reference_t<T>>() : declval<remove_reference_t<U>>())> type;
```

It also makes `common_type` underspecified in the case where one of the operands is of type `void`; in that case, the resulting type depends on whether the expression is a throw-expression, which is not specified (but used to be).

Also on the subject of this wording: the changes to 24.4.1 [iterator.traits] say that `iterator_traits<T>` "shall have no members" in some cases. That's wrong. It's a class type; it always has at least a copy constructor, a copy assignment operator, and a destructor. Plus this removes the usual library liberty to add additional members with names that don't collide with normal usage (for instance, if a later version of the standard adds members, they can't be present here as a conforming extension). Perhaps this should instead require that the class doesn't have members with any of those five names? That's what [2408](#) does for `common_type`'s type member.

[2016-08 Chicago]

This issue has two parts, one dealing with `common_type`, the other with `iterator_traits`. The first of these is resolved by [2465](#). See below for the proposed resolution for the other one.

Wed PM: Move to Tentatively Ready

Proposed resolution:

Change 24.4.1 [iterator.traits] p.2:

[...] as publicly accessible members ~~and no other members~~:

[...]

Otherwise, `iterator_traits<Iterator>` shall have no members by any of the above names.

2468. Self-move-assignment of library types

Section: 17.6.4.9 [res.on.arguments], 17.6.3.1 [utility.arg.requirements], 17.6.5.15 [lib.types.movedfrom], 23.2.1 [container.requirements.general] **Status:** [Tentatively Ready](#) **Submitter:** Matt Austern **Opened:** 2015-01-22 **Last modified:** 2016-10-07

Priority: 2

View all other [issues](#) in [res.on.arguments].

View all issues with [Tentatively Ready](#) status.

Discussion:

Suppose we write

```
vector<string> v{"a", "b", "c", "d"};
v = move(v);
```

What should be the state of `v` be? The standard doesn't say anything specific about self-move-assignment. There's relevant text in several parts of the standard, and it's not clear how to reconcile them.

17.6.4.9 [res.on.arguments] writes that, for all functions in the standard library, unless explicitly stated otherwise, "If a function argument binds to an rvalue reference parameter, the implementation may assume that this parameter is a unique reference to this argument." The `MoveAssignable` requirements table in 17.6.3.1 [utility.arg.requirements] writes that, given `t = rv`, `t`'s state is equivalent to `rv`'s from before the assignment and `rv`'s state is unspecified (but valid). For containers specifically, the requirements table in 23.2.1 [container.requirements.general] says that, given `a = rv`, `a` becomes equal to what `rv` was before the assignment (and doesn't say anything about `rv`'s state post-assignment).

Taking each of these pieces in isolation, without reference to the other two:

- 17.6.4.9 [res.on.arguments] would clearly imply that the effect of `v = move(v)` is undefined.
- 17.6.3.1 [utility.arg.requirements] would clearly imply that `v = move(v)` has defined behavior. It might be read to imply that this is a no-op, or might be read to imply that it leaves `v` in a valid but unspecified state; I'm not sure which reading is more natural.
- 23.2.1 [container.requirements.general] would clearly imply that `v = move(v)` is a no-op.

It's not clear from the text how to put these pieces together, because it's not clear which one takes precedence. Maybe 17.6.4.9 [res.on.arguments] wins (it imposes an implicit precondition that isn't mentioned in the `MoveAssignable` requirements, so `v = move(v)` is undefined), or maybe 23.2.1 [container.requirements.general] wins (it explicitly gives additional guarantees for `Container::operator=` beyond what's guaranteed for library functions in general, so `v = move(v)` is a no-op), or maybe something else.

On the existing implementations that I checked, for what it's worth, `v = move(v)` appeared to clear the vector; it didn't leave the vector unchanged and it didn't cause a crash.

Proposed wording:

Informally: change the `MoveAssignable` and `Container` requirements tables (and any other requirements tables that mention move assignment, if any) to make it explicit that `x = move(x)` is defined behavior and it leaves `x` in a valid but unspecified state. That's probably not what the standard says today, but it's probably what we intended and it's consistent with what we've told users and with what implementations actually do.

[2015-10, Kona Saturday afternoon]

JW: So far, the library forbids self-assignment since it assumes that anything bound to an rvalue reference has no aliases. But self-assignment can happen in real code, and it can be implemented. So I want to add an exception to the Standard that this should be allowed and leave the object in a valid-but-unspecified state.

STL: When this is resolved, I want to see a) VBU for library types after self-move, but also b) requirements on user types for self-moves. E.g. should algorithms be required to avoid self-assignments (since a user-defined type might blow up)? HH: In other words, should we require that you can assign from moved-from values.

WEB: What can one generally do with moved-from values?

VV: Call any member function that has no preconditions.

JW: That's certainly the library requirement, and it's also good guidance for user types.

JW: I'm writing wording. I care about this.

Move to Open; Jonathan to provide wording

[2016-08-01, Howard provided wording]

[2016-08 Chicago]

Tuesday AM: Move to Tentatively Ready

Previous resolution [SUPERSEDED]:

In 17.6.3.2 [swappable.requirements], modify Table 23 — MoveAssignable requirements [moveassignable]:

Table 23 — MoveAssignable requirements [moveassignable]

Expression	Return type	Return value	Post-condition
<code>t = rv</code>	<code>T&</code>	<code>t</code>	<code>If <code>addressof(t) != addressof(rv)</code>, <code>t</code> is equivalent to the value of <code>rv</code> before the assignment</code> <code>rv's state is unspecified. [Note: <code>rv</code> must still meet the requirements of the library component that is using it, <code>whether or not <code>addressof(t) == addressof(rv)</code></code>. The operations listed in those requirements must work as specified whether <code>rv</code> has been moved from or not. — end note]</code>

[2016-08-07, Daniel reopens]

With the acceptance of LWG 2598, the proposed wording is invalid code, because it attempts to call `std::addressof` with an rvalue argument. It should be pointed out that the new restriction caused by 2598 doesn't affect real code, because any identity test within a move assignment operator (or any comparable function) would act on the current function argument, which is an lvalue in the context of the function body. The existing wording form of the issue could still be kept, if a helper variable would be introduced such as:

`Let refrv denote a reference initialized as if by const T& refrv = rv;. Then if addressof(t) != addressof(refrv), t is equivalent to the value of rv before the assignment`

But it seems to me that the same effect could be much easier realized by replacing the code form by a non-code English phrase that realizes the same effect.

[2016-09-09 Issues Resolution Telecon]

Move to Tentatively Ready

[2016-10-05, Tim Song comments]

The current P/R of LWG 2468 simply adds to MoveAssignable the requirement to tolerate self-move-assignment, but that doesn't actually do much about self-move-assignment of library types. Very few types in the library are explicitly required to satisfy MoveAssignable, so as written the restriction in 17.6.4.9 [res.on.arguments] would seem to still apply for any type that's not explicitly required to be CopyAssignable OR MoveAssignable.

The current P/R also doesn't address the issue with 23.2.1 [container.requirements.general] noted in the issue discussion.

Proposed resolution:

This wording is relative to N4606.

1. In 17.6.3.2 [swappable.requirements], modify Table 23 — MoveAssignable requirements [moveassignable]:

Table 23 — MoveAssignable requirements [moveassignable]

	Return	Return	
--	--------	--------	--

Expression	type	value	Post-condition
<code>t = rv</code>	<code>T&</code>	<code>t</code>	If <code>t</code> and <code>rv</code> do not refer to the same object, <code>t</code> is equivalent to the value of <code>rv</code> before the assignment
<code>rv</code> 's state is unspecified. [Note: <code>rv</code> must still meet the requirements of the library component that is using it, whether or not <code>t</code> and <code>rv</code> refer to the same object. The operations listed in those requirements must work as specified whether <code>rv</code> has been moved from or not. — end note]			

2475. Allow overwriting of `std::basic_string` terminator with `charT()` to allow cleaner interoperation with legacy APIs

Section: 21.3.1.5 [string.access] **Status:** [Tentatively Ready](#) **Submitter:** Matt Weber **Opened:** 2015-02-21 **Last modified:** 2016-08-08

Priority: 3

View all other [issues](#) in [string.access].

View all issues with [Tentatively Ready](#) status.

Discussion:

It is often desirable to use a `std::basic_string` object as a buffer when interoperating with libraries that mutate null-terminated arrays of characters. In many cases, these legacy APIs write a null terminator at the specified end of the provided buffer. Providing such a function with an appropriately-sized `std::basic_string` results in undefined behavior when the `charT` object at the `size()` position is overwritten, even if the value remains unchanged.

Absent the ability to allow for this, applications are forced into pessimizations such as: providing appropriately-sized `std::vector`s of `charT` for interoperating with the legacy API, and then copying the `std::vector` to a `std::basic_string`; providing an oversized `std::basic_string` object and then calling `resize()` later.

A trivial example:

```
#include <string>
#include <vector>

void legacy_function(char *out, size_t count) {
    for (size_t i = 0; i < count; ++i) {
        *out++ = '0' + (i % 10);
    }
    *out = '\0'; // if size() == count, this results in undefined behavior
}

int main() {
    std::string s(10, '\0');
    legacy_function(&s[0], s.size()); // undefined behavior

    std::vector<char> buffer(11);
    legacy_function(&buffer[0], buffer.size() - 1);
    std::string t(&buffer[0], buffer.size() - 1); // potentially expensive copy

    std::string u(11, '\0');
    legacy_function(&u[0], u.size() - 1);
    u.resize(u.size() - 1); // needlessly complicates the program's logic
}
```

A slight relaxation of the requirement on the returned object from the element access operator would allow for this interaction with no semantic change to existing programs.

[2016-08 Chicago]

Tues PM: This should also apply to non-const `data()`. Billy to update wording.

Fri PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4296.

- Edit 21.3.1.5 [string.access] as indicated:

```
const_reference operator[](size_type pos) const;
reference operator[](size_type pos);
```

-1- Requires: [...]

-2- Returns: `*(begin() + pos)` if `pos < size()`. Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object to any value other than `charT()` leads to undefined behavior.

[...]

2503. multiline option should be added to `syntax_option_type`

Section: 28.5.1 [re.synopt] **Status:** [Tentatively Ready](#) **Submitter:** Nozomu Katō **Opened:** 2015-05-22 **Last modified:** 2016-08-02

Priority: 2

View other [active issues](#) in [re.synopt].

View all other [issues](#) in [re.synopt].

View all issues with [Tentatively Ready](#) status.

Discussion:

The specification of ECMAScript defines the Multiline property for its RegExp and the regular expressions `^` and `$` behave differently according to the value of this property. Thus, this property should be available also in the ECMAScript compatible engine in `std::regex`.

[2015-05-22, Daniel comments]

This issue interacts somewhat with LWG [2343](#).

[Telecom 2015-07]

Set the priority to match LWG [2343](#).

[2016-08, Chicago]

Monday PM: Moved to Tentatively Ready. This also resolves [2343](#)

Proposed resolution:

This wording is relative to N4431.

1. Change 28.5.1 [re.synopt] as indicated:

```
namespace std::regex_constants {
    typedef T1 syntax_option_type;
    constexpr syntax_option_type icae = unspecified ;
    constexpr syntax_option_type nosubs = unspecified ;
    constexpr syntax_option_type optimize = unspecified ;
    constexpr syntax_option_type collate = unspecified ;
    constexpr syntax_option_type ECMAScript = unspecified ;
    constexpr syntax_option_type basic = unspecified ;
    constexpr syntax_option_type extended = unspecified ;
    constexpr syntax_option_type awk = unspecified ;
    constexpr syntax_option_type grep = unspecified ;
    constexpr syntax_option_type egrep = unspecified ;
    constexpr syntax_option_type multiline = unspecified ;
}
```

2. Change 28.5.2 [re.matchflag], Table 138 — "syntax_option_type effects" as indicated:

Table 138 — syntax_option_type effects

Element	Effect(s) if set
	...
<u>multiline</u>	<u>Specifies that <code>^</code> shall match the beginning of a line and <code>\$</code> shall match the end of a line, if the ECMAScript engine is selected.</u>
	...

2510. Tag types should not be `DefaultConstructible`

Section: 18.6 [support.dynamic], 20.2 [utility], 20.4.5 [pair.piecewise], 20.10.2 [memory.syn], 20.10.6 [allocator.tag], 30.4 [thread.mutex] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2015-06-13 **Last modified:** 2016-08-08

Priority: 2

View all other [issues](#) in [support.dynamic].

View all issues with [Tentatively Ready](#) status.

Discussion:

`std::experimental::optional`, for certain reasons, specifies its `nullopt` type to not be `DefaultConstructible`. It doesn't do so for its tag type `in_place_t` and neither does the standard proper for any of its tag types. That turns out to be very unfortunate, consider the following:

```
#include <memory>
#include <array>

void f(std::array<int, 1>, int) {} // #1
void f(std::allocator_arg_t, int) {} // #2

int main()
{
    f({}, 666); // #3
}
```

The call at #3 is ambiguous. What's even worse is that if the overload #1 is removed, the call works just fine. The whole point of a tag type is that it either needs to be mentioned in a call or it needs to be a forwarded argument, so being able to construct a tag type like that makes no sense.

Making the types have an *explicit default constructor* might have helped, but [CWG 1518](#) is going against that idea.

[optional.nullopt]/3 solves this problem for `nullopt`:

Type `nullopt_t` shall not have a default constructor. It shall be a literal type. Constant `nullopt` shall be initialized with an argument of literal type.

[2015-06, Telecom]

Move to Tentatively Ready.

[2015-10, Kona Saturday afternoon]

Move back to Open

JW: The linked Core issue (CWG 1518) gives us a better tool to solve this (explicit default constructors). [The CWG Issue means that an explicit default constructor will no longer match "{}".] JW explains that it's important that tag types cannot be constructed from "{}" (e.g. the allocator tag in the tuple constructors).

WEB: Should we now go back and update our constructors? JW: For tag types, yes.

VV: The guideline is that anything that does not mention the type name explicitly should not invoke an explicit constructor.

Ville will provide wording.

Discussion about pair/tuple's default constructor - should they now be explicit?

[2016-01-31]

Ville provides revised wording.

Previous resolution [SUPERSEDED]:

This wording is relative to N4527.

1. In 18.6 [support.dynamic]/1, change the header `<new>` synopsis:

```
[...]
struct nothrow_t ; see below
extern const nothrow_t nothrow;
[...]
```

2. Add a new paragraph after 18.6 [support.dynamic]/1 (*following* the header `<new>` synopsis):

-?- Type `nothrow_t` shall not have a default constructor.

3. In 20.2 [utility]/2, change the header <utility> synopsis:

```
[...]
// 20.3.5, pair piecewise construction
struct piecewise_construct_t {}; see below
constexpr piecewise_construct_t piecewise_construct{ unspecified };
[...]
```

4. Add a new paragraph after 20.2 [utility]/2 (following the header <utility> synopsis):

-?- Type `piecewise_construct_t` shall not have a default constructor. It shall be a literal type. Constant `piecewise_construct` shall be initialized with an argument of literal type.

5. In 20.4.5 [pair.piecewise], apply the following edits:

```
struct piecewise_construct_t {};
constexpr piecewise_construct_t piecewise_construct{ unspecified };
```

6. In 20.10.2 [memory.syn]/1, change the header <memory> synopsis:

```
[...]
// 20.7.6, allocator argument tag
struct allocator_arg_t {}; see below
constexpr allocator_arg_t allocator_arg{ unspecified };
[...]
```

7. Add a new paragraph after 20.10.2 [memory.syn]/1 (following the header <memory> synopsis):

-?- Type `allocator_arg_t` shall not have a default constructor. It shall be a literal type. Constant `allocator_arg` shall be initialized with an argument of literal type.

8. In 20.10.6 [allocator.tag], apply the following edits:

```
namespace std {
    struct allocator_arg_t {};
    constexpr allocator_arg_t allocator_arg{ unspecified };
}
```

Editorial drive-by: `piecewise_construct_t` is written, in 20.4.5 [pair.piecewise] like

```
struct piecewise_construct_t { };
constexpr piecewise_construct_t piecewise_construct{};
```

whereas other tag types such as `allocator_construct_t` are, in e.g. 20.10.6 [allocator.tag], written like

```
namespace std {
    struct allocator_arg_t { };
    constexpr allocator_arg_t allocator_arg{};
}
```

We should decide whether or not to write out the `std` namespace in such paragraphs. I would suggest not to write it out.

9. In 30.4 [thread.mutex]/1, change the header <mutex> synopsis:

```
[...]
struct defer_lock_t {}; see below
struct try_to_lock_t {}; see below
struct adopt_lock_t {}; see below

constexpr defer_lock_t defer_lock { unspecified };
constexpr try_to_lock_t try_to_lock { unspecified };
constexpr adopt_lock_t adopt_lock { unspecified };
[...]
```

10. Add three new paragraphs after [thread.mutex]/1 (following the header <mutex> synopsis):

-?- Type `defer_lock_t` shall not have a default constructor. It shall be a literal type. Constant `defer_lock` shall be initialized with an argument of literal type.

-?- Type `try to lock_t` shall not have a default constructor. It shall be a literal type. Constant

try_to_lock shall be initialized with an argument of literal type.

-?- Type adopt_lock_t shall not have a default constructor. It shall be a literal type. Constant adopt_lock shall be initialized with an argument of literal type.

[2016-03 Jacksonville]

AM: should have note about compatibility in Annex C

HH: like this idiom well enough that I've started using it in my own code

AM: why are pair and tuple involved here?

GR: they are the only types which forward explicitness with EXPLICIT

AM: British spelling of behaviour

AM: happy to drop my issue about Annex C

[2016-06 Oulu]

This is waiting on Core issue 1518

Saturday: Core 1518 was resolved in Oulu

[2016-07 Chicago]

This is related to [2736](#)

Monday PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4567.

1. In 18.6 [support.dynamic]/1, change the header <new> synopsis:

```
[...]  
struct nothrow_t { explicit nothrow t() = default; };  
extern const nothrow_t nothrow;  
[...]
```

2. In 20.2 [utility]/2, change the header <utility> synopsis:

```
[...]  
// 20.3.5, pair piecewise construction  
struct piecewise_construct_t { explicit piecewise_construct t() = default; };  
constexpr piecewise_construct_t piecewise_construct{};  
[...]
```

3. In 20.4.2 [pairs.pair], change the class template pair synopsis:

```
[...]  
pair(pair&&) = default;  
EXPLICIT constexpr pair();  
EXPLICIT constexpr pair(const T1& x, const T2& y);  
[...]
```

4. Around 20.4.2 [pairs.pair] p3, apply the following edits:

EXPLICIT constexpr pair();

-3- *Effects*: Value-initializes first and second.

-4- *Remarks*: This constructor shall not participate in overload resolution unless `is_default_constructible<first_type>::value` is true and `is_default_constructible<second_type>::value` is true. [Note: This behaviour can be implemented by a constructor template with default template arguments. — end note] The constructor is explicit if and only if either first type or second type is not implicitly default-constructible. [Note: This behaviour can be implemented with a trait that checks whether a const first type& or a const second type& can be initialized with {}. — end note]

5. In 20.4.5 [pair.piecewise], apply the following edits:

```
struct piecewise_construct_t { explicit piecewise_construct t() = default; };  
constexpr piecewise_construct_t piecewise_construct{};
```

6. In 20.5.2 [tuple.tuple], change the class template tuple synopsis:

```
[...]
// 20.4.2.1, tuple construction
EXPLICIT constexpr tuple();
EXPLICIT constexpr tuple(const Types&...); // only if sizeof...(Types) >= 1
[...]
```

7. Around 20.5.2.1 [tuple.cnstr] p4, apply the following edits:

```
EXPLICIT constexpr tuple();
```

-4- *Effects*: Value initializes each element.

-5- *Remarks*: This constructor shall not participate in overload resolution unless `is_default_constructible<Ti>::value` is true for all *i*. [Note: This behaviour can be implemented by a constructor template with default template arguments. — end note] The constructor is explicit if and only if *T_i* is not implicitly default-constructible for at least one *i*. [Note: This behaviour can be implemented with a trait that checks whether a `const Ti&` can be initialized with `{}`. — end note]

8. In 20.10.2 [memory.syn]/1, change the header <memory> synopsis:

```
[...]
// 20.7.6, allocator argument tag
struct allocator_arg_t { explicit allocator_arg_t() = default; };
constexpr allocator_arg_t allocator_arg{};
[...]
```

9. In 20.10.6 [allocator.tag], apply the following edits:

```
namespace std {
    struct allocator_arg_t { explicit allocator_arg_t() = default; };
    constexpr allocator_arg_t allocator_arg{};
}
```

Editorial drive-by: `piecewise_construct_t` is written, in 20.4.5 [pair.piecewise] like

```
struct piecewise_construct_t { };
constexpr piecewise_construct_t piecewise_construct{};
```

whereas other tag types such as `allocator_construct_t` are, in e.g. 20.10.6 [allocator.tag], written like

```
namespace std {
    struct allocator_arg_t { };
    constexpr allocator_arg_t allocator_arg{};
}
```

We should decide whether or not to write out the `std` namespace in such paragraphs. I would suggest not to write it out.

10. In 30.4 [thread.mutex]/1, change the header <mutex> synopsis:

```
[...]
struct defer_lock_t { explicit defer_lock_t() = default; };
struct try_to_lock_t { explicit try_to_lock_t() = default; };
struct adopt_lock_t { explicit adopt_lock_t() = default; };

constexpr defer_lock_t defer_lock { };
constexpr try_to_lock_t try_to_lock { };
constexpr adopt_lock_t adopt_lock { };
[...]
```

2514. Type traits must not be final

Section: 20.15.1 [meta.rqmts] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2015-07-03 **Last modified:** 2016-08-04

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

We should make it clear that all standard `UnaryTypeTraits`, `BinaryTypeTraits` and `TransformationTraits` are not final.

Otherwise it is not safe to use them as arguments to a template like this:

```
template<typename C1, typename C2>
struct conjunction
    : conditional_t<C1::value, C2, C1>
{ };
```

[2016-08-03 Chicago LWG]

Walter, Nevin, and Jason provide initial Proposed Resolution.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Change 20.15.1 [meta.rqmts] as indicated:

- 1- A *UnaryTypeTrait* describes a property of a type. It shall be a **non-final** class template [...]
- 2- A *BinaryTypeTrait* describes a relationship between two types. It shall be a **non-final** class template [...]
- 3- A *TransformationTrait* modifies a property of a type. It shall be a **non-final** class template [...]

[2016-08-04 Chicago LWG]

LWG discusses and expresses preference for a more general, Library-wide, resolution. Walter and Nevin provide a new Proposed Resolution consistent with such guidance.

[2016-08 - Chicago]

Thurs PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Add a new paragraph add the end of 17.6.5.11 [derivation] as indicated:

?- All types specified in the C++ standard library shall be non-final types unless otherwise specified.

2519. Iterator operator-= has gratuitous undefined behaviour

Section: 24.2.7 [random.access.iterators] **Status:** [Tentatively Ready](#) **Submitter:** Hubert Tong **Opened:** 2015-07-15 **Last modified:** 2016-08-02

Priority: 2

View all other [issues](#) in [random.access.iterators].

View all issues with [Tentatively Ready](#) status.

Discussion:

In subclause 24.2.7 [random.access.iterators], Table 110, the operational semantics for the expression "*r* -= *n*" are defined as

```
return r += -n;
```

Given a `difference_type` of a type `int` with range `[-32768, 32767]`, if the value of `n` is `-32768`, then the evaluation of `-n` causes undefined behaviour (Clause 5 [expr] paragraph 4).

The operational semantics may be changed such that the undefined behaviour is avoided.

Suggested wording:

Replace the operational semantics for "*r* -= *n*" with:

```
{
    difference_type m = n;
    if (m >= 0)
        while (m--)
```

```
--r;
else
    while (m++)
        ++r;
return r;
}
```

Jonathan Wakely:

I'm now convinced we *don't* want to change the definition of -= and instead we should explicitly state the (currently implicit) precondition that `n != numeric_limits<difference_type>::min()`.

[2016-08, Chicago]

Monday PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4527.

- 1. Change Table 110 "Random access iterator requirements (in addition to bidirectional iterator)" as indicated:

Table 110 — Random access iterator requirements (in addition to bidirectional iterator)			
Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
...			
<code>r -= n</code>	<code>X&</code>	<code>return r += -n;</code>	pre: the absolute value of <code>n</code> is in the range of representable values of <code>difference_type</code> .
...			

2531. future::get should explicitly state that the shared state is released

Section: 30.6.6 [futures.unique_future] **Status:** [Tentatively Ready](#) **Submitter:** Agustín K-ballo Bergé **Opened:** 2015-09-03 **Last modified:** 2016-08-04

Priority: 3

View other [active issues](#) in [futures.unique_future].

View all other [issues](#) in [futures.unique_future].

View all issues with [Tentatively Ready](#) status.

Discussion:

The standard is usually very explicit on when a shared state is released, except for `future::get` for which it only states `valid() == false` as a postcondition.

[2016-08 - Chicago]

Thurs AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4527.

- 1. Modify 30.6.6 [futures.unique_future] as indicated:

```
R future::get();
R& future<R&>::get();
void future<void>::get();
```

-14- *Note:* as described above, the template and its two required specializations differ only in the return type and return value of the member function `get`.

-15- *Effects:*

- `wait()`s until the shared state is ready, then retrieves the value stored in the shared state;
- releases any shared state (30.6.4 [futures.state]).

2534. Constrain rvalue stream operators

Section: 27.7.3.9 [ostream.rvalue], 27.7.2.6 [istream.rvalue] **Status:** [Tentatively Ready](#) **Submitter:** Robert Haberlach **Opened:** 2015-09-08 **Last modified:** 2016-08-04

Priority: 3

View other [active issues](#) in [ostream.rvalue].

View all other [issues](#) in [ostream.rvalue].

View all issues with [Tentatively Ready](#) status.

Discussion:

The rvalue stream insertion and extraction operators should be constrained to not participate in overload resolution unless the expression they evaluate is well-formed. Programming code that tests the validity of stream insertions (or extractions) using SFINAE can result in false positives, as the present declarations accept virtually any right-hand side argument. Moreover, there is no need for pollution of the candidate set with ill-formed specializations.

[2016-08 - Chicago]

Thurs AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4527.

1. Modify 27.7.3.9 [ostream.rvalue] as indicated:

```
template <class charT, class traits, class T>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>&& os, const T& x);
```

-1- *Effects:* os << x

-2- *Returns:* os

-?- *Remarks:* This function shall not participate in overload resolution unless the expression os << x is well-formed.

2. Modify 27.7.2.6 [istream.rvalue] as indicated:

```
template <class charT, class traits, class T>
    basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>&& is, T& x);
```

-1- *Effects:* is >> x

-2- *Returns:* is

-?- *Remarks:* This function shall not participate in overload resolution unless the expression is >> x is well-formed.

2536. What should <complex.h> do?

Section: D.4 [depr.c.headers] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2015-09-10 **Last modified:** 2016-09-11

Priority: 2

View all other [issues](#) in [depr.c.headers].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG issue [1134](#) removed the resolution of LWG [551](#), leaving an incorrect specification for the behavior of <complex.h>. This header is currently required to make std::complex (and associated functions) visible in the global namespace, but should not be so

required.

[2016-09-09 Issues Resolution Telecom]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4527.

- 1. Add a new paragraph before D.4 [depr.c.headers]/2:

-?- The header <complex.h> behaves as if it simply includes the header <ccomplex>.

- 2. Change in D.4 [depr.c.headers]/2:

-2- Every other C header, each of which has a name of the form name.h, behaves as if each name placed in the standard library namespace by the corresponding cname header is placed within the global namespace scope. It is unspecified whether these names are first declared or defined within namespace scope (3.3.6) of the namespace std and are then injected into the global namespace scope by explicit using-declarations (7.3.3).

2540. unordered_multimap::insert hint iterator

Section: 23.2.5 [unord.req] Status: Tentatively Ready Submitter: Isaac Hier Opened: 2015-09-16 Last modified: 2016-08-04

Priority: 3

View other active issues in [unord.req].

View all other issues in [unord.req].

View all issues with Tentatively Ready status.

Discussion:

I have been wondering about the C++ standard requirements regarding the hint iterator for insertion into an unordered_multimap (and I imagine a similar question could be asked of unordered_map, but I have not researched that topic). As far as I can tell, it seems perfectly valid for an implementation to allow only valid dereferenceable iterators to be used as the hint argument for this member function. If that is correct, it means that one could not expect the end iterator to be used as a valid hint nor could one use the begin iterator of an empty unordered_multimap as the hint. However, this essentially precludes all uses of inserter on an empty unordered_multimap seeing as the inserter functor requires that a hint iterator be passed to its constructor.

Howard Hinnant:

The intent of the standard is that the iterator produced from container c by c.end() is a valid (but non-dereferenceable) iterator into container c. It is reachable by every other iterator into c.

It appears to me that you and the Bloomberg implementation have fallen victim to a type-o in the Unordered associative container requirements, Table 102. The row containing:

```
a.insert(q, t);
```

should read instead:

```
a.insert(p, t);
```

The distinction is that p is valid, and q is both valid and dereferenceable. The correction of this type-o would make unordered container insert consistent with unordered emplace_hint, associative insert, and associative emplace_hint.

[2016-08 - Chicago]

Thurs AM: Moved to Tentatively Ready

Proposed resolution:

Change the insert-with-hint row in Table 102 Unordered associative container requirements like so:

		Requires: If t is a non-const	Average Case
--	--	-------------------------------	--------------

a.insert(pp, t); iterator ...	...
-------------------------------	-----

2543. LWG 2148 (hash support for enum types) seems under-specified

Section: 20.14.14 [unord.hash] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2015-09-27 **Last modified:** 2016-09-11

Priority: 2

View other [active issues](#) in [unord.hash].

View all other [issues](#) in [unord.hash].

View all issues with [Tentatively Ready](#) status.

Discussion:

The rationale in issue [2148](#) says:

This proposed resolution doesn't specify anything else about the primary template, allowing implementations to do whatever they want for non-enums: `static_assert` nicely, explode horribly at compiletime or runtime, etc.

libc++ seems to implement it by defining the primary template and `static_asserting is_enum` inside it. However, that brings forth a problem; there are reasonable SFINAE uses broken by it:

```
#include <type_traits>
#include <functional>

class S{}; // No hash specialization

template<class T>
auto f(int) -> decltype(std::hash<T>(), std::true_type());

template<class T>
auto f(...) -> decltype(std::false_type());

static_assert(!decltype(f<S>(0))::value, "");
```

MSVC doesn't seem to accept that code either.

There is a way to implement LWG [2148](#) so that `hash` for enums is supported without breaking that sort of SFINAE uses:

1. Derive the main `hash` template from a library-internal uglified-named base template that takes a type and a `bool`, pass as argument for the base the result of `is_enum`.
2. Partially specialize that base template so that the false-case has a suitable set of private special member function declarations so that it's not an aggregate nor usable in almost any expression.

[2015-10, Kona Saturday afternoon]

EricWF to come back with wording; move to Open

[2016-05-08, Ericniebler & Ville provide wording]

[2016-05-25, Tim Song comments]

I see two issues with this P/R:

1. "for which neither the library nor the user provides an explicit specialization" should probably be "for which neither the library nor the user provides an explicit or partial specialization".
2. Saying that the specialization "is not `DefaultConstructible` nor `MoveAssignable`" is not enough to guarantee that common SFINAE uses will work. Both of those requirements have several parts, and it's not too hard to fail only some of them. For instance, not meeting the assignment postcondition breaks `MoveAssignable`, but is usually not SFINAE-detectable. And for `DefaultConstructible`, it's easy to write something in a way that breaks `τ()` but not `τ{}` (due to aggregate initialization in the latter case).

[2016-06-14, Daniel comments]

The problematic part of the P/R is that it describes constraints that would be suitable if they were constraints for user-code, but they are not suitable as requirements imposed on implementations to provide certain guarantees for clients of the Library. The guarantees should be written in terms of testable compile-time expressions, e.g. based on negative results of `is_default_constructible<hash<X>>::value`, `std::is_copy_constructible<hash<X>>::value`, and possibly also `std::is_destructible<hash<X>>::value`. *How* an implementation realizes these negative results shouldn't be specified, though, but the expressions need to be well-formed and well-defined.

[2016-08-03, Ville provides revised wording as response to Daniel's previous comment]

Previous resolution [SUPERSEDED]:

This wording is relative to N4582.

1. Insert a new paragraph after 20.14.14 [unord.hash]/2

-2- The template specializations shall meet the requirements of class template `hash` (20.12.14).

-?- For any type that is not of integral or enumeration type, or for which neither the library nor the user provides an explicit specialization of the class template `hash`, the specialization of `hash` does not meet any of the `Hash` requirements, and is not `DefaultConstructible` nor `MoveAssignable`. [Note: this means that the specialization of `hash` exists, but any attempts to use it as a `Hash` will be ill-formed. — end note]

[2016-08 - Chicago]

Thurs AM: Moved to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Insert a new paragraph after 20.14.14 [unord.hash]/2

[Drafting note: I see no reason to specify whether `H<T>` is destructible. There's no practical use case for which that would need to be covered. `libstdc++` makes it so that `H<T>` is destructible.]

-2- The template specializations shall meet the requirements of class template `hash` (20.12.14).

-?- For any type τ that is not of integral or enumeration type, or for which neither the library nor the user provides an explicit or partial specialization of the class template `hash`, the specialization of `hash<T>` has the following properties:

- `is_default_constructible v<hash<T>>` is false
- `is_copy_constructible v<hash<T>>` is false
- `is_move_constructible v<hash<T>>` is false
- `is_copy_assignable v<hash<T>>` is false
- `is_move_assignable v<hash<T>>` is false
- `is_callable v<hash<T>, T&>` is false
- `is_callable v<hash<T>, const T&>` is false

[Note: this means that the specialization of `hash` exists, but any attempts to use it as a `Hash` will be ill-formed. — end note]

[2016-08-09 Daniel reopens]

As pointed out by Eric, the usage of `is_callable` is incorrect. Eric provides new wording.

[2016-09-09 Issues Resolution Telecom]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Insert a new paragraph after 20.14.14 [unord.hash]/2

[Drafting note: I see no reason to specify whether $H<T>$ is destructible. There's no practical use case for which that would need to be covered. libstdc++ makes it so that $H<T>$ is destructible.]

-2- The template specializations shall meet the requirements of class template hash (20.12.14).

-? For any type τ that is not of integral or enumeration type, or for which neither the library nor the user provides an explicit or partial specialization of the class template hash, the specialization of $hash<T>$ has the following properties:

- is_default_constructible $v<hash<T>>$ is false
- is_copy_constructible $v<hash<T>>$ is false
- is_move_constructible $v<hash<T>>$ is false
- is_copy_assignable $v<hash<T>>$ is false
- is_move_assignable $v<hash<T>>$ is false
- $hash<T>$ is not a function object type (20.14 [function.objects])

[Note: this means that the specialization of hash exists, but any attempts to use it as a Hash will be ill-formed. — end note]

2544. istreambuf_iterator(basic_streambuf<charT, traits>* s) effects unclear when s is 0

Section: 24.6.3.2 [istreambuf.iterator.cons] **Status:** [Tentatively Ready](#) **Submitter:** S. B. Tam **Opened:** 2015-10-05 **Last modified:** 2016-08-08

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

N4527 24.6.3.2 [istreambuf.iterator.cons] does not mention what the effect of calling `istreambuf_iterator(basic_streambuf<charT, traits>* s)` is when `s` is a null pointer. It should be made clear that this case is well-formed and the result is a end-of-stream iterator.

Daniel:

According to 24.6.3 [istreambuf.iterator] p1:

[...] The default constructor `istreambuf_iterator()` and the constructor `istreambuf_iterator(0)` both construct an end-of-stream iterator object suitable for use as an end-of-range. [...]

This indicates that the described constructor creates an end-of-stream iterator, but this wording is part of the introductory wording and I recommend to make 24.6.3.2 [istreambuf.iterator.cons] clearer, because the existing specification is already flawed, e.g. it never specifies when and how the exposition-only-member `sbuf_` is initialized. The proposed wording below attempts to solve these problems as well.

Previous resolution [SUPERSEDED]:

This wording is relative to N4527.

1. Change 24.6.3.2 [istreambuf.iterator.cons] as indicated:

[Editorial note: The proposed wording changes also performs some editorial clean-up of the existing mismatches of the declarations in the class template synopsis and the individual member specifications. The below wording intentionally does not say anything about the concrete value of `sbuf_` for end-of-stream iterator values, because that was never specified before; in theory, this could be some magic non-null pointer that can be used in constant expressions. But the wording could be drastically simplified by requiring `sbuf_` to be a null pointer for an end-of-stream iterator value, since I have not yet seen any implementation where this requirement does not hold. — end editorial note]

```
constexpr istreambuf_iterator() noexcept;
```

-1- *Effects:* Constructs the end-of-stream iterator.

```
istreambuf_iterator(basic_istream<charT,traits>istream_type& s) noexcept;  
istreambuf_iterator(basic_streambuf<charT,traits>* s) noexcept;
```

-2- *Effects:* If `s.rdbuf()` is a null pointer, constructs an end-of-stream iterator; otherwise initializes `sbuf` with `s.rdbuf()` and constructs an `istreambuf_iterator` that uses the `streambuf` type object `*sbuf`. ~~Constructs an `istreambuf_iterator` that uses the `basic_streambuf` object `*(s.rdbuf())`, or `*s`, respectively. Constructs an end-of-stream iterator if `s.rdbuf()` is null.~~

```
istreambuf_iterator(streambuf_type* s) noexcept;
```

~~-?- *Effects:* If `s` is a null pointer, constructs an end-of-stream iterator; otherwise initializes `sbuf` with `s` and constructs an `istreambuf_iterator` that uses the `streambuf` type object `*sbuf`.~~

```
istreambuf_iterator(const proxy& p) noexcept;
```

-3- *Effects:* Initializes `sbuf` with `p.sbuf` and constructs an `istreambuf_iterator` that uses the `streambuf` type object `*sbuf`. ~~Constructs a `istreambuf_iterator` that uses the `basic_streambuf` object pointed to by the proxy object's constructor argument `p`.~~

[2015-10-20, Daniel provides alternative wording]

[2016-08-03 Chicago]

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 24.6.3.2 [istreambuf.iterator.cons] as indicated:

[Drafting note: The proposed wording changes also performs some editorial clean-up of the existing mismatches of the declarations in the class template synopsis and the individual member specifications. The below wording is simplified by requiring `sbuf_` to be a null pointer for an end-of-stream iterator value, since I have not yet seen any implementation where this requirement does not hold. Even if there *were* such an implementation, this would still be conforming, because concrete exposition-only member values are not part of public API. — end drafting note]

For each `istreambuf_iterator` constructor in this section, an end-of-stream iterator is constructed if and only if the exposition-only member `sbuf_` is initialized with a null pointer value.

```
constexpr istreambuf_iterator() noexcept;
```

-1- *Effects:* Initializes `sbuf_` with `nullptr`. ~~Constructs the end-of-stream iterator.~~

```
istreambuf_iterator(basic_istream<charT,traits>istream_type& s) noexcept;  
istreambuf_iterator(basic_streambuf<charT,traits>* s) noexcept;
```

-2- *Effects:* Initializes `sbuf_` with `s.rdbuf()`. ~~Constructs an `istreambuf_iterator` that uses the `basic_streambuf` object `*(s.rdbuf())`, or `*s`, respectively. Constructs an end-of-stream iterator if `s.rdbuf()` is null.~~

```
istreambuf_iterator(streambuf_type* s) noexcept;
```

-?- *Effects:* Initializes `sbuf_` with `s`.

```
istreambuf_iterator(const proxy& p) noexcept;
```

-3- *Effects:* Initializes `sbuf_` with `p.sbuf`. ~~Constructs a `istreambuf_iterator` that uses the `basic_streambuf` object pointed to by the proxy object's constructor argument `p`.~~

2556. Wide contract for future::share()

Section: 30.6.6 [futures.unique_future] **Status:** [Tentatively Ready](#) **Submitter:** Agustín K-ballo Bergé **Opened:** 2015-11-05 **Last modified:** 2016-08-06

Priority: 3

View other [active issues](#) in [futures.unique_future].

View all other [issues](#) in [futures.unique_future].

View all issues with [Tentatively Ready](#) status.

Discussion:

`future::share()` is not `noexcept`, it has a narrow contract requiring `valid()` as per the blanket wording in 30.6.6 [futures.unique_future] p3. Its effects, however, are `return shared_future<R>(std::move(*this))`, which is `noexcept` as it has a wide contract. If the source future isn't valid then the target `shared_future` simply won't be valid either. There appears to be no technical reason preventing `future::share()` from having a wide contract, and thus being `noexcept`.

[2016-08-03 Chicago]

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4567.

1. Change 30.6.6 [futures.unique_future] as indicated:

-3- The effect of calling any member function other than the destructor, the move-assignment operator, `share`, or `valid` on a future object for which `valid() == false` is undefined. [Note: Implementations are encouraged to detect this case and throw an object of type `future_error` with an error condition of `future_errc::no_state`. — end note]

```
namespace std {
    template <class R>
    class future {
    public:
        [...]
        shared_future<R> share() noexcept;
        [...]
    };
}
```

[...]

```
shared_future<R> share() noexcept;
```

-12- *Returns*: `shared_future<R>(std::move(*this))`.

-13- *Postcondition*: `valid() == false`.

[...]

2562. Consistent total ordering of pointers by comparison functors

Section: 20.14.6 [comparisons] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2015-11-18 **Last modified:** 2016-08-05

Priority: 3

View other [active issues](#) in [comparisons].

View all other [issues](#) in [comparisons].

View all issues with [Tentatively Ready](#) status.

Discussion:

N4567 20.14.6 [comparisons]/14 specifies that the comparison functors provide a total ordering for pointer types:

For templates `greater`, `less`, `greater_equal`, and `less_equal`, the specializations for any pointer type yield a total order, even if the built-in operators `<`, `>`, `<=`, `>=` do not.

It notably does not specify:

- whether the specializations of all of the named templates for a given pointer type yield the *same* total order

- whether the total order imposed respects the partial order imposed by the built-in operators
- whether the total order imposed is consistent with the partition induced by ==

All of which are important for sane semantics and provided by common implementations, since the built-in operators provide a total order and the comparison functors yield that same order.

It would be extremely confusing — if not outright insane — for e.g.:

- `less<int*>` and `greater<int*>` to yield different orders
- `less<int*>` to disagree with `<` on the relative order of two pointers for which `<` is defined
- `less<int*>` to order `a` before `b` when `a == b`, i.e., not preserve equality.

Consistent semantics for the various comparison functors and the built-in operators is so intuitive as to be assumed by most programmers.

Related issues: [2450](#), [2547](#).

Previous resolution [SUPERSEDED]:

This wording is relative to N4567.

1. Alter 20.14.6 [comparisons]/14 to read:

For templates `greater`, `less`, `greater_equal`, and `less_equal`, the specializations for any pointer type yield **at the same** total order, even if the built-in operators `<`, `>`, `<=`, `>=` do not. The total order shall respect the partial order imposed by the built-in operators.

[2016-05-20, Casey Carter comments and suggests revised wording]

The new proposed wording is attempting to address the issue raised in the 2016-02-04 telecon.

The real issue I'm trying to address here is ensure that "weird" implementations provide the same kind of consistency for pointer orderings as "normal" implementations that use a flat address spaces and have totally ordered `<`. If `a < b` is true for `int` pointers `a` and `b`, then `less<int*>(a, b)`, `less_equal<int*>(a, b)`, `less<char*>(a, b)`, `less<void*>(a, b)`, and `greater<int*>(b, a)` should all hold. I think this wording is sufficient to provide that.

Previous resolution [SUPERSEDED]:

This wording is relative to N4582.

1. Alter 20.14.6 [comparisons] to read:

~~-14-~~ For templates `greater`, `less`, `greater_equal`, and `less_equal`, the specializations for any pointer type yield **at the same** total order. That total order is consistent with the partial order imposed by, even if the built-in operators `<`, `>`, `<=`, and `>` ~~do not~~. [Note: When `a < b` is well-defined for pointers `a` and `b` of type `P`, this implies `(a < b) == less<P>(a, b)`, `(a > b) == greater<P>(a, b)`, and so forth. — end note] For template specializations `greater<void>`, `less<void>`, `greater_equal<void>`, and `less_equal<void>`, if the call operator calls a built-in operator comparing pointers, the call operator yields a total order.

[2016-08-04 Chicago LWG]

LWG discusses and concludes that we are trying to accomplish the following:

1.

```
T* a = /* ... */;
T* b = /* ... */;
```

if `a < b` is valid, `a < b == less<T*>(a, b)`, and analogously for `>`, `<=`, `>=`.

2.

```
less<void>(a, b) == less<T*>(a, b);
less<T*>(a, b) == greater<T*>(b, a);
```

etc.

3. `less<T*>` produces a strict total ordering with which the other three function objects are consistent

4. `less<void>` when applied to pointers produces a strict total ordering with which the other three are consistent
5. `less<void>` when applied to pointers of the same type produces the same strict total ordering as `less<T*>`, and analogously for the other three
6. we are not addressing `less<void>` (and the other three) when applied to pointers of differing types

Walter and Nevin revise Proposed Wording accordingly.

[2016-08 - Chicago]

Thurs PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 20.14.6 [comparisons] p14 as indicated:

-14- For templates ~~greater, less, greater_equal, and less_equal~~ `less, greater, less_equal, and greater_equal`, the specializations for any pointer type yield a strict total order that is consistent among those specializations and is also consistent with the partial order imposed by, ~~even if~~ the built-in operators `<`, `>`, `<=`, `>=` ~~do not~~. [Note: When `a < b` is well-defined for pointers `a` and `b` of type `P`, this implies `(a < b) == less<P>(a, b)`, `(a > b) == greater<P>(a, b)`, and so forth. — end note] For template specializations ~~greater<void>, less<void>, greater_equal<void>, and less_equal<void>~~ `less<void>, greater<void>, less_equal<void>, and greater_equal<void>`, if the call operator calls a built-in operator comparing pointers, the call operator yields a strict total order that is consistent among those specializations and is also consistent with the partial order imposed by those built-in operators.

2567. Specification of logical operator traits uses `BaseCharacteristic`, which is defined only for `UnaryTypeTraits` and `BinaryTypeTraits`

Section: 20.15.8 [meta.logical] Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2015-12-10 Last modified: 2016-08-08

Priority: 2

View other [active issues](#) in [meta.logical].

View all other [issues](#) in [meta.logical].

View all issues with [Tentatively Ready](#) status.

Discussion:

The specification of conjunction and disjunction uses the term `BaseCharacteristic`, which is problematic in several ways:

- That term is defined in 20.15.1 [meta.rqmts], but only for `UnaryTypeTraits` and `BinaryTypeTraits`. conjunction and disjunction seem to be neither.
- 20.15.1 [meta.rqmts] also requires the `BaseCharacteristic` for both `UnaryTypeTraits` and `BinaryTypeTraits` to be a specialization of `integral_constant`, which is inconsistent with the current design of conjunction and disjunction.
- The requirement in 20.15.1 [meta.rqmts] that "member names of the `BaseCharacteristic` shall not be hidden and shall be unambiguously available" seems impossible to meet in every case, since the arbitrary base class from which a specialization of conjunction or disjunction derives may contain members called `conjunction` or `disjunction` that will necessarily be hidden.

[2016-08 Chicago]

Ville provided wording for both [2567](#) and [2568](#)

Previous resolution [SUPERSEDED]:

In [meta.logical]/3, edit as follows:

The ~~BaseCharacteristic of a~~ specialization `conjunction<B1, ..., BN>` has a public and unambiguous base that is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value == false`, or if every `Bi::value != false`, the aforementioned base ~~BaseCharacteristic~~ is the last type in the list. [Note: This means a specialization of conjunction does not necessarily ~~have a BaseCharacteristic of~~ derive from either `true_type` or `false_type`. — end note]

In [meta.logical]/6, edit as follows:

The ~~BaseCharacteristic of a~~ specialization `disjunction<B1, ..., BN>` has a public and unambiguous base that is the first type `Bi` in the list `false_type, B1, ..., BN` for which `Bi::value != false`, or if every `Bi::value == false`, the ~~aforementioned base~~ BaseCharacteristic is the last type in the list. [Note: This means a specialization of `disjunction` does not necessarily ~~have a BaseCharacteristic of~~ derive from either `true_type` or `false_type`. — end note]

Previous resolution [SUPERSEDED]:

In [meta.logical]/3, edit as follows:

The ~~BaseCharacteristic of a~~ specialization `conjunction<B1, ..., BN>` has a public and unambiguous base that is either
* the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value == false`, or
* if there is no such `Bi`, the last type in the list.
~~is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value == false`, or if every `Bi::value != false`, the~~
~~BaseCharacteristic is the last type in the list.~~
[Note: This means a specialization of `conjunction` does not necessarily ~~have a BaseCharacteristic of~~ derive from either `true_type` or `false_type`. — end note]

In [meta.logical]/6, edit as follows:

The ~~BaseCharacteristic of a~~ specialization `disjunction<B1, ..., BN>` has a public and unambiguous base that is either
* the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value != false`, or
* if there is no such `Bi`, the last type in the list.
~~is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value != false`, or if every `Bi::value == false`, the~~
~~BaseCharacteristic is the last type in the list.~~
[Note: This means a specialization of `disjunction` does not necessarily ~~have a BaseCharacteristic of~~ derive from either `true_type` or `false_type`. — end note]

Merged the resolution of [2587](#) with this issue. This proposed resolution resolves both, and includes fixes from Daniel for negation. Last review of this with LWG turned up a `true_type` typo in the definition of `disjunction`, and some editorial changes.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. In 20.15.8 [meta.logical] p3, edit as follows:

```
template<class... B> struct conjunction : see below { };
```

-3- The ~~BaseCharacteristic of a~~ specialization `conjunction<B1, ..., BN>` has a public and unambiguous base that is either

- the first type `Bi` in the list `true_type, B1, ..., BN` for which `bool(Bi::value)` is false, or
- if there is no such `Bi`, the last type in the list.

~~is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value == false`, or if every `Bi::value != false`, the~~
~~BaseCharacteristic is the last type in the list.~~

-?- The member names of the base class, other than `conjunction` and `operator=`, shall not be hidden and shall be unambiguously available in `conjunction`. [Note: This means a specialization of `conjunction` does not necessarily ~~have a BaseCharacteristic of~~ inherit from either `true_type` or `false_type`. —end note]

2. In 20.15.8 [meta.logical] p6, edit as follows:

```
template<class... B> struct disjunction : see below { };
```

-6- The ~~BaseCharacteristic of a~~ specialization `disjunction<B1, ..., BN>` has a public and unambiguous base that is either

- the first type `Bi` in the list `true_type, B1, ..., BN` for which `bool(Bi::value)` is true, or,
- if there is no such `Bi`, the last type in the list.

~~is the first type `Bi` in the list `true_type, B1, ..., BN` for which `Bi::value != false`, or if every `Bi::value == false`, the~~
~~BaseCharacteristic is the last type in the list.~~

-?- The member names of the base class, other than `disjunction` and `operator=`, shall not be hidden

and shall be unambiguously available in disjunction. [Note: This means a specialization of disjunction does not necessarily have a BaseCharacteristic of inherit from either true type or false type. —end note]

3. In 20.15.8 [meta.logical] p8, edit as follows

```
template<class B> struct negation : bool_constant<!bool(B::value)> { };
```

-8- The class template negation forms the logical negation of its template type argument. The type negation is a UnaryTypeTrait with a BaseCharacteristic of bool_constant<!bool(B::value)>.

[2016-08-03 Chicago]

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. In 20.15.8 [meta.logical] p3, edit as follows:

```
template<class... B> struct conjunction : see below { };
```

[...]

-3- The BaseCharacteristic of a specialization conjunction<B1, ..., BN> has a public and unambiguous base that is either

- the first type Bi in the list true_type, B1, ..., BN for which bool(Bi::value) is false, or
- if there is no such Bi, the last type in the list.

is the first type Bi in the list true_type, B1, ..., BN for which Bi::value == false, or if every Bi::value != false, the BaseCharacteristic is the last type in the list. [Note: This means a specialization of conjunction does not necessarily have a BaseCharacteristic of inherit from either true_type or false_type. —end note]

-?- The member names of the base class, other than conjunction and operator=, shall not be hidden and shall be unambiguously available in conjunction.

2. In 20.15.8 [meta.logical] p6, edit as follows:

```
template<class... B> struct disjunction : see below { };
```

[...]

-6- The BaseCharacteristic of a specialization disjunction<B1, ..., BN> has a public and unambiguous base that is either

- the first type Bi in the list false_type, B1, ..., BN for which bool(Bi::value) is true, or
- if there is no such Bi, the last type in the list.

is the first type Bi in the list false_type, B1, ..., BN for which Bi::value != false, or if every Bi::value == false, the BaseCharacteristic is the last type in the list. [Note: This means a specialization of disjunction does not necessarily have a BaseCharacteristic of inherit from either true_type or false_type. —end note]

-?- The member names of the base class, other than disjunction and operator=, shall not be hidden and shall be unambiguously available in disjunction.

3. In 20.15.8 [meta.logical] p8, edit as follows

```
template<class B> struct negation : see below bool_constant<!B::value> { };
```

-8- The class template negation forms the logical negation of its template type argument. The type negation is a UnaryTypeTrait with a BaseCharacteristic of bool_constant<!bool(B::value)>.

2569. conjunction and disjunction requirements are too strict

Section: 20.15.8 [meta.logical] Status: [Tentatively Ready](#) Submitter: Tim Song Opened: 2015-12-11 Last modified: 2016-08-08

Priority: 2

View other [active issues](#) in [meta.logical].

View all other [issues](#) in [meta.logical].

View all issues with [Tentatively Ready](#) status.

Discussion:

20.15.8 [meta.logical]/2 and /5 impose the following requirement on the arguments of conjunction and disjunction:

Every template type argument shall be usable as a base class and shall have a static data member value which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

Since the requirement is unconditional, it applies even to type arguments whose instantiation is not required due to short circuiting. This seems contrary to the design intent, expressed in [P0013R1](#), that it is valid to write `conjunction_v<is_class<T>, is_foo<T>>` even if instantiating `is_foo<T>::value` is ill-formed for non-class types.

[2016-08 Chicago]

Ville provided wording for both [2569](#) and [2570](#).

Tuesday AM: Move to Tentatively Ready

Proposed resolution:

[We recommend applying the proposed resolution for LWG issues 2567 and 2568 before this proposed resolution, lest the poor editor gets confused.]

In [meta.logical],

- insert a new paragraph before paragraph 2:

The class template `conjunction` forms the logical conjunction of its template type arguments.

- move paragraph 4 before paragraph 2, and edit paragraph 2 as follows:

~~The class template `conjunction` forms the logical conjunction of its template type arguments.~~ Every template type argument **for which `Bi::value` is instantiated** shall be usable as a base class and shall have a member value which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

- insert a new paragraph before paragraph 5:

The class template `disjunction` forms the logical disjunction of its template type arguments.

- move paragraph 7 before paragraph 5, and edit paragraph 5 as follows:

~~The class template `disjunction` forms the logical disjunction of its template type arguments.~~ Every template type argument **for which `Bi::value` is instantiated** shall be usable as a base class and shall have a member value which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

[2570](#). [fund.ts.v2] conjunction and disjunction requirements are too strict

Section: 99 [fund.ts.v2::meta.logical] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2015-12-11 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [fund.ts.v2::meta.logical].

View all other [issues](#) in [fund.ts.v2::meta.logical].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

99 [meta.logical]/2 and /5 impose the following requirement on the arguments of conjunction and disjunction:

Every template type argument shall be usable as a base class and shall have a static data member value which is convertible to `bool`, is not hidden, and is unambiguously available in the type.

Since the requirement is unconditional, it applies even to type arguments whose instantiation is not required due to short circuiting. This seems contrary to the design intent, expressed in [P0013R1](#), that it is valid to write `conjunction_v<is_class<T>, is_foo<T>>` even if instantiating `is_foo<T>::value` is ill-formed for non-class types.

[2016-06 Oulu]

Alisdair has a paper in progress addressing this

[2016-08 Chicago]

Ville provided wording for both [2569](#) and [2570](#).

Tuesday AM: Move to Tentatively Ready

Proposed resolution:

[2578](#). Iterator requirements should reference iterator traits

Section: 24.2 [iterator.requirements], 24.4.1 [iterator.traits] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2016-01-05 **Last modified:** 2016-08-06

Priority: 3

View other [active issues](#) in [iterator.requirements].

View all other [issues](#) in [iterator.requirements].

View all issues with [Tentatively Ready](#) status.

Discussion:

See [this reflector discussion](#) for background.

24.2 [iterator.requirements] attempts to establish requirements for iterators, but 24.4.1 [iterator.traits]/1 establishes further requirements that must be met in order to author a portable iterator that works with existing implementations. Failing to meet the requirements of the latter will fail to work in practice. The former requirements should reference the latter, normatively.

[2016-08-03 Chicago]

Fri AM: Moved to Tentatively Ready

Proposed resolution:

After [iterator.requirements.general]/5, insert the following new paragraph:

[-? In addition to the requirements in this sub-clause, the nested *typedef-names* specified in ([iterator.traits]) shall be provided for the iterator type. [Note: Either the iterator type must provide the *typedef-names* directly (in which case iterator traits pick them up automatically), or an iterator traits specialization must provide them. -end note]

[2584](#). `<regex>` ECMAScript IdentityEscape is ambiguous

Section: 28.13 [re.grammar] **Status:** [Tentatively Ready](#) **Submitter:** Billy O'Neal III **Opened:** 2016-01-13 **Last modified:** 2016-08-02

Priority: 2

View other [active issues](#) in [re.grammar].

View all other [issues](#) in [re.grammar].

View all issues with [Tentatively Ready](#) status.

Discussion:

Stephan and I are seeing differences in implementation for how non-special characters should be handled in the IdentityEscape part of the ECMAScript grammar. For example:

```
#include <stdio.h>
#include <iostream>
#ifdef USE_BOOST
#include <boost/regex.hpp>
```

```

using namespace boost;
#else
#include <regex>
#endif
using namespace std;

int main() {
    try {
        const regex r("\\z");
        cout << "Constructed \\z." << endl;
        if (regex_match("z", r))
            cout << "Matches z" << endl;
    } catch (const regex_error& e) {
        cout << e.what() << endl;
    }
}

```

libstdc++, boost, and browsers I tested with (Microsoft Edge, Google Chrome) all happily interpret `\z`, which otherwise has no meaning, as an identity character escape for the letter z. libc++ and msvc++ say that this is invalid, and throw `regex_error` with `error_escape`.

ECMAScript 3 (which is what C++ currently points to) seems to agree with libc++ and msvc++:

```

IdentityEscape ::
    SourceCharacter but not IdentifierPart

IdentifierPart ::
    IdentifierStart
    UnicodeCombiningMark
    UnicodeDigit
    UnicodeConnectorPunctuation
    \ UnicodeEscapeSequence

IdentifierStart ::
    UnicodeLetter
    $
    _
    \ UnicodeEscapeSequence

```

But this doesn't make any sense — it prohibits things like `\$` which users absolutely need to be able to escape. So let's look at ECMAScript 6. I believe this says much the same thing, but updates the spec to better handle Unicode by referencing what the Unicode standard says is an identifier character:

```

IdentityEscape ::
    SyntaxCharacter
    /
    SourceCharacter but not UnicodeIDContinue

UnicodeIDContinue ::
    any Unicode code point with the Unicode property "ID_Continue", "Other_ID_Continue", or "Other_ID_Start"

```

However, ECMAScript 6 has an appendix B defining "additional features for web browsers" which says:

```

IdentityEscape ::
    SourceCharacter but not c

```

which appears to agree with what libstdc++, boost, and browsers are doing.

What should be the correct behavior here?

[2016-08, Chicago]

Monday PM: Move to tentatively ready

Proposed resolution:

This wording is relative to N4567.

1. Change 28.13 [re.grammar]/3 as indicated:

-3- The following productions within the ECMAScript grammar are modified as follows:

```

ClassAtom ::
    -
    ClassAtomNoDash
    ClassAtomExClass
    ClassAtomCollatingElement

```

IdentityEscape ::
SourceCharacter but not c

2589. match_results can't satisfy the requirements of a container

Section: 28.10 [re.results] **Status:** [Tentatively Ready](#) **Submitter:** S. B. Tam **Opened:** 2016-01-24 **Last modified:** 2016-08-04

Priority: 3

View other [active issues](#) in [re.results].

View all other [issues](#) in [re.results].

View all issues with [Tentatively Ready](#) status.

Discussion:

N4567 28.10 [re.results] p2 mentions

The class template `match_results` shall satisfy the requirements of an allocator-aware container and of a sequence container, as specified in 23.2.3, except that only operations defined for const-qualified sequence containers are supported.

However, this is impossible because `match_results` has a `operator==` whose semantics differs from the one required in Table 95 — "Container requirements".

Table 95 requires that `a == b` is an equivalence relation and means `equal(a.begin(), a.end(), b.begin(), b.end())`. But for `match_results`, `a == b` and `equal(a.begin(), a.end(), b.begin(), b.end())` can give different results. For example:

```
#include <iostream>
#include <regex>
#include <string>
#include <algorithm>

int main()
{
    std::regex re("a*");
    std::string target("baaab");
    std::smatch a;

    std::regex_search(target, a, re);

    std::string target2("raaaau");
    std::smatch b;

    std::regex_search(target2, b, re);

    std::cout << std::boolalpha;
    std::cout << (a == b) << '\n'; // false
    std::cout << std::equal(a.begin(), a.end(), b.begin(), b.end()) << '\n'; // true
}
```

[2016-02, Issues Telecon]

Marshall: The submitter is absolutely right, but the proposed resolution is insufficient. We should avoid "shall", for once.

Jonathan: This is NAD, because the container comparison functions say "unless otherwise stated", 23.3.1p14 and table 97.

Ville: wrong, table 95 is relevant for `==`.

Jonathan: good point

2016-05: Marshall cleans up the wording around the change

[2016-08 - Chicago]

Thurs AM: Moved to Tentatively Ready

Proposed resolution:

Previous resolution [SUPERSEDED]:

This wording is relative to N4567.

1. Change 28.10 [re.results] p2 as indicated:

-2- The class template `match_results` shall satisfy the requirements of an allocator-aware container and of a sequence container, as specified in 23.2.3, except that only operations defined for const-qualified sequence containers are supported and that the semantics of comparison functions are different from those required for a container.

This wording is relative to N4567.

1. Change 28.10 [re.results] p2 as indicated:

-2- The class template `match_results` ~~shall satisfy~~ satisfies the requirements of an allocator-aware container and of a sequence container, ~~as specified in (23.2.3),~~ except that only operations defined for const-qualified sequence containers are supported and the semantics of comparison functions are different from those required for a container.

2591. `std::function`'s member template `target()` should not lead to undefined behaviour

Section: 20.14.12.2.5 [func.wrap.func.targ] **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krüger **Opened:** 2016-01-31 **Last modified:** 2016-08-08

Priority: 3

View all other [issues](#) in [func.wrap.func.targ].

View all issues with [Tentatively Ready](#) status.

Discussion:

This issue is a spin-off of LWG [2393](#), it solely focuses on the pre-condition of 20.14.12.2.5 [func.wrap.func.targ] p2:

Requires: τ shall be a type that is Callable (20.9.12.2) for parameter types `ArgTypes` and return type `R`.

Originally, the author of this issue here had assumed that simply removing the precondition as a side-step of fixing LWG [2393](#) would be uncontroversial. Discussions on the [library reflector](#) indicated that this is not the case, although it seemed that there was agreement on removing the undefined behaviour edge-case.

There exist basically the following positions:

1. The constraint should be removed completely, the function is considered as having a [wide contract](#).
2. The pre-condition should be replaced by a *Remarks* element, that has the effect of making the code ill-formed, if τ is a type that is not Lvalue-Callable (20.9.11.2) for parameter types `ArgTypes` and return type `R`. Technically this approach is still conforming with a wide contract function, because the definition of this contract form depends on runtime constraints.

Not yet explicitly discussed, but a possible variant of bullet (2) could be:

3. The pre-condition should be replaced by a *Remarks* element, that has the effect of SFINAE-constraining this member: "This function shall not participate in overload resolution unless τ is a type that is Lvalue-Callable (20.9.11.2) for parameter types `ArgTypes` and return type `R`".

The following describes a list of some selected arguments that have been provided for one or the other position using corresponding list items. Unless explicitly denoted, no difference has been accounted for option (3) over option (2).

1. a. It reflects existing implementation practice, Visual Studio 2015 SR1, gcc 6 libstdc++, and clang 3.8.0 libc++ do accept the following code:

```
#include <functional>
#include <iostream>
#include <typeinfo>
#include "boost/function.hpp"

void foo(int) {}

int main() {
    std::function<void(int)> f(foo);
    std::cout << f.target<void(*)>() << std::endl;
    boost::function<void(int)> f2(foo);
```

```
std::cout << f2.target<void(*)>() << std::endl;
}
```

and consistently output the implementation-specific result for **two null pointer** values.

- b. The current [Boost documentation](#) does not indicate *any* precondition for calling the `target` function, so it is natural that programmers would expect similar specification and behaviour for the corresponding standard component.
- c. There is a consistency argument in regard to the free function template `get_deleter`

```
template<class D, class T>
D* get_deleter(const shared_ptr<T>& p) noexcept;
```

This function also does not impose any pre-conditions on its template argument `D`.

2.
 - a. Programmers have control over the type they're passing to `target<T>()`. Passing a non-callable type can't possibly retrieve a non-null target, so it seems highly likely to be programmer error. Diagnosing that at compile time seems highly preferable to allowing this to return null, always, at runtime.
 - b. If τ is a reference type then the return type τ^* is ill-formed anyway. This implies that one can't blindly call `target<T>` without knowing what τ is.
 - c. It has been pointed out that some real world code, boiling down to

```
void foo() {}

int main() {
    std::function<void()> f = foo;
    if (f.target<decltype(foo)>()) {
        // fast path
    } else {
        // slow path
    }
}
```

had manifested as a performance issue and preparing a patch that made the library `static_assert` in that case solved this problem (Note that `decltype(foo)` evaluates to `void()`, but a proper argument of `target()` would have been the function *pointer* type `void(*)()`, because a function type `void()` is not any *Callable* type).

It might be worth adding that if use case (2 c) is indeed an often occurring idiom, it would make sense to consider to provide an explicit conversion to a function pointer (w/o template parameters that could be provided incorrectly), if the `std::function` object at runtime conditions contains a pointer to a real function, e.g.

```
R(*) (ArgTypes...) target_func_ptr() const noexcept;
```

[2016-08 Chicago]

Tues PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4567.

1. Change 20.14.12.2.5 [func.wrap.func.targ] p2 as indicated:

```
template<class T> T* target() noexcept;
template<class T> const T* target() const noexcept;
```

~~-2- Requires: τ shall be a type that is Callable (20.14.12.2 [func.wrap.func]) for parameter types `ArgTypes` and return type `R`.~~

-3- Returns: If `target_type() == typeid(T)` a pointer to the stored function target; otherwise a null pointer.

2598. addressof works on temporaries

Section: 20.10.10.1 [specialized.addressof] **Status:** [Tentatively Ready](#) **Submitter:** Brent Friedman **Opened:** 2016-03-06 **Last modified:** 2016-08-08

Priority: 3

View all other [issues](#) in [specialized.addressof].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG issue [970](#) removed the rvalue reference overload for `addressof`. This allows const prvalues to bind to a call to `addressof`, which is dissimilar from the behavior of `operator&`.

```
const vector<int> a();

void b()
{
    auto x = addressof(a()); // "ok"
    auto y = addressof<const int>(0); // "ok"
    auto z = &a(); //error: cannot take address of a temporary
}
```

[2016-08 Chicago]

Tues PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4582.

1. Change 20.10.2 [memory.syn], header `<memory>` synopsis, as indicated:

```
[...]
// 20.9.12, specialized algorithms:
template <class T> constexpr T* addressof(T& r) noexcept;
template <class T> const T* addressof(const T&& elem) = delete;
[...]
```

2. Change 20.10.10.1 [specialized.addressof] p1 as indicated:

```
template <class T> constexpr T* addressof(T& r) noexcept;
template <class T> const T* addressof(const T&& elem) = delete;
```

-1- *Returns:* The actual address of the object or function referenced by `r`, even in the presence of an overloaded `operator&`.

[2664](#). `operator/` (and other append) semantics not useful if argument has root

Section: 27.10.8.4.3 [path.append], 27.10.8.6 [path.non-member] **Status:** [Tentatively Ready](#) **Submitter:** Peter Dimov **Opened:** 2014-05-30 **Last modified:** 2016-08-06

Priority: 2

View other [active issues](#) in [path.append].

View all other [issues](#) in [path.append].

View all issues with [Tentatively Ready](#) status.

Discussion:

In a recent discussion on the Boost developers mailing list, the semantics of `operator /` and other append operations were questioned:

In brief, currently `p1 / p2` is required to concatenate the lexical representation of `p1` and `p2`, inserting a preferred separator as needed.

This means that, for example, `"c:\x" / "d:\y"` gives `"c:\x\d:\y"`, and that `"c:\x" / "\\server\share"` gives `"c:\x\\server\share"`. This is rarely, if ever, useful.

An alternative interpretation of `p1 / p2` could be that it yields a path that is the approximation of what `p2` would mean if interpreted in an environment in which `p1` is the starting directory. Under this interpretation, `"c:\x" / "d:\y"` gives `"d:\y"`, which is more likely to match what was intended.

I am not saying that this second interpretation is the right one, but I do say that we have reasons to suspect that the first one (lexical concatenation using a separator) may not be entirely correct. This leads me to think that the behavior of `p1 / p2`, when `p2` has a root, needs to be left implementation-defined, so that implementations are not required to do the wrong thing, as above.

This change will not affect the ordinary use case in which `p2` is a relative, root-less, path.

[17 Jun 2014 Rapperswil LWG will investigate issue at a subsequent meeting.]

[2016-02, Jacksonville]

Beman to provide wording.

[2016-06-13, Beman provides wording and rationale]

Rationale: The purpose of the `append` operations is to provide a simple concatenation facility for users wishing to extend a path by appending one or more additional elements, and to do so without worrying about the details of when a separator is needed. In that context it makes no sense to provide an argument that has a *root-name*. The simplest solution is simply to require `!p.has_root_name()`. The other suggested solutions IMO twist the functions into something harder to reason about yet any advantages for users are purely speculative. The concatenation functions can be used instead for corner cases.

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

[2016-07-03, Daniel comments]

The same wording area is touched by LWG [2732](#).

Previous resolution [SUPERSEDED]:

This wording is relative to N4594.

1. Change 27.10.8.4.3 [*path.append*] *path* appends as indicated:

```
path& operator/=(const path& p);
```

-?- Requires: `!p.has_root_name()`.

-2- *Effects*: Appends `path::preferred_separator` to `pathname` unless:

- an added *directory-separator* would be redundant, or
- an added *directory-separator* would change a relative path to an absolute path [Note: An empty path is relative. — *end note*], or
- `p.empty()` is true, or
- `*p.native().cbegin()` is a *directory-separator*.

Then appends `p.native()` to `pathname`.

-3- *Returns*: `*this`.

```
template <class Source>
    path& operator/=(const Source& source);
template <class Source>
    path& append(const Source& source);
template <class InputIterator>
    path& append(InputIterator first, InputIterator last);
```

-?- Requires: `!source.has_root_name()` OR `!*first.has_root_name()`, respectively.

-4- *Effects*: Appends `path::preferred_separator` to `pathname`, converting format and encoding if required (27.10.8.2 [*path.cvt*]), unless:

- an added *directory-separator* would be redundant, or
- an added *directory-separator* would change a relative path to an absolute path, or
- `source.empty()` is true, or
- `*source.native().cbegin()` is a *directory-separator*.

Then appends the effective range of `source` (27.10.8.3 [*path.req*]) or the range [`first`, `last`) to `pathname`, converting format and encoding if required (27.10.8.2 [*path.cvt*]).

-5- Returns: *this.

2. Change 27.10.8.6 [path.non-member] path non-member functions as indicated:

```
path operator/(const path& lhs, const path& rhs);
```

~~?- Requires:~~ !rhs.has_root_name().

-13- Returns: path(lhs) /= rhs.

[2016-08-03 Chicago]

After discussion on [2732](#), it was determined that the PR for that issue should be applied to this issue before it is accepted. That PR changes all the path appends to go through operator/=. so only one requires element remains necessary.

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606, and assumes that the PR for [2732](#) is applied.

1. Change 27.10.8.4.3 [path.append] as indicated:

```
path& operator/=(const path& p);
```

~~?- Requires:~~ !p.has_root_name().

-2- Effects: Appends path::preferred_separator to pathname unless:

- an added *directory-separator* would be redundant, or
- an added *directory-separator* would change a relative path to an absolute path [Note: An empty path is relative. — end note], or
- p.empty() is true, or
- *p.native().cbegin() is a *directory-separator*.

Then appends p.native() to pathname.

-3- Returns: *this.

2. Change 27.10.8.6 [path.non-member] p13 as indicated:

```
path operator/(const path& lhs, const path& rhs);
```

-13- ~~Returns~~ Effects: Equivalent to return path(lhs) /= rhs;.

[2672](#). Should is_empty use error_code in its specification?

Section: 27.10.15.19 [fs.op.is_empty] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2014-08-01 **Last modified:** 2016-08-03

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

The descriptions of most functions are explicit about the use of the ec argument, either saying something like "foo(p) or foo(p, ec), respectively" or using the ec argument like foo(p[, ec]), but is_empty does not.

27.10.15.19 [fs.op.is_empty]/2 refers to ec unconditionally, but more importantly 27.10.15.19 [fs.op.is_empty]/3 doesn't pass ec to the directory_iterator constructor or the file_size function.

[9 Oct 2014 Beman supplies proposed wording.]

[Apr 2016 Issue updated to address the C++ Working Paper. Previously addressed File System TS]

Previous resolution [SUPERSEDED]:

- bool is_empty(const path& p);
bool is_empty(const path& p, error_code& ec) noexcept;

Effects:

- Determine file_status s, as if by `status(p)` or `status(p, ec)`, respectively.
- For the signature with argument `ec`, return false if an error occurred.
- Otherwise, if `is_directory(s)`:
 - Create directory_iterator itr, as if by `directory_iterator(p)` or `directory_iterator(p, ec)`, respectively.
 - For the signature with argument `ec`, return false if an error occurred.
 - Otherwise, return `itr == directory_iterator()`.
- Otherwise:
 - Determine `uintmax_t sz`, as if by `file_size(p)` or `file_size(p, ec)`, respectively.
 - For the signature with argument `ec`, return false if an error occurred.
 - Otherwise, return `sz == 0`.

Remarks: The temporary objects described in *Effects* are for exposition only. Implementations are not required to create them.

Returns: `is_directory(s) ? directory_iterator(p) == directory_iterator() : file_size(p) == 0`; See *Effects*.

The signature with argument `ec` returns false if an error occurs.

Throws: As specified in Error reporting (7).

[2016-08 Chicago]

Wed PM: Move to Tentatively Ready

Proposed resolution:

1.

```
bool is_empty(const path& p);  
bool is_empty(const path& p, error_code& ec) noexcept;
```

Effects:

- Determine file_status s, as if by `status(p)` or `status(p, ec)`, respectively.
- For the signature with argument `ec`, return false if an error occurred.
- Otherwise, if `is_directory(s)`:
 - Create directory_iterator itr, as if by `directory_iterator(p)` or `directory_iterator(p, ec)`, respectively.
 - For the signature with argument `ec`, return false if an error occurred.
 - Otherwise, return `itr == directory_iterator()`.
- Otherwise:
 - Determine `uintmax_t sz`, as if by `file_size(p)` or `file_size(p, ec)`, respectively.
 - For the signature with argument `ec`, return false if an error occurred.
 - Otherwise, return `sz == 0`.

Returns: `is_directory(s) ? directory_iterator(p) == directory_iterator() : file_size(p) == 0`;

The signature with argument `ec` returns false if an error occurs.

Throws: As specified in Error reporting (7).

2678. std::filesystem enum classes overspecified

Section: 27.10.10.1 [enum.file_type], 27.10.10.2 [enum.copy_options], 27.10.10.4 [enum.directory_options] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2016-03-19 **Last modified:** 2016-08-08

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

The enum class `std::filesystem::file_type` specifies a collection of enumerators for describing the type of a file, but also specifies (1) the numeric values of the enumerators, and (2) that an implementation cannot extend the list with additional types known to it (and by extension, the standard cannot extend the list in future versions without breakage).

- These both seem like mistakes in the design. I would suggest we remove the specification of numerical values, and maybe also
- we allow implementations to add new implementation-defined value
 - we replace `std::filesystem::file_type::unknown` with a value that *only* means "exists but type could not be determined" and use implementation-specific values for other possibilities.

Similar overspecification exists for `std::filesystem::copy_options` and `std::filesystem::directory_options`, where again precise numerical values are specified.

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Proposed resolution:

1. Strike the "Value" column from:
 - 27.10.10.1 [enum.file_type], Table 145 Enum class file_type
 - 27.10.10.2 [enum.copy_options], Table 146 Enum class copy_options
 - 27.10.10.4 [enum.directory_options], Table 148 Enum class enum.directory_options.
2. Change 27.10.10.1 [enum.file_type] Table 145 Enum class file_type as indicated:

Constant	Meaning
none	The type of the file has not been determined or an error occurred while trying to determine the type.
not_found	Pseudo-type indicating the file was not found. [<i>Note:</i> The file not being found is not considered an error while determining the type of a file. — <i>end note</i>]
regular	Regular file
directory	Directory file
symlink	Symbolic link file
block	Block special file
character	Character special file
fifo	FIFO or pipe file
socket	Socket file
implementation-defined	Implementations that support file systems having file types in addition to the above file type types shall supply implementation-defined file type constants to separately identify each of those additional file types
unknown	The file does exist, but is of an operating system dependent type not covered by any of the other cases or the process does not have permission to query the file type The file exists but the type could not be determined

2679. Inconsistent Use of Effects and Equivalent To

Section: 17.5.1.4 [structure.specifications] **Status:** [Tentatively Ready](#) **Submitter:** Dawn Perchik **Opened:** 2016-04-14 **Last modified:** 2016-08-06

Priority: 3

View other [active issues](#) in [structure.specifications].

View all other [issues](#) in [structure.specifications].

View all issues with [Tentatively Ready](#) status.

Discussion:

- Some *Effects*: were intended to include the "Equivalent to" phrasing where no or different words were used. See examples in `std::filesystem` and throughout the library.
- The wording in `[structure.specifications]/4` is incorrect and *Effects*: throughout the library which use (or were intended to use) the "Equivalent to" phrasing need to be checked to make sure they fit the intended wording.

[2016-04, Issues Telecon]

The PR is fine; but bullet #1 in the report really should have a list of places to change.

Jonathan checked throughout the library for bullet #2 and found two problems in `[string.access]`, which have been added to the PR.

[2016-08-03 Chicago]

Fri PM: Move to Tentatively Ready

Proposed resolution:

1. Change `[structure.specifications]/4` as indicated:

"[...] if `F` has no *Returns*: element, a non-void return from `F` is specified by the ~~Returns: elements~~ **return statements** in the code sequence."

2. Add two `return` keywords to `[string.access]`:

```
const charT& front() const;
charT& front();
```

-7- *Requires*: `!empty()`.

-8- *Effects*: Equivalent to **return** `operator[] (0)`.

```
const charT& back() const;
charT& back();
```

-9- *Requires*: `!empty()`.

-10- *Effects*: Equivalent to **return** `operator[] (size() - 1)`.

[2680](#). Add "Equivalent to" to filesystem

Section: 27.10 `[filesystems]` **Status:** [Tentatively Ready](#) **Submitter:** Beman Dawes **Opened:** 2016-04-15 **Last modified:** 2016-08-08

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

The IS project editors have requested that filesystem *Effects* elements specified purely as C++ code include the phrase "Equivalent to" if this is the intent. They do not consider this change to be editorial.

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Proposed resolution:

27.10.8.4.5 `[path.modifiers]`

```
path& replace_filename(const path& replacement);
```

Effects: **Equivalent to:**

```
remove_filename();
operator/=(replacement);
```

27.10.8.6 `[path.non-member]`

```
void swap(path& lhs, path& rhs) noexcept;
```


Effects: **Equivalent to:** lhs.swap(rhs)

27.10.8.6.1 [path.io]

```
template <class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os, const path& p);
```

Effects: **Equivalent to:** os << quoted(p.string<charT, traits>()).

27.10.8.6.1 [path.io]

```
template <class charT, class traits>
basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, path& p);
```

Effects: **Equivalent to:**

```
basic_string<charT, traits> tmp;
is >> quoted(tmp);
p = tmp;
```

27.10.15.3 [fs.op.copy]

```
void copy(const path& from, const path& to);
void copy(const path& from, const path& to, error_code& ec) noexcept;
```

Effects: **Equivalent to:** copy(from, to, copy_options::none) OR copy(from, to, copy_options::none, ec), respectively.

27.10.15.5 [fs.op.copy_symlink]

```
void copy_symlink(const path& existing_symlink, const path& new_symlink);
void copy_symlink(const path& existing_symlink, const path& new_symlink,
error_code& ec) noexcept;
```

Effects: **Equivalent to:** *function*(read_symlink(existing_symlink), new_symlink) OR *function*(read_symlink(existing_symlink, ec), new_symlink, ec), respectively, where *function* is create_symlink OR create_directory_symlink, as appropriate.

2681. filesystem::copy() cannot copy symlinks

Section: 27.10.15.3 [fs.op.copy] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-04-19 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [fs.op.copy].

View all other [issues](#) in [fs.op.copy].

View all issues with [Tentatively Ready](#) status.

Discussion:

27.10.15.3 [fs.op.copy] paragraph 3 bullet (3.4) says that if is_symlink(f) and copy_options is set in options then it should copy a symlink, but that case cannot be reached.

is_symlink(f) can only be true if f was set using symlink_status(from), but that doesn't happen unless one of create_symlinks or skip_symlinks is set in options. It should depend on copy_symlinks too.

I'm not sure what the correct behaviour is, but I don't think we want to simply add a check for copy_symlinks in bullet (3.1) because that would mean that t = symlink_status(to), and I don't think we want that. Consider 'touch file; mkdir dir; ln -s dir link;' and then filesystem::copy("file", "link", filesystem::copy_options::copy_symlinks). If t = symlink_status(to) then is_directory(t) == false, and we don't use bullet (3.5.4) but go to (3.5.5) instead and fail. So when copy_symlinks is set we still need to use t = status(to) as specified today.

[2016-05 Issues Telecom]

This is related to [2682](#); and should be considered together.

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Proposed resolution:

Modify paragraph 3 of 27.10.15.3 [fs.op.copy] as shown:

Effects: Before the first use of `f` and `t`:

— If `(options & copy_options::create_symlinks) != copy_options::none || (options & copy_options::skip_symlinks) != copy_options::none` then auto `f = symlink_status(from)` and if needed auto `t = symlink_status(to)`.

— Otherwise, if `(options & copy_options::copy_symlinks) != copy_options::none` then auto `f = symlink_status(from)` and if needed auto `t = status(to)`.

— Otherwise, auto `f = status(from)` and if needed auto `t = status(to)`.

2686. Why is `std::hash` specialized for `error_code`, but not `error_condition`?

Section: 19.5.1 [system_error.syn] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-05-03 **Last modified:** 2016-08-04

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

Both `error_code` and `error_condition` have an `operator<` overload, enabling their use in associative containers without having to write a custom comparator.

However, only `error_code` has a `std::hash` specialization. So it's possible to have a `set<error_code>`, a `set<error_condition>`, an `unordered_set<error_code>`, but not an `unordered_set<error_condition>`. This seems...odd.

[2016-08 - Chicago]

Thurs AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4582.

1. Edit 19.5.1 [system_error.syn], header `<system_error>` synopsis, as indicated:

```
namespace std {
    // ...

    // 19.5.6 Hash support
    template<class T> struct hash;
    template<> struct hash<error_code>;
    template<> struct hash<error_condition>;

    // ...
}
```

2. Edit 19.5.6 [syserr.hash] as indicated:

```
template <> struct hash<error_code>;
template <> struct hash<error_condition>;
```

-1- The template specializations shall meet the requirements of class template `hash` (20.12.14).

2694. Application of LWG 436 accidentally deleted definition of "facet"

Section: 22.3.1.1.2 [locale.facet] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-04-15 **Last modified:** 2016-08-06

Priority: 3

View all other [issues](#) in [locale.facet].

View all issues with [Tentatively Ready](#) status.

Discussion:

[`locale.facet`]/1 (then-called [`lib.locale.facet`]) read in C++03:

Class `facet` is the base class for locale feature sets. A class is a *facet* if it is publicly derived from another facet, or if it is a class derived from `locale::facet` and containing a publicly-accessible declaration as follows: *[Footnote: This is a complete list of requirements; there are no other requirements. Thus, a facet class need not have a public copy constructor, assignment, default constructor, destructor, etc.]*

```
static ::std::locale::id id;
```

Template parameters in this clause which are required to be facets are those named `Facet` in declarations. A program that passes a type that is *not* a facet, as an (explicit or deduced) template parameter to a locale function expecting a facet, is ill-formed.

LWG [436](#)'s intent appears to be to ban `volatile`-qualified facets and permitting `const`-qualified ones. The PR was somewhat poorly worded, however, and the editor in applying it deleted the whole first half of the paragraph, including the definition of *facet* and the requirement for a static data member named `id`.

As a result, we have things like 22.3.1.1.3 [`locale.id`]/2 and 22.3.2 [`locale.global.templates`]/1 referring to the `id` member that's not defined anywhere in the working draft.

[2016-08-03 Chicago]

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4582.

1. Insert the following paragraph before 22.3.1.1.2 [`locale.facet`]/1:

-?- Class `facet` is the base class for locale feature sets. A class is a *facet* if it is publicly derived from another facet, or if it is a class derived from `locale::facet` and containing a publicly accessible declaration as follows: *[Footnote: This is a complete list of requirements; there are no other requirements. Thus, a facet class need not have a public copy constructor, assignment, default constructor, destructor, etc.]*

```
static ::std::locale::id id;
```

-1- Template parameters in this Clause which are required to be facets are those named `Facet` in declarations. A program that passes a type that is *not* a facet, or a type that refers to a `volatile`-qualified facet, as an (explicit or deduced) template parameter to a locale function expecting a facet, is ill-formed. A `const`-qualified facet is a valid template argument to any locale function that expects a `Facet` template parameter.

[2696](#). Interaction between `make_shared` and `enable_shared_from_this` is underspecified

Section: 20.11.2.2.6 [`util.smartptr.shared.create`] **Status:** [Tentatively Ready](#) **Submitter:** Joe Gottman **Opened:** 2016-04-02 **Last modified:** 2016-08-02

Priority: 2

View other [active issues](#) in [`util.smartptr.shared.create`].

View all other [issues](#) in [`util.smartptr.shared.create`].

View all issues with [Tentatively Ready](#) status.

Discussion:

For each public constructor of `std::shared_ptr`, the standard says that constructor *enables `shared_from_this`* if that constructor is expected to initialize the internal `weak_ptr` of a contained `enable_shared_from_this<X>` object. But there are other ways to construct a `shared_ptr` than by using a public constructor. The template functions `make_shared` and `allocate_shared` both require calling a private constructor, since no public constructor can fulfill the requirement that at most one allocation is made. The standard does not specify that that private constructor enables `shared_from_this`; therefore in the following code:

```
struct Foo : public std::enable_shared_from_this<Foo> {};  
  
int main() {  
    auto p = std::make_shared<Foo>();  
    assert(p == p->shared_from_this());  
    return 0;  
}
```

it is unspecified whether the assertion will fail.

[2016-05 Issues Telecom]

Jonathan Wakely to provide updated wording.

[2016-08 Chicago]

Monday PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4582.

1. Change 20.11.2.2.6 [util.smartptr.shared.create] indicated:

```
template<class T, class... Args> shared_ptr<T> make_shared(Args&&... args);
template<class T, class A, class... Args>
    shared_ptr<T> allocate_shared(const A& a, Args&&... args);
```

[...]

-6- Remarks: The shared_ptr constructor called by this function enables shared from this with the address of the newly constructed object of type T. Implementations should perform no more than one memory allocation. [Note: This provides efficiency equivalent to an intrusive smart pointer. — end note]

2699. Missing restriction in [numeric.requirements]

Section: 26.3 [numeric.requirements] **Status:** [Tentatively Ready](#) **Submitter:** Hubert Tong **Opened:** 2016-05-03 **Last modified:** 2016-08-06

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

In N4582 subclause 26.3 [numeric.requirements], the "considerable flexibility in how arrays are initialized" do not appear to allow for replacement of calls to the default constructor with calls to the copy constructor with an appropriate source value.

[2016-08-03 Chicago]

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4582.

1. Adjust 26.3 [numeric.requirements]/1 as indicated:

-1- The `complex` and `valarray` components are parameterized by the type of information they contain and manipulate. [...]

(1.1) — τ is not an abstract class (it has no pure virtual member functions);

[...]

(1.8) — If τ is a class, its assignment operator, copy and default constructors, and destructor shall correspond to each other in the following sense: Initialization of raw storage using the copy constructor on the value of $\tau()$, however obtained, is semantically equivalent to value initialization of the same raw storage. Initialization of raw storage using the default constructor, followed by assignment, is semantically equivalent to initialization of raw storage using the copy constructor. Destruction of an object, followed by initialization of its raw storage using the copy constructor, is semantically equivalent to assignment to the original object. [Note: This rule states, in part, that there shall not be any subtle differences in the semantics of initialization versus assignment. This gives an implementation considerable flexibility in how arrays are initialized. [Example: An implementation is allowed to initialize a `valarray` by allocating storage using the `new` operator (which implies a call to the default constructor for each element) and then assigning each element its value. Or the implementation can allocate raw storage and use the copy constructor to initialize each element. — end example] If the distinction between initialization and assignment is important for a class, or if it fails to

satisfy any of the other conditions listed above, the programmer should use `vector` (23.3.11) instead of `valarray` for that class; — *end note*

[2712.](#) `copy_file(from, to, ...)` has a number of unspecified error conditions

Section: 27.10.15.4 [fs.op.copy_file] **Status:** [Tentatively Ready](#) **Submitter:** Eric Fiselier **Opened:** 2016-05-10 **Last modified:** 2016-08-08

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

There are a number of error cases that `copy_file(from, to, ...)` does not take into account. Specifically the cases where:

- A. `from` does not exist
- B. `from` is not a regular file
- C. `to` exists and is not a regular file

These error cases should be specified as such.

[2016-05 Issues Telecom]

Eric to provide wording.

[2016-05-28, Eric Fiselier provides wording]

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4582.

1. Modify 27.10.15.4 [fs.op.copy_file] as indicated:

```
bool copy_file(const path& from, const path& to, copy_options options);
bool copy_file(const path& from, const path& to, copy_options options,
               error_code& ec) noexcept;
```

-3- *Requires:* At most one constant from each `copy_options` option group (27.10.10.2) is present in `options`.

-4- *Effects:* Report a file already exists error as specified in Error reporting (27.5.6.5) if:

- `!is_regular_file(from)`, OR
- `exists(to) and !is_regular_file(to)`, OR
- `exists(to) and equivalent(from, to)`, OR
- `exists(to) and (options & (copy_options::skip_existing | copy_options::overwrite_existing | copy_options::update_existing)) == copy_options::none`.

[2722.](#) `equivalent` incorrectly specifies throws clause

Section: 27.10.15.13 [fs.op.equivalent] **Status:** [Tentatively Ready](#) **Submitter:** Eric Fiselier **Opened:** 2016-05-28 **Last modified:** 2016-08-08

Priority: 3

View all issues with [Tentatively Ready](#) status.

Discussion:

The spec for `equivalent` has a throws clause which reads: [fs.op.equivalent]/5

Throws: `filesystem_error` if `(!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2))`, otherwise as specified in Error reporting (27.10.7).

This explicit requirement to throw is incorrect for the `equivalent` overload which takes an `error_code`.

Previous resolution [SUPERSEDED]:

This wording is relative to N4582.

1. Modify 27.10.15.13 [fs.op.equivalent] as follows:

```
bool equivalent(const path& p1, const path& p2);
bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;
```

-1- *Effects*: Determines file_status s1 and s2, as if by status(p1) and status(p2), respectively.

-2- *Returns*: If (!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2)) an error is reported (27.10.7 [fs.err.report]). Otherwise true, if s1 == s2 and p1 and p2 resolve to the same file system entity, else false. The signature with argument ec returns false if an error occurs.

-3- Two paths are considered to resolve to the same file system entity if two candidate entities reside on the same device at the same location. This is determined as if by the values of the POSIX stat structure, obtained as if by stat() for the two paths, having equal st_dev values and equal st_ino values.

-4- *Throws*: filesystem_error if (!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2)), otherwise as specified in Error reporting (27.10.7 [fs.err.report]).

[2016-06 Oulu — Daniel provides wording improvements]

mc: do we have an error reporting clause?

jw: there is no such clause

gr: it should go into the effects clause

dk: we have the same issue for file_size

dk: the right place is the effects clause

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4594.

1. Modify 27.10.15.13 [fs.op.equivalent] as follows:

```
bool equivalent(const path& p1, const path& p2);
bool equivalent(const path& p1, const path& p2, error_code& ec) noexcept;
```

-1- Let s1 and s2 be file_status, determined as if by status(p1) and status(p2), respectively.

-2- *Effects*: Determines s1 and s2. If (!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2)) an error is reported (27.10.7 [fs.err.report]).

-3- *Returns*: true, if s1 == s2 and p1 and p2 resolve to the same file system entity, else false. The signature with argument ec returns false if an error occurs.

-4- Two paths are considered to resolve to the same file system entity if two candidate entities reside on the same device at the same location. This is determined as if by the values of the POSIX stat structure, obtained as if by stat() for the two paths, having equal st_dev values and equal st_ino values.

-5- *Throws*: filesystem_error if (!exists(s1) && !exists(s2)) || (is_other(s1) && is_other(s2)), otherwise as specified in Error reporting (27.10.7 [fs.err.report]).

2729. Missing SFINAE on std::pair::operator=

Section: 20.4.2 [pairs.pair], 20.5.2.2 [tuple.assign] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2016-06-07 **Last modified:** 2016-08-04

Priority: 2

View other [active issues](#) in [pairs.pair].

View all other [issues](#) in [pairs.pair].

View all issues with [Tentatively Ready](#) status.

Discussion:

std::is_copy_assignable<std::pair<int, std::unique_ptr<int>>>::value is true, and should be false. We're missing a "shall not participate in overload resolution unless" for pair's operator=, and likewise for tuple.

[2016-08-03 Chicago LWG]

Inspired by Eric Fiselier and Ville, Walter and Nevin provide initial Proposed Resolution.

[2016-08 - Chicago]

Thurs PM: Moved to Tentatively Ready

Lots of discussion, but no one had a better idea.

Proposed resolution:

This wording is relative to N4606.

1. Change 20.4.2 [pairs.pair] as indicated:

```
pair& operator=(const pair& p);
```

-15- **RequiresRemarks:** This operator shall be defined as deleted unless is_copy_assignable_v<first_type> is true and is_copy_assignable_v<second_type> is true.

[...]

```
template<class U, class V> pair& operator=(const pair<U, V>& p);
```

-18- **RequiresRemarks:** This operator shall not participate in overload resolution unless is_assignable_v<first_type&, const U&> is true and is_assignable_v<second_type&, const V&> is true.

[...]

```
pair& operator=(pair&& p) noexcept(see below);
```

-21- **Remarks:** The expression inside noexcept is equivalent to:

```
is_nothrow_move_assignable_v<T1> && is_nothrow_move_assignable_v<T2>
```

-22- **RequiresRemarks:** This operator shall be defined as deleted unless is_move_assignable_v<first_type> is true and is_move_assignable_v<second_type> is true.

[...]

```
template<class U, class V> pair& operator=(pair<U, V>&& p);
```

-25- **RequiresRemarks:** This operator shall not participate in overload resolution unless is_assignable_v<first_type&, U&&> is true and is_assignable_v<second_type&, V&&> is true.

2. Change 20.5.2.2 [tuple.assign] as indicated:

```
tuple& operator=(const tuple& u);
```

-2- **RequiresRemarks:** This operator shall be defined as deleted unless is_copy_assignable_v<T_{*i*}> is true for all *i*.

[...]

```
tuple& operator=(tuple&& u) noexcept(see below);
```

-5- **Remark:** The expression inside noexcept is equivalent to the logical AND of the following expressions:

```
is_nothrow_move_assignable_v<Ti>
```

where T_{*i*} is the *i*th type in Types.

-6- **RequiresRemarks:** This operator shall be defined as deleted unless `is_move_assignable_v<Ti>` is true for all *i*.

[...]

```
template <class... UTypes>
tuple& operator=(const tuple<UTypes...>& u);
```

-9- **RequiresRemarks:** This operator shall not participate in overload resolution unless `sizeof...(Types) == sizeof...(UTypes)` and `is_assignable_v<Ti&, const Ui&>` is true for all *i*.

[...]

```
template <class... UTypes>
tuple& operator=(tuple<UTypes...>&& u);
```

-12- **RequiresRemarks:** This operator shall not participate in overload resolution unless `is_assignable_v<Ti&, Ui&&> == true` for all *i*; and `sizeof...(Types) == sizeof...(UTypes)`.

[...]

```
template <class U1, class U2> tuple& operator=(const pair<U1, U2>& u);
```

-15- **RequiresRemarks:** This operator shall not participate in overload resolution unless `sizeof...(Types) == 2`; and `is_assignable_v<T0&, const U1&>` is true for the first type `T0` in `Types` and `is_assignable_v<T1&, const U2&>` is true for the second type `T1` in `Types`.

[...]

```
template <class U1, class U2> tuple& operator=(pair<U1, U2>&& u);
```

-18- **RequiresRemarks:** This operator shall not participate in overload resolution unless `sizeof...(Types) == 2`; and `is_assignable_v<T0&, U1&&>` is true for the first type `T0` in `Types` and `is_assignable_v<T1&, U2&&>` is true for the second type `T1` in `Types`.

2732. Questionable specification of `path::operator/=` and `path::append`

Section: 27.10.8.4.3 [path.append] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-06-14 **Last modified:** 2016-08-08

Priority: 2

View other [active issues](#) in [path.append].

View all other [issues](#) in [path.append].

View all issues with [Tentatively Ready](#) status.

Discussion:

The current specification of `operator/=` taking a `const Source&` parameter, and of `path::append` in 27.10.8.4.3 [path.append] appears to require `Source` to have a `native()` and an `empty()` member, and seemingly requires different behavior for `append(empty_range)` and `append(first, last)` when `first == last` (the last two bullet points being specified with `source` alone), which doesn't make any sense.

It appears that these overloads can just be specified using the `operator/=(const path&)` overload.

[2016-07-03, Daniel comments]

The same wording area is affected by LWG [2664](#).

[2016-08 Chicago]

Wed AM: Move to Tentatively Ready

Friday AM, in discussing [2664](#) a comment about missing "equivalent to" language was made, so PR updated.

Previous Resolution [SUPERSEDED]

This wording is relative to N4594.

1. Edit 27.10.8.4.3 [path.append]/4-5 as indicated:

```
template <class Source>
    path& operator/=(const Source& source);
template <class Source>
    path& append(const Source& source);
```

~~[-?- Effects: operator/=(path(source)).~~

~~[-?- Returns: *this.~~

```
template <class InputIterator>
    path& append(InputIterator first, InputIterator last);
```

~~-4- Effects: Appends path::preferred_separator to pathname, converting format and encoding if required (27.10.8.2 [path.cvt]), unless:~~

- ~~o an added directory-separator would be redundant, or~~
- ~~o an added directory-separator would change an relative path to an absolute path, or~~
- ~~o source.empty() is true, or~~
- ~~o *source.native().cbegin() is a directory-separator.~~

~~Then appends the effective range of source (27.10.8.3 [path.req]) or the range {first, last} to pathname, converting format and encoding if required (27.10.8.2 [path.cvt])~~
~~operator/=(path(first, last)).~~

~~-5- Returns: *this.~~

Proposed resolution:

This wording is relative to N4606.

1. Edit 27.10.8.4.3 [path.append]/4-5 as indicated:

```
template <class Source>
    path& operator/=(const Source& source);
template <class Source>
    path& append(const Source& source);
```

~~[-?- Effects: Equivalent to return operator/=(path(source)).~~

```
template <class InputIterator>
    path& append(InputIterator first, InputIterator last);
```

~~-4- Effects: Equivalent to return operator/=(path(first, last)).~~
~~Appends path::preferred_separator to pathname, converting format and encoding if required (27.10.8.2 [path.cvt]), unless:~~

- ~~— an added directory-separator would be redundant, or~~
- ~~— an added directory-separator would change an relative path to an absolute path, or~~
- ~~— source.empty() is true, or~~
- ~~— *source.native().cbegin() is a directory-separator.~~

~~Then appends the effective range of source (27.10.8.3 [path.req]) or the range {first, last} to pathname, converting format and encoding if required (27.10.8.2 [path.cvt]).~~

~~-5- Returns: *this.~~

2733. [fund.ts.v2] gcd / lcm and bool

Section: 99 [fund.ts.v2::numeric.ops.gcd], 99 [fund.ts.v2::numeric.ops.lcm] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith
Opened: 2016-06-15 **Last modified:** 2016-08-02

Priority: 4

View all issues with [Tentatively Ready](#) **status.**

Discussion:

Addresses: [fund.ts.v2](#)

According to [N4562](#), gcd and lcm support bool as the operand type. The wording doesn't appear to cover the behavior for that case, since bool does not have a zero value and gcd / lcm are not normally mathematically defined over {false, true}.

Presumably gcd and lcm shouldn't accept arguments of type bool.

[2016-08-01, Walter Brown suggests wording]

A corresponding issue has been added addressing the WP, see LWG [2759](#).

[2016-08, Chicago]

Monday PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to [N4600](#).

1. Adjust 99 [numeric.ops.gcd] p3 as indicated:

```
template<class M, class N>
constexpr common_type_t<M, N> gcd(M m, N n);
```

[...]

-3- *Remarks:* If either M or N is not an integer type, or if either is (possibly cv-qualified) bool, the program is ill-formed.

2. Adjust 99 [numeric.ops.lcm] p3 as indicated:

```
template<class M, class N>
constexpr common_type_t<M, N> lcm(M m, N n);
```

[...]

-3- *Remarks:* If either M or N is not an integer type, or if either is (possibly cv-qualified) bool, the program is ill-formed.

[2735](#). std::abs(short), std::abs(signed char) and others should return int instead of double in order to be compatible with C++98 and C

Section: 26.9 [c.math] **Status:** [Tentatively Ready](#) **Submitter:** Jörn Heusipp **Opened:** 2016-06-16 **Last modified:** 2016-08-08

Priority: 3

View other [active issues](#) in [c.math].

View all other [issues](#) in [c.math].

View all issues with [Tentatively Ready](#) status.

Discussion:

Consider this C++98 program:

```
#include <cmath>
#include <cstdlib>

int main() {
    return std::abs(static_cast<short>(23)) % 42;
}
```

This works fine with C++98 compilers. At the std::abs(short) call, short gets promoted to int and std::abs(int) is called.

C++11 added the [following wording](#) on page 1083 §26.9 p15 b2 [c.math]:

Otherwise, if any argument of arithmetic type corresponding to a double parameter has type double or an integer type, then all arguments of arithmetic type corresponding to double parameters are effectively cast to double.

C++17 draft [additionally adds](#) on page 1080 §26.9 p10 [c.math]:

If `abs()` is called with an argument of type `X` for which `is_unsigned<X>::value` is true and if `X` cannot be converted to `int` by integral promotion (4.5), the program is ill-formed. [Note: Arguments that can be promoted to `int` are permitted for compatibility with C. — end note]

It is somewhat confusing and probably even contradictory to on the one hand specify `abs()` in terms of integral promotion in §26.9 p10 and on the other hand demand all integral types to be converted to `double` in §26.9 p15 b2.

Most compilers (each with their own respective library implementation) I tested (MSVC, Clang, older GCC) appear to not consider §26.9 p15 b2 for `std::abs` and compile the code successfully. GCC 4.5-5.3 (for `std::abs` but not for `::abs`) as well as GCC >=6.0 (for both `std::abs` and `::abs`) fail to compile in the following way: Taking §26.9 p15 b2 literally and applying it to `abs()` (which is listed in §26.9 p12) results in `abs(short)` returning `double`, and with `operator%` not being specified for `double`, this makes the program ill-formed.

I do acknowledge the reason for the wording and semantics demanded by §26.9 p15 b2, i.e. being able to call math functions with integral types or with partly floating point types and partly integral types. Converting integral types to `double` certainly makes sense here for all the other floating point math functions. However, `abs()` is special. `abs()` has overloads for the 3 wider integral types which return integral types. `abs()` originates in the C standard in `stdlib.h` and had originally been specified for integral types only. Calling it in C with a short argument returns an `int`. Calling `std::abs(short)` in C++98 also returns an `int`. Calling `std::abs(short)` in C++11 and later with §26.9 p15 b2 applied to `abs()` suddenly returns a `double`.

Additionally, this behaviour also breaks third-party C headers which contain macros or inline functions calling `abs(short)`.

As per discussion on [std-discussion](#), my reading of the standard as well as GCC's interpretation seem valid. However, as can be seen, this breaks existing code.

In addition to the compatibility concerns, having `std::abs(short)` return `double` is also very confusing and unintuitive.

The other (possibly, depending on their respective size relative to `int`) affected types besides `short` are `signed char`, `unsigned char` and `unsigned short`, and also `char`, `char16_t`, `char32_t` and `wchar_t`, (all of these are or may be promotable to `int`). Wider integral types are not affected because explicit overloads are specified for those types by §26.9 p6, §26.9 p7 and §26.9 p9. `div()` is also not affected because it is neither listed in §26.9 p12, nor does it actually provide any overload for `double` at all.

As far as I can see, the proposed or implemented solutions for LWG [2294](#), [2192](#) and/or [2086](#) do not resolve this issue.

I think both, §26.9 p10 [c.math] and §26.9 p15 [c.math] need some correction and clarification.

(Note: These changes would explicitly render the current implementation in GCC's `libstdc++` non-conforming, which would be a good thing, as outlined above.)

Previous resolution [SUPERSEDED]:

This wording is relative to N4594.

1. Modify 26.9 [c.math] as indicated:

-10- If `abs()` is called with an argument of type `X` for which `is_unsigned<X>::value` is true and if `X` cannot be converted to `int` by integral promotion (4.5), the program is ill-formed. If `abs()` is called with an argument of type `X` which can be converted to `int` by integral promotion (4.5), the argument is promoted to `int`. [Note: Arguments that can be promoted to `int` are promoted to `int` in order to keep permitted for compatibility with C. — end note]

[...]

-15- Moreover, there shall be additional overloads for these functions, with the exception of `abs()`, sufficient to ensure:

1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type (3.9.1) corresponding to `double` parameters are effectively cast to `long double`.
2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.
3. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.

See also: ISO C 7.5, 7.10.2, 7.10.6.

[2016-07 Chicago]

Monday: Some of this has been changed in N4606; the rest of the changes may be editorial.

Fri PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 26.9.1 [cmath.syn] as indicated:

-2- For each set of overloaded functions within `<cmath>`, **with the exception of `abs`,** there shall be additional overloads sufficient to ensure:

1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type (3.9.1) corresponding to `double` parameters are effectively cast to `long double`.
2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.
3. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.

[Note: `abs` is exempted from these rules in order to stay compatible with C. — end note]

See also: ISO C 7.5, 7.10.2, 7.10.6.

2736. `nullopt_t` insufficiently constrained

Section: 20.6.4 [optional.nullopt] **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2016-06-17 **Last modified:** 2016-08-08

Priority: 2

View all issues with [Tentatively Ready](#) status.

Discussion:

20.6.4 [optional.nullopt]/2 requires of `nullopt_t` that

Type `nullopt_t` shall not have a default constructor. It shall be a literal type. Constant `nullopt` shall be initialized with an argument of literal type.

This does not appear sufficient to foreclose the following implementation:

```
struct nullopt_t
{
    constexpr nullopt_t(const nullopt_t&) = default;
};

constexpr nullopt_t nullopt(nullopt_t{});
```

- `nullopt_t` has no default constructor because it has a user-declared (copy) constructor;
- `nullopt_t` has a trivial destructor, is an aggregate, and is a literal type;
- `nullopt` has been initialized with an argument of literal type, to wit, `nullopt_t`.

But such a `nullopt_t` is still constructible from `{}` and so still makes `opt = {}` ambiguous.

[2016-08 Chicago]

This is related to LWG [2510](#).

Monday PM: Ville to provide updated wording

Fri AM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Edit 20.6.4 [optional.nullopt]/2 as indicated:

[Drafting note: {} can do one of three things for a class type: it may be aggregate initialization, it may call a default constructor, or it may call an initializer-list constructor (see 8.6.4 [dcl.init.list], 13.3.1.7 [over.match.list]). The wording below forecloses all three possibilities. — end drafting note]

-2- Type `nullopt_t` shall not have a default constructor **or an initializer-list constructor**. It **shall not be an aggregate** **and** shall be a literal type. Constant `nullopt` shall be initialized with an argument of literal type.

2738. `is_constructible` with void types

Section: 20.15.4.3 [meta.unary.prop] **Status:** [Tentatively Ready](#) **Submitter:** S. B. Tam **Opened:** 2016-06-22 **Last modified:** 2016-08-08

Priority: Not Prioritized

View other [active issues](#) in [meta.unary.prop].

View all other [issues](#) in [meta.unary.prop].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2560](#) mention that there is no variable of function type. There's also no variable of `void` type, so should 20.15.4.3 [meta.unary.prop] also explicitly say that for a void type `T`, `is_constructible<T, Args...>::value` is false?

[2016-07-03, Daniel provides wording]

[2016-08 Chicago]

Wed PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4594.

1. Change 20.15.4.3 [meta.unary.prop], Table 52 — "Type property predicates", as indicated:

Table 52 — Type property predicates		
Template	Condition	Preconditions
...		
template <class T, class... Args> struct is_constructible;	For a function type <code>τ</code> or for a (possibly cv-qualified) void type <code>τ</code> , <code>is_constructible<T, Args...>::value</code> is false, otherwise <i>see below</i>	<code>τ</code> and all types in the parameter pack <code>Args</code> shall be complete types, (possibly cv-qualified) void, or arrays of unknown bound.
...		

2739. Issue with `time_point` non-member subtraction with an unsigned duration

Section: 20.17.6.5 [time.point.nonmember] **Status:** [Tentatively Ready](#) **Submitter:** Michael Winterberg **Opened:** 2016-06-23 **Last modified:** 2016-08-08

Priority: 0

View all other [issues](#) in [time.point.nonmember].

View all issues with [Tentatively Ready](#) status.

Discussion:

In N4594, 20.17.6.5 [time.point.nonmember], `operator-(time_point, duration)` is specified as:

```
template <class Clock, class Duration1, class Rep2, class Period2>
```

```
constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
```

-3- *Returns:* lhs + (-rhs).

When Rep2 is an unsigned integral type, the behavior is quite different with arithmetic of the underlying integral types because of the requirement to negate the incoming duration and then add that. It also ends up producing different results than the underlying durations as well as the non-member `time_point::operator-=`.

Consider this program:

```
#include <chrono>
#include <iostream>
#include <stdint>

using namespace std;
using namespace std::chrono;

int main()
{
    const duration<uint32_t> unsignedSeconds{5};

    auto someValue = system_clock::from_time_t(200);
    cout << system_clock::to_time_t(someValue) << '\n';
    cout << system_clock::to_time_t(someValue - unsignedSeconds) << '\n';
    someValue -= unsignedSeconds;
    cout << system_clock::to_time_t(someValue) << '\n';

    std::chrono::seconds signedDur{200};
    cout << signedDur.count() << '\n';
    cout << (signedDur - unsignedSeconds).count() << '\n';
    signedDur -= unsignedSeconds;
    cout << signedDur.count() << '\n';
}
```

The goal of the program is to compare the behavior of `time_point` non-member `operator-`, `time_point` member `operator-=`, `duration` non-member `operator-`, and `duration` member `operator-=` with basically the same inputs.

libc++ produces this output, which appears mandated by the standard:

```
200
4294967491
195
200
195
195
```

On the other hand, libstdc++ produces this output, which is what I "intuitively" expect and behaves more consistently:

```
200
195
195
200
195
195
```

Given the seemingly brief coverage of durations with unsigned representations in the standard, this seems to be an oversight rather than a deliberate choice for this behavior. Additionally, there may be other "unexpected" behaviors with durations with an unsigned representation, this is just the one that I've come across.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4594.

1. Change 20.17.6.5 [time.point.nonmember] as indicated:

```
template <class Clock, class Duration1, class Rep2, class Period2>
constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
```

-3- *Returns:* lhs + (-rhs) *C7*(lhs.time_since_epoch() - rhs), where *C7* is the type of the return value.

[2740](#). constexpr optional<T>::operator->

Section: 20.6.3.5 [optional.object.observe] **Status:** [Tentatively Ready](#) **Submitter:** Agustín K-ballo Bergé **Opened:** 2016-07-02 **Last modified:** 2016-08-08

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

optional<T>::operator->S are constrained to be constexpr functions only when T is not a type with an overloaded unary operator&. This constrain comes from the need to use addressof (or a similar mechanism), and the inability to do so in a constant expression in C++14. Given that addressof is now constexpr, this constrain is no longer needed.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4594.

1. Modify 20.6.3.5 [optional.object.observe] as indicated:

```
constexpr T const* operator->() const;
constexpr T* operator->();
```

-1- *Requires:* *this contains a value.

-2- *Returns:* val.

-3- *Throws:* Nothing.

-4- *Remarks:* ~~Unless T is a user-defined type with overloaded unary operator&, t~~ These functions shall be constexpr functions.

[2742](#). Inconsistent string interface taking string_view

Section: 21.3.1.2 [string.cons] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2016-07-06 **Last modified:** 2016-08-06

Priority: 1

View all other [issues](#) in [string.cons].

View all issues with [Tentatively Ready](#) status.

Discussion:

Generally, basic_string has a constructor matching each assign function and vice versa (except the constructor takes an allocator where assign does not). [P0254R2](#) violates this by adding an assign(basic_string_view, size_type pos, size_type n = npos) but no corresponding constructor.

[2016-08-04 Chicago LWG]

Robert Douglas provides initial wording.

We decided against another constructor overload to avoid the semantic confusion between:

```
basic_string(char const*, size_type length, Allocator = Allocator())
```

and

```
template<class T, class Foo = is_convertible_v<T const&, basic_string_view<charT, traits>>
basic_string(T const&, size_type pos, Allocator = Allocator())
```

where someone might call:

```
basic_string("HelloWorld", 5, 5);
```

and get "World", but then call

```
basic_string("HelloWorld", 5);
```

and instead get "Hello". The second parameter changes between length and position.

[08-2016, Chicago]

Fri PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. In 21.3.1 [basic.string] add the following constructor overload:

```
[...]
basic_string(const basic_string& str, size_type pos,
             const Allocator& a = Allocator());
basic_string(const basic_string& str, size_type pos, size_type n,
             const Allocator& a = Allocator());
template<class T>
basic_string(const T& t, size_type pos, size_type n, const Allocator& a = Allocator());
explicit basic_string(basic_string_view<charT, traits> sv,
                    const Allocator& a = Allocator());
[...]
```

2. In 21.3.1.2 [string.cons] add the following ctor definition:

```
template<class T>
basic_string(const T& t, size_type pos, size_type n, const Allocator& a = Allocator());
```

-?- *Effects:* Creates a variable, *sv*, as if by `basic_string_view<charT, traits> sv = t;` and then behaves the same as:

```
basic_string(sv.substr(pos, n), a)
```

-?- *Remarks:* This constructor shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true.

2744. any's in_place constructors

Section: 20.8.3.1 [any.cons] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen **Opened:** 2016-07-10 **Last modified:** 2016-08-08

Priority: 0

View other [active issues](#) in [any.cons].

View all other [issues](#) in [any.cons].

View all issues with [Tentatively Ready](#) status.

Discussion:

The `in_place` constructor that takes an `initializer_list` has both a *Requires:* for `is_constructible` and a *Remarks:* for `is_constructible`. The one that takes just a pack has just a *Requires:* for `is_constructible`.

I think both of those should be *Remarks:*, i.e. SFINAEable constraints. Otherwise querying `is_constructible` for an `any` with `in_place_t` will not give a reasonable answer, and I utterly fail to see any implementation burden in SFINAEing those constructors.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.8.3.1 [any.cons] as indicated:

```
template<class ValueType>
any(ValueType&& value);

[...]
```


-7- *Requires*: T shall satisfy the CopyConstructible requirements. If `is_copy_constructible_v<T>` is false, the program is ill-formed.

-8- *Effects*: Constructs an object of type `any` that contains an object of type T direct-initialized with `std::forward<ValueType>(value)`.

-9- *Remarks*: This constructor shall not participate in overload resolution if `decay_t<ValueType>` is the same type as any or if `ValueType` is a specialization of `in_place_type_t`.

[...]

```
template <class T, class... Args>
explicit any(in_place_type_t<T>, Args&&... args);
```

~~-11- *Requires*: `is_constructible_v<T, Args...>` is true.~~

-?- *Remarks*: This constructor shall not participate in overload resolution unless `is_constructible_v<T, Args...>` is true

[...]

```
template <class T, class U, class... Args>
explicit any(in_place_type_t<T>, initializer_list<U> il, Args&&... args);
```

~~-15- *Requires*: `is_constructible_v<T, initializer_list<U>&, Args...>` is true.~~

[...]

-19- *Remarks*: ~~The function~~ This constructor shall not participate in overload resolution unless `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

2745. [fund.ts.v2] Implementability of LWG 2451

Section: 99 [fund.ts.v2::optional.object] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2016-07-10 **Last modified:** 2016-08-08

Priority: 0

View all other [issues](#) in [fund.ts.v2::optional.object].

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

LWG [2451](#) adds conditionally explicit converting constructors to `optional<T>` that accept:

1. Types convertible to T : `template <class U> constexpr optional(T&&);`
2. Rvalue `optional<U>` when `U&&` is convertible to T : `template <class U> constexpr optional(optional<U>&&);`
3. Lvalue `const optional<U>` when `const U&` is convertible to T : `template <class U> constexpr optional(const optional<U>&);`

All three of these constructors are required to be `constexpr` "If T 's selected constructor is a `constexpr` constructor". While this is not problematic for #1, it is not possible in the current language to implement signatures #2 and #3 as `constexpr` functions for the same reasons that `optional`'s *non-converting* constructors from `optional<T>&&` and `const optional<T>&` cannot be `constexpr`.

We should remove the "`constexpr`" specifier from the declarations of the conditionally explicit converting constructors that accept `optional<U>&&` and `const optional<U>&`, and strike the remarks requiring these constructors to be `constexpr`.

[2016-07 Chicago]

Monday: P0 - tentatively ready

This needs to be considered for C++17 as well

Proposed resolution:

This wording is relative to [N4600](#).

Wording relative to N4600 + LWG [2451](#), although it should be noted that this resolution should be applied wherever

1. Edit 99 [optional.object] as indicated:

```
template <class T>
class optional
{
public:
    typedef T value_type;

    // 5.3.1, Constructors
    [...]
    template <class U> constexpr optional(U&&);
    template <class U> constexpr optional(const optional<U>&);
    template <class U> constexpr optional(optional<U>&&);
    [...]
};
```

2. In 99 [optional.object.ctor], modify the new signature specifications added by LWG [2451](#)

```
template <class U>
constexpr optional(const optional<U>& rhs);

[...]
```

~~-48- Remarks: If T's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor.~~ This constructor shall not participate in overload resolution unless [...]

```
template <class U>
constexpr optional(optional<U>&& rhs);

[...]
```

~~-53- Remarks: If T's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor.~~ This constructor shall not participate in overload resolution unless [...]

[2747](#). Possibly redundant std::move in [alg.foreach]

Section: 25.3.4 [alg.foreach] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-07-15 **Last modified:** 2016-08-08

Priority: 0

View all other [issues](#) in [alg.foreach].

View all issues with [Tentatively Ready](#) status.

Discussion:

25.3.4 [alg.foreach] p3 says *Returns:* std::move(f).

12.8 [class.copy] says that since f is a function parameter overload resolution to select the constructor for the return value is first performed as if for an rvalue, so the std::move is redundant.

It could be argued that it isn't entirely redundant, because it says that implementations can't do something slightly different like return an lvalue reference that is bound to f, which would prevent it being treated as an rvalue. We should discuss it.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4606.

1. Change 25.3.4 [alg.foreach] as indicated:

```
template<class InputIterator, class Function>
Function for_each(InputIterator first, InputIterator last, Function f);

[...]
```

-3- Returns: `std::move(f)`.

[...]

2748. swappable traits for optionals

Section: 20.6.3.4 [optional.object.swap], 20.6.9 [optional.specalg] **Status:** [Tentatively Ready](#) **Submitter:** Agustín K-ballo Bergé **Opened:** 2016-07-19 **Last modified:** 2016-08-08

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

optional didn't benefit from the wording modifications by P0185 "Adding [nothrow_]swappable traits"; as such, it suffers from LWG [2456](#), and does not play nice with swappable traits.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.6.3.4 [optional.object.swap] as indicated:

```
void swap(optional<T>& rhs) noexcept(see below);
```

[...]

-4- *Remarks:* The expression inside noexcept is equivalent to:

```
is_nothrow_move_constructible_v<T> && is_nothrow_swappable_v<T>noexcept(swap(declval<T>(), declval<T>()))
```

2. Modify 20.6.9 [optional.specalg] as indicated:

```
template <class T> void swap(optional<T>& x, optional<T>& y) noexcept(noexcept(x.swap(y)));
```

-1- *Effects:* Calls `x.swap(y)`.

-?- *Remarks:* This function shall not participate in overload resolution unless `is_move_constructible_v<T>` is true and `is_swappable_v<T>` is true.

2749. swappable traits for variants

Section: 20.7.2.6 [variant.swap], 20.7.9 [variant.specalg] **Status:** [Tentatively Ready](#) **Submitter:** Agustín K-ballo Bergé **Opened:** 2016-07-19 **Last modified:** 2016-09-11

Priority: 1

View all issues with [Tentatively Ready](#) status.

Discussion:

variant does not play nice with swappable traits, the non-member specialized swap overload is not SFINAE friendly. On the other hand, the member swap is SFINAE friendly, albeit with an incomplete condition, when arguably it shouldn't be. Given the *Effects*, *Throws*, and *Remarks* clauses, the SFINAE condition should include `is_move_constructible_v` and `is_move_assignable_v` to account for the involvement of variant's move constructor and move assignment operator (the noexcept specification is correct as is, since the move assignment operator would only be called for variants with different alternatives). This SFINAE condition should apply to the non-member swap overload, while the member swap should require all alternatives are swappable (as defined by 17.6.3.2 [swappable.requirements]).

[2016-07 Chicago]

Monday: P1 - review later in the week

Fri PM: Move to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.7.2.6 [variant.swap] as indicated:

```
void swap(variant& rhs) noexcept(see below);
```

-?- Requires: Lvalues of type τ_i shall be swappable and is move constructible $v<\tau_i>$ && is move assignable $v<\tau_i>$ is true for all i .

[...]

~~-3- Remarks: This function shall not participate in overload resolution unless is_swappable_v< τ_i > is true for all i .~~ If an exception is thrown during the call to function swap(get< i >(*this), get< i >(rhs)), the states of the contained values of *this and of rhs are determined by the exception safety guarantee of swap for lvalues of τ_i with i being index(). If an exception is thrown during the exchange of the values of *this and rhs, the states of the values of *this and of rhs are determined by the exception safety guarantee of variant's move constructor and move assignment operator. The expression inside noexcept is equivalent to the logical AND of is_nothrow_move_constructible_v< τ_i > && is_nothrow_swappable_v< τ_i > for all i .

2. Modify 20.7.9 [variant.specalg] as indicated:

```
template <class... Types> void swap(variant<Types...> v, variant<Types...> w) noexcept(see below);
```

-1- Effects: Equivalent to `v.swap(w)`.

-2- Remarks: This function shall not participate in overload resolution unless is_move_constructible_v< τ_i > && is_move_assignable_v< τ_i > && is swappable_v< τ_i > is true for all i . The expression inside noexcept is equivalent to `noexcept(v.swap(w))`.

[2016-08-13, Reopened by Casey Carter]

It is possible to exchange the value of two variants using only move construction on the alternative types, as if by

```
auto tmp = move(x);
x.emplace< $i$ >(move(y));
y.emplace< $j$ >(move(tmp));
```

where i is `y.index()` and j is `tmp.index()`. Consequently, variant's member swap need not require move assignable alternatives.

[2016-09-09 Issues Resolution Telecom]

Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.7.2.6 [variant.swap] as indicated:

```
void swap(variant& rhs) noexcept(see below);
```

-?- Requires: Lvalues of type τ_i shall be swappable and is move constructible $v<\tau_i>$ shall be true for all i .

[...]

-2- Throws: If `index() == rhs.index()`, aAny exception thrown by swap(get< i >(*this), get< i >(rhs)) with i being index() and variant's move constructor and assignment operator. Otherwise, any exception thrown by the move constructor of τ_i or τ_j with i being index() and j being `rhs.index()`.

-3- Remarks: ~~This function shall not participate in overload resolution unless is_swappable_v< τ_i > is true for all i .~~ If an exception is thrown during the call to function swap(get< i >(*this), get< i >(rhs)), the states of the contained values of *this and of rhs are determined by the exception safety guarantee of swap for lvalues of τ_i with i being index(). If an exception is thrown during the exchange of the values of *this and rhs, the states of the values of *this and of rhs are determined by the exception safety guarantee of variant's move constructor ~~and move assignment operator~~. The expression inside noexcept is equivalent

to the logical AND of `is_nothrow_move_constructible_v<Ti>` && `is_nothrow_swappable_v<Ti>` for all *i*.

2. Modify 20.7.9 [variant.specalg] as indicated:

```
template <class... Types> void swap(variant<Types...>& v, variant<Types...>& w) noexcept(see below);
```

-1- *Effects*: Equivalent to `v.swap(w)`.

-2- *Remarks*: This function shall not participate in overload resolution unless `is_move_constructible_v<Ti>` && `is_swappable_v<Ti>` is true for all *i*. The expression inside `noexcept` is equivalent to `noexcept(v.swap(w))`.

[2750](#). [fund.ts.v2] LWG 2451 conversion constructor constraint

Section: 99 [fund.ts.v2::optional.object.ctor] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2016-07-20 **Last modified:** 2016-08-08

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

Addresses: fund.ts.v2

LWG [2451](#) adds a converting constructor to `optional` with signature:

```
template <class U>
constexpr optional(U&& v);
```

and specifies that "This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is true and `U` is not the same type as `T`." This suffices to avoid this constructor being selected by overload resolution for arguments that should match the move constructor, but not for arguments that should match the copy constructor. The recent churn around tuple's constructors suggests that we want this constructor to not participate in overload resolution if `remove_cv_t<remove_reference_t<U>>` is the same type as `T`.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to [N4600](#).

Wording relative to N4600 + LWG [2451](#), although it should be noted that this resolution should be applied wherever LWG [2451](#) is applied, be that to the fundamentals TS or the specification of `optional` in the C++ Working Paper.

1. In 99 [optional.object.ctor], modify as indicated:

```
template <class U>
constexpr optional(U&& v);
```

[...]

-43- *Remarks*: If `T`'s selected constructor is a `constexpr` constructor, this constructor shall be a `constexpr` constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is true and `decay_t<U>` is not the same type as `T`. The constructor is explicit if and only if `is_convertible_v<U&&, T>` is false.

[2752](#). "Throws:" clauses of `async` and `packaged_task` are unimplementable

Section: 30.6.8 [futures.async], 30.6.9.1 [futures.task.members] **Status:** [Tentatively Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2016-07-07 **Last modified:** 2016-08-08

Priority: 3

View other [active issues](#) in [futures.async].

View all other [issues](#) in [futures.async].

View all issues with [Tentatively Ready](#) status.

Discussion:

`std::async` is a request from the user for type erasure; as any given function gets passed to `async` which returns only `future<ReturnType>`. Therefore, it needs to be able to allocate memory, as other issues (e.g. LWG [2202](#)) indicate. However, `async`'s *Throws* clause doesn't allow an implementation to do this, as it permits only `future_error`.

`std::packaged_task`'s constructor allocates memory using a user supplied allocator. An implementation needs to call the user's `allocate` to allocate such memory. The user's `allocate` function is not constrained to throwing only `bad_alloc`; it can raise whatever it wants, but `packaged_task`'s constructor prohibits this.

[2016-07 Chicago]

Alisdair thinks the third bullet is not quite right.

Previous resolution [SUPERSEDED]:

1. Change 30.6.8 [futures.async] p6 to:

Throws: `system_error` if `policy == launch::async` and the implementation is unable to start a new thread, or `std::bad_alloc` if memory for the internal data structures could not be allocated.

2. Change 30.6.9.1 [futures.task.members] p5 to:

```
template <class F>
    packaged_task(F&& f);
template <class F, class Allocator>
    packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

Throws:

(?) — Any exceptions thrown by the copy or move constructor of `f`, or
(?) — For the first version, `std::bad_alloc` if memory for the internal data structures could not be allocated.
(?) — For the second version, any exceptions thrown by
`std::allocator_traits<Allocator>::template rebind<unspecified>::allocate.`

[2016-08 Chicago]

Wed PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 30.6.8 [futures.async] p6 to:

Throws: `system_error` if `policy == launch::async` and the implementation is unable to start a new thread, or
`std::bad_alloc` if memory for the internal data structures could not be allocated.

2. Change 30.6.9.1 [futures.task.members] p5 to:

```
template <class F>
    packaged_task(F&& f);
template <class F, class Allocator>
    packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

Throws:

(?) — Any exceptions thrown by the copy or move constructor of `f`, or
(?) — For the first version, `std::bad_alloc` if memory for the internal data structures could not be allocated.
(?) — For the second version, any exceptions thrown by
`std::allocator_traits<Allocator>::template rebind traits<unspecified>::allocate.`

Section: 20.6.3.1 [optional.object.ctor], 20.6.3.3 [optional.object.assign] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter
Opened: 2016-07-22 **Last modified:** 2016-08-08

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

To use `optional<T>` as if it were a `T` in generic contexts, `optional<T>`'s "generic" operations must behave as do those of `T` under overload resolution. At minimum, `optional`'s constructors and assignment operators should not participate in overload resolution with argument types that cannot be used to construct/assign the contained `T` so that `is_constructible_v<optional<T>, Args...>` (respectively `is_assignable_v<optional<T>&, RHS>`) is equivalent to `is_constructible_v<T, Args...>` (respectively `is_assignable_v<T&, RHS>`).

In passing, note that the `Requires` element for `optional`'s in-place `initializer_list` constructor unnecessarily duplicates its `Remarks` element; it should be removed.

It should also be noted that the resolution of LWG [2451](#) adds constructors to `optional` with appropriate constraints, but does not constrain the additional assignment operators. If LWG chooses to apply the resolution of 2451 to the WP, the `Requires` elements of the additional assignment operators should also be converted to constraints as the wording herein does for the assignment operators in N4606.

[2016-07 Chicago]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4606.

1. Remove 20.6.3.1 [optional.object.ctor] p3, and add a new paragraph after p6:

```
optional(const optional<T>& rhs);
```

~~-3- Requires: is_copy_constructible_v<T> is true.~~

[...]

~~-?- Remarks: The function shall not participate in overload resolution unless is_copy_constructible_v<T> is true.~~

2. Remove 20.6.3.1 [optional.object.ctor] p7, and change p11 to:

```
optional(optional<T>&& rhs) noexcept(see below);
```

~~-7- Requires: is_move_constructible_v<T> is true.~~

[...]

~~-11- Remarks: The expression inside noexcept is equivalent to is_nothrow_move_constructible_v<T>. The function shall not participate in overload resolution unless is_move_constructible_v<T> is true.~~

3. Remove 20.6.3.1 [optional.object.ctor] p12, and change p16 to:

```
constexpr optional(const T& v);
```

~~-12- Requires: is_copy_constructible_v<T> is true.~~

[...]

~~-16- Remarks: If T's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. The function shall not participate in overload resolution unless is_copy_constructible_v<T> is true.~~

4. Remove 20.6.3.1 [optional.object.ctor] p17, and change p21 to:

```
constexpr optional(T&& v);
```

~~-17- Requires: is_move_constructible_v<T> is true.~~

[...]

-21- *Remarks:* If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. The function shall not participate in overload resolution unless `is_move_constructible_v<T>` is true.

5. Remove 20.6.3.1 [optional.object.ctor] p22, and change p26 to:

```
template <class... Args>
constexpr explicit optional(in_place_t, Args&&... args);
```

~~-22- *Requires:* `is_constructible_v<T, Args&&...>` is true.~~

[...]

-26- *Remarks:* If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. The function shall not participate in overload resolution unless `is_constructible_v<T, Args...>` is true.

6. Remove 20.6.3.1 [optional.object.ctor] p27.

```
template <class U, class... Args>
constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

~~-27- *Requires:* `is_constructible_v<T, initializer_list<U>&, Args&&...>` is true.~~

[...]

7. Remove 20.6.3.3 [optional.object.assign] p4, and change p8 to:

```
optional<T>& operator=(const optional<T>& rhs);
```

~~-4- *Requires:* `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true.~~

[...]

-8- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to τ 's copy constructor, no effect. If an exception is thrown during the call to τ 's copy assignment, the state of its contained value is as defined by the exception safety guarantee of τ 's copy assignment. The function shall not participate in overload resolution unless `is_copy_constructible_v<T>` && `is_copy_assignable_v<T>` is true.

8. Remove 20.6.3.3 [optional.object.assign] p9, and add a new paragraph after p14:

```
optional<T>& operator=(optional<T>&& rhs) noexcept(see below);
```

~~-9- *Requires:* `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.~~

[...]

-14- *Remarks:* [...] If an exception is thrown during the call to τ 's move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of τ 's move assignment.

The function shall not participate in overload resolution unless `is_move_constructible_v<T>` && `is_move_assignable_v<T>` is true.

9. Remove 20.6.3.3 [optional.object.assign] p15, and change p19 to (yes, this wording is odd - the intent is that it will "do the right thing" after incorporation of LWG [2451](#)):

```
template <class U> optional<T>& operator=(U&& v);
```

~~-15- *Requires:* `is_constructible_v<T, U>` is true and `is_assignable_v<T&, U>` is true.~~

[...]

-19- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to τ 's constructor, the state of `v` is determined by the exception safety guarantee of τ 's constructor. If an exception is thrown during the call to τ 's assignment, the state of `*val` and `v` is determined by the exception safety guarantee of τ 's assignment. The function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` && `is_constructible_v<T, U>` && `is_assignable_v<T&, U>` is true.

Section: 20.8.3.1 [any.cons], 20.8.3.2 [any.assign], 20.8.3.3 [any.modifiers] **Status:** [Tentatively Ready](#) **Submitter:** Ville Voutilainen
Opened: 2016-07-05 **Last modified:** 2016-08-04

Priority: 1

View other [active issues](#) in [any.cons].

View all other [issues](#) in [any.cons].

View all issues with [Tentatively Ready](#) status.

Discussion:

The `in_place` constructors and `emplace` functions added by [P0032R3](#) don't require `CopyConstructible`.

They must. Otherwise copying an `any` that's made to hold a non-`CopyConstructible` type must fail with a run-time error. Since that's crazy, we want to prevent storing non-`CopyConstructible` types in an `any`.

Previously, the requirement for `CopyConstructible` was just on the converting constructor template and the converting assignment operator template on `any`. Now that we are adding two `in_place` constructor overloads and two `emplace` overloads, it seems reasonable to require `CopyConstructible` in some more general location, in order to avoid repeating that requirement all over the place.

[2016-07 — Chicago]

Monday: P1

Tuesday: Ville/Billy/Billy provide wording

[2016-08-02: Daniel comments]

The P/R wording of this issue brought to my attention that the recently added `emplace` functions of `std::any` introduced a breakage of a previous class invariant that only a decayed type could be stored as object into an `any`, this prevented storing arrays, references, functions, and cv-qualified types. The new constraints added by Ville do prevent some of these types (e.g. neither arrays nor functions meet the `CopyConstructible` requirements), but we need to cope with cv-qualified types and reference types.

[2016-08-02: Agustín K-ballo Bergé comments]

Presumably the constructors `any(in_place_type_t<T>, ...)` would need to be modified in the same way the `emplace` overloads were.

[2016-08-02: Ville adjusts the P/R to cope with the problems pointed out by Daniel's and Agustín's comments]

Ville notes that [2746](#), [2754](#) and [2756](#) all go together.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

Drafting note: this P/R doesn't turn the *Requires*-clauses into *Remarks*-clauses. We might want to do that separately, because SFINAEing the constructors allows users to query for `is_constructible` and get the right answer. Failing to mandate the SFINAE will lead to non-portable answers for `is_constructible`. Currently, libstdc++ SFINAEs. That should be done as a separate issue, as this issue is an urgent bug-fix but the mandated SFINAE is not.

1. Change 20.8.3 [any.class], class `any` synopsis, as indicated:

```
class any {
public:
    [...]
    template <class T, class... Args>
        explicit any(in_place_type_t<T>, Args&&...);
    template <class T, class U, class... Args>
        explicit any(in_place_type_t<T>, initializer_list<U>, Args&&...);

    [...]
    template <class T, class... Args>
        void emplace(Args&& ...);
    template <class T, class U, class... Args>
        void emplace(initializer_list<U>, Args&&...);
    [...]
```

```
};
```

2. Change 20.8.3.1 [any.cons] as indicated:

```
template<class ValueType>
any(ValueType&& value);
```

-6- Let T be equal to `decay_t<ValueType>`.

-7- *Requires:* T shall satisfy the `CopyConstructible` requirements. ~~If `is_copy_constructible_v<T>` is false, the program is ill-formed.~~

[...]

-9- *Remarks:* This constructor shall not participate in overload resolution ~~if~~ unless `decay_t<ValueType>` is not the same type as any and is copy_constructible v<T> is true.

```
template <class TValueType, class... Args>
explicit any(in_place_type_t<TValueType>, Args&&... args);
```

-?- Let T be equal to `remove_cv_t<ValueType>`.

-11- *Requires:* T shall satisfy the `CopyConstructible` requirements ~~`is_constructible_v<T, Args...>` is true.~~

[...]

-?- *Remarks:* This constructor shall not participate in overload resolution unless `is_reference v<T>` is false, `is_array v<T>` is false, `is_function v<T>` is false, `is_copy_constructible v<T>` is true and `is_constructible v<T, Args...>` is true.

```
template <class TValueType, class U, class... Args>
explicit any(in_place_type_t<TValueType>, initializer_list<U> il, Args&&... args);
```

-?- Let T be equal to `remove_cv_t<ValueType>`.

-15- *Requires:* T shall satisfy the `CopyConstructible` requirements ~~`is_constructible_v<T, initializer_list<U>&, Args...>` is true.~~

[...]

-19- *Remarks:* The function shall not participate in overload resolution unless `is_reference v<T>` is false, `is_array v<T>` is false, `is_function v<T>` is false, `is_copy_constructible v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

3. Change 20.8.3.2 [any.assign] as indicated:

```
template<class ValueType>
any& operator=(ValueType&& rhs);
```

-7- Let T be equal to `decay_t<ValueType>`.

-8- *Requires:* T shall satisfy the `CopyConstructible` requirements. ~~If `is_copy_constructible_v<T>` is false, the program is ill-formed.~~

[...]

-11- *Remarks:* This operator shall not participate in overload resolution ~~if~~ unless `decay_t<ValueType>` is not the same type as any and is copy_constructible v<T> is true.

4. Change 20.8.3.3 [any.modifiers] as indicated:

```
template <class TValueType, class... Args>
void emplace(Args&&... args);
```

-?- Let T be equal to `remove_cv_t<ValueType>`.

-1- *Requires:* T shall satisfy the `CopyConstructible` requirements ~~`is_constructible_v<T, Args...>` is true.~~

[...]

-5- *Remarks:* If an exception is thrown during the call to T 's constructor, $*this$ does not contain a value, and any previously contained object has been destroyed. This function shall not participate in overload resolution unless $is_reference\ v<T>$ is false, $is_array\ v<T>$ is false, $is_function\ v<T>$ is false, $is_copy_constructible\ v<T>$ is true and $is_constructible\ v<T, Args\dots>$ is true.

```
template <class TValueType, class U, class... Args>
void emplace(initializer_list<U> il, Args&&... args);
```

-?- Let T be equal to $remove_cv\ t<ValueType>$.

-?- *Requires:* T shall satisfy the `CopyConstructible` requirements.

-6- *Effects:* [...]

[...]

-9- *Remarks:* If an exception is thrown during the call to T 's constructor, $*this$ does not contain a value, and any previously contained object has been destroyed. The function shall not participate in overload resolution unless $is_reference\ v<T>$ is false, $is_array\ v<T>$ is false, $is_function\ v<T>$ is false, $is_copy_constructible\ v<T>$ is true and $is_constructible_v<T, initializer_list<U>\&, Args\dots>$ is true.

[2016-08-03: Ville comments and revises his proposed wording]

After discussing the latest P/R, here's an update. What this update does is that:

1. It strikes the *Requires*-clauses and does *not* add `CopyConstructible` to the *Requires*-clauses.

Rationale: any doesn't care whether the type it holds satisfies the semantic requirements of the `CopyConstructible` concept. The syntactic requirements are now SFINAE constraints in *Requires*-clauses.

2. It reverts back towards `decay_t` rather than `remove_cv_t`, and does *not* add the suggested SFINAE constraints for `is_reference/is_array/is_function`.

Rationale:

- a. any decays by design. It's to some extent inconsistent to not protect against decay in the `ValueType` constructor/assignment operator, but to protect against decay in the `in_place_t` constructors and `emplace` functions
- b. I think it's saner to just decay than to potentially run into situations where I need to `remove_reference` inside `in_place_t`.

Based on that, this P/R should supersede the previous one. We want to look at this new P/R in LWG and potentially send it to LEWG for verification. Personally, I think this P/R is the more conservative one, doesn't add significant new functionality, and is consistent, and is thus not really Library-Evolutionary.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Change 20.8.3 [any.class], class any synopsis, as indicated:

```
class any {
public:
    [...]
    template <class TValueType, class... Args>
        explicit any(in_place_type_t<TValueType>, Args&&...);
    template <class TValueType, class U, class... Args>
        explicit any(in_place_type_t<TValueType>, initializer_list<U>, Args&&...);

    [...]
    template <class TValueType, class... Args>
        void emplace(Args&& ...);
    template <class TValueType, class U, class... Args>
        void emplace(initializer_list<U>, Args&&...);
    [...]
};
```

2. Change 20.8.3.1 [any.cons] as indicated:

```
template<class ValueType>
```

any(ValueType&& value);

-6- Let τ be equal to `decay_t<ValueType>`.

~~-7- *Requires:* τ shall satisfy the CopyConstructible requirements. If `is_copy_constructible_v<T>` is false, the program is ill-formed.~~

[...]

-9- *Remarks:* This constructor shall not participate in overload resolution ~~if~~ unless `decay_t<ValueType>` is not the same type as any and is copy_constructible v<T> is true.

```
template <class τValueType, class... Args>
explicit any(in_place_type_t<τValueType>, Args&&... args);
```

-?- Let τ be equal to `decay_t<ValueType>`.

~~-11- *Requires:* `is_constructible_v<T, Args...>` is true.~~

[...]

-?- *Remarks:* This constructor shall not participate in overload resolution unless `is_copy_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.

```
template <class τValueType, class U, class... Args>
explicit any(in_place_type_t<τValueType>, initializer_list<U> il, Args&&... args);
```

-?- Let τ be equal to `decay_t<ValueType>`.

~~-15- *Requires:* `is_constructible_v<T, initializer_list<U>&, Args...>` is true.~~

[...]

-19- *Remarks:* The function shall not participate in overload resolution unless `is_copy_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

3. Change 20.8.3.2 [any.assign] as indicated:

```
template<class ValueType>
any& operator=(ValueType&& rhs);
```

-7- Let τ be equal to `decay_t<ValueType>`.

~~-8- *Requires:* τ shall satisfy the CopyConstructible requirements. If `is_copy_constructible_v<T>` is false, the program is ill-formed.~~

[...]

-11- *Remarks:* This operator shall not participate in overload resolution ~~if~~ unless `decay_t<ValueType>` is not the same type as any and is copy_constructible v<T> is true.

4. Change 20.8.3.3 [any.modifiers] as indicated:

```
template <class τValueType, class... Args>
void emplace(Args&&... args);
```

-?- Let τ be equal to `decay_t<ValueType>`.

~~-1- *Requires:* `is_constructible_v<T, Args...>` is true.~~

[...]

-5- *Remarks:* If an exception is thrown during the call to τ 's constructor, `*this` does not contain a value, and any previously contained object has been destroyed. This function shall not participate in overload resolution unless `is_copy_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.

```
template <class τValueType, class U, class... Args>
void emplace(initializer_list<U> il, Args&&... args);
```

-?- Let τ be equal to `decay_t<ValueType>`.

-6- *Effects:* [...]

[...]

-9- *Remarks:* If an exception is thrown during the call to τ 's constructor, $*this$ does not contain a value, and any previously contained object has been destroyed. The function shall not participate in overload resolution unless $is_copy_constructible_v<\tau>$ is true and $is_constructible_v<\tau, initializer_list<U>\&, Args\dots>$ is true.

[2016-08-03: Ville comments and revises his proposed wording]

This P/R brings back the CopyConstructible parts of the relevant Requires-clauses but removes the other parts of the Requires-clauses.

[2016-08 - Chicago]

Thurs PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 20.8.3 [any.class], class any synopsis, as indicated:

```
class any {
public:
    [...]
    template <class  $\tau$ ValueType, class... Args>
        explicit any(in_place_type_t< $\tau$ ValueType>, Args&&...);
    template <class  $\tau$ ValueType, class U, class... Args>
        explicit any(in_place_type_t< $\tau$ ValueType>, initializer_list<U>, Args&&...);

    [...]
    template <class  $\tau$ ValueType, class... Args>
        void emplace(Args&& ...);
    template <class  $\tau$ ValueType, class U, class... Args>
        void emplace(initializer_list<U>, Args&&...);
    [...]
};
```

2. Change 20.8.3.1 [any.cons] as indicated:

```
template<class ValueType>
any(ValueType&& value);
```

-6- Let τ be $decay_t<ValueType>$.

-7- *Requires:* τ shall satisfy the CopyConstructible requirements. If $is_copy_constructible_v<\tau>$ is false, the program is ill-formed.

[...]

-9- *Remarks:* This constructor shall not participate in overload resolution if $is_copy_constructible_v<\tau>$ is false, the program is ill-formed. unless τ is the same type as any and $is_copy_constructible_v<\tau>$ is true.

```
template <class  $\tau$ ValueType, class... Args>
explicit any(in_place_type_t< $\tau$ ValueType>, Args&&... args);
```

-?- Let τ be $decay_t<ValueType>$.

-11- *Requires:* τ shall satisfy the CopyConstructible requirements $is_constructible_v<\tau, Args\dots>$ is true.

[...]

-?- *Remarks:* This constructor shall not participate in overload resolution unless $is_copy_constructible_v<\tau>$ is true and $is_constructible_v<\tau, Args\dots>$ is true.

```
template <class  $\tau$ ValueType, class U, class... Args>
explicit any(in_place_type_t< $\tau$ ValueType>, initializer_list<U> il, Args&&... args);
```

-?- Let τ be $decay_t<ValueType>$.

-15- *Requires:* τ shall satisfy the CopyConstructible requirements $is_constructible_v<\tau, initializer_list<U>\&, Args\dots>$ is true.

[...]

-19- *Remarks:* The function shall not participate in overload resolution unless `is_copy_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

3. Change 20.8.3.2 [any.assign] as indicated:

```
template<class ValueType>
any& operator=(ValueType&& rhs);
```

-7- Let τ be `decay_t<ValueType>`.

-8- *Requires:* τ shall satisfy the CopyConstructible requirements. If `is_copy_constructible_v<\tau>` is false, the program is ill-formed.

[...]

-11- *Remarks:* This operator shall not participate in overload resolution if `unless` `decay_t<ValueType>` is not the same type as any `and is_copy_constructible_v<T>` is true.

4. Change 20.8.3.3 [any.modifiers] as indicated:

```
template <class \tauValueType, class... Args>
void emplace(Args&&... args);
```

-?- Let τ be `decay_t<ValueType>`.

-1- *Requires:* `\tau` shall satisfy the CopyConstructible requirements `is_constructible_v<\tau, Args...>` is true.

[...]

-5- *Remarks:* If an exception is thrown during the call to τ 's constructor, `*this` does not contain a value, and any previously contained object has been destroyed. This function shall not participate in overload resolution unless `is_copy_constructible_v<T>` is true and `is_constructible_v<T, Args...>` is true.

```
template <class \tauValueType, class U, class... Args>
void emplace(initializer_list<U> il, Args&&... args);
```

-?- Let τ be `decay_t<ValueType>`.

-?- *Requires:* τ shall satisfy the CopyConstructible requirements.

-6- *Effects:* [...]

[...]

-9- *Remarks:* If an exception is thrown during the call to τ 's constructor, `*this` does not contain a value, and any previously contained object has been destroyed. The function shall not participate in overload resolution unless `is_copy_constructible_v<T>` is true and `is_constructible_v<T, initializer_list<U>&, Args...>` is true.

2755. §[string.view.io] uses non-existent `basic_string_view::to_string` function

Section: 21.4.4 [string.view.io], 21.3.2.9 [string.io] **Status:** [Tentatively Ready](#) **Submitter:** Billy Baker **Opened:** 2016-07-26 **Last modified:** 2016-08-08

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

In looking at N4606, [string.view.io] has an *Effects* clause that references `basic_string_view::to_string` which no longer exists after the application of [P0254R2](#).

[2016-07-26, Marshall suggests concrete wording]

[2016-07 Chicago LWG]

Monday: P0 - tentatively ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 21.3.2.9 [string.io] as indicated:

```
template<class charT, class traits, class Allocator>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os,
          const basic_string<charT, traits, Allocator>& str);
```

~~-5- Effects: Equivalent to: return os << basic_string_view<charT, traits>(str); Behaves as a formatted output function (27.7.3.6.1 [ostream.formatted.reqmts]) of os. Forms a character sequence seq, initially consisting of the elements defined by the range [str.begin(), str.end()). Determines padding for seq as described in 27.7.3.6.1 [ostream.formatted.reqmts]. Then inserts seq as if by calling os.rdbuf()->sputn(seq, n), where n is the larger of os.width() and str.size(); then calls os.width(0);~~

~~-6- Returns: os~~

2. Modify 21.4.4 [string.view.io] as indicated:

```
template<class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& os,
          basic_string_view<charT, traits> str);
```

~~-1- Effects: Equivalent to: return os << str.to_string(); Behaves as a formatted output function (27.7.3.6.1 [ostream.formatted.reqmts]) of os. Forms a character sequence seq, initially consisting of the elements defined by the range [str.begin(), str.end()). Determines padding for seq as described in 27.7.3.6.1 [ostream.formatted.reqmts]. Then inserts seq as if by calling os.rdbuf()->sputn(seq, n), where n is the larger of os.width() and str.size(); then calls os.width(0);~~

~~-?- Returns: os~~

2756. C++ WP optional<T> should 'forward' T's implicit conversions

Section: 20.6.3 [optional.object] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2016-07-26 **Last modified:** 2016-10-07

Priority: 1

View other [active issues](#) in [optional.object].

View all other [issues](#) in [optional.object].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2451](#) adds converting constructors and assignment operators to optional. The committee voted to apply it to the Library Fundamentals 2 TS WP in Oulu as part of LWG Motion 3. In both LWG and LEWG discussion of this issue, it was considered to be critical to apply to the specification of optional before shipping C++17 — it was an oversight on the part of LWG that there was no motion brought to apply this resolution to the C++ WP.

LWG [2745](#) proposes removal of the constexpr specifier from the declarations of the converting constructors from const optional<U>& and optional<U>&& since they are not implementable as constexpr constructors in C++17.

This issue proposes application of the resolution of LWG [2451](#) as amended by LWG [2745](#) to the specification of optional in the C++ WP.

[2016-07 — Chicago]

Monday: Priority set to 1; will re-review later in the week

[2016-08-03, Tomasz comments]

1. Value forwarding constructor (template<typename U> optional(U&&)) is underspecified.

For the following use code:

```
optional<T> o1;
```

```
optional<T> o2(o1);
```

The second constructor will invoke value forwarding ($U = \text{optional}<T>\&$) constructor (better match) instead of the $\text{optional}<T>$ copy constructor, in case if T can be constructed from $\text{optional}<T>$. This happens for any type T that has unconstrained perfect forwarding constructor, especially $\text{optional}<\text{any}>$.

2. The behavior of the construction of the $\text{optional}<T>$ ot from $\text{optional}<U>$ ou is inconsistent for classes T than can be constructed both from $\text{optional}<U>$ and U . There are two possible semantics for such operation:

- *unwrapping*: if ou is initialized ($\text{bool}(ou)$), initialize contained value of ot from $*ou$, otherwise make ot uninitialized ($!\text{bool}(ot)$)
- *value forwarding*: initialize contained value of ot from ou , ot is always initialized ($\text{bool}(ot)$).

With the proposed resolution, construction is preferring *value forwarding*, while assignment is always using *unwrapping*.

For example, if we consider following class:

```
struct Widget
{
    Widget(int);
    Widget(optional<int>);
};
```

Notice, that such set of constructor is pretty common in situation when the construction of the `Widget` from known value is common and usage of `optional` version is rare. In such situation, presence of `Widget(int)` construction is an optimization used to avoid unnecessary empty checks and construction `optional<int>`.

For the following declarations:

```
optional<Widget> w1(make_optional(10));
optional<Widget> w2;
w2 = make_optional(10);
```

The `w1` will contain a value created using `Widget(optional<int>)` constructor, as corresponding *unwrapping* constructor ($\text{optional}<U>$ $\text{const}\&$) is eliminated by `is_constructible_v<T, const optional<U>\&>` (`is_constructible_v<Widget, const optional<int>\&>`) having a true value. In contrast `w2` will contain a value created using `Widget(int)` constructor, as corresponding value forwarding assignment ($U\&\&$) is eliminated by the fact that `std::decay_t<U>` (`optional<int>`) is specialization of `optional`.

In addition, the construction is having a preference for *value forwarding* and assignment is always using *unwrapping*. That means that for the following class:

```
struct Thingy
{
    Thingy(optional<string>);
};

optional<Thingy> t1(optional<string>("test"));
```

The `t1` has a contained value constructed from using `Thingy(optional<std::string>)`, as *unwrapping* constructor ($\text{optional}<U>$ $\text{const}\&$) are eliminated by the `is_constructible<T, U const\&>` (`is_constructible<Thingy, std::string const\&>`) being false. However the following:

```
t1 = optional<std::string>("test2");
```

will not compile, because, the *value forwarding* assignment ($U\&\&$) is eliminated by `std::decay_t<U>` (`optional<std::string>`) being `optional` specialization and the *unwrapping* assignment ($\text{optional}<U>$ $\text{const}\&$) is ill-formed because `is_constructible<T, U const\&>` (`is_constructible<Thingy, std::string const\&>`) is false.

3. The semantics of construction and assignment, of $\text{optional}<\text{optional}<V>\>$ from $\text{optional}<U>$ where U is convertible to/ same as T is also inconsistent. Firstly, in this situation the $\text{optional}<V>$ is a type that can be constructed both from $\text{optional}<U>$ and U so it fails into set of problem described above. However in addition we have following non-template constructor in $\text{optional}<T>$:

```
optional(T const\&);
```

Which for $\text{optional}<\text{optional}<V>\>$ is equivalent to:

```
optional(optional<V> const\&);
```

While there is no corresponding non-template assignment from T $\text{const}\&$, to make sure that `o = {};` always clear an `optional o`.

So for the following declarations:

```
optional<int> oi;  
optional<short> os;  
  
optional<optional<int>> ooi1(oi);  
optional<optional<int>> ooi2(os);
```

The `ooi1` uses *from- τ* constructor, while `ooi2` uses *value forwarding* constructor. In this case both `ooi1` and `ooi2` are initialized and their contained values `*ooi1`, `*ooi2` are uninitialized `optionals`. However, if we will attempt to make construction consistent with assignment, by preferring *unwrapping* (`optional<U> const&`), then `ooi2` will end up being uninitialized.

In summary, I believe that relation between *unwrapping*, *value forwarding* and *from- τ* construction/assignment is to subtle to being handled as defect report and requires a full paper analyzing possible design and their pros/cons.

Tuesday PM: Ville and Eric to implement and report back in Issaquah. Moved to Open

Ville notes that [2746](#), [2754](#) and [2756](#) all go together.

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.6.3 [optional.object] as indicated:

```
template <class T> class optional  
{  
public:  
    using value_type = T;  
  
    // 20.6.3.1, Constructors  
    constexpr optional() noexcept;  
    constexpr optional(nullopt_t) noexcept;  
    optional(const optional &);  
    optional(optional &&) noexcept(see below);  
    constexpr optional(const T &);  
    constexpr optional(T &&);  
    template <class... Args> constexpr explicit optional(in_place_t, Args &&...);  
    template <class U, class... Args>  
        constexpr explicit optional(in_place_t, initializer_list<U>, Args &&...);  
    template <class U> constexpr optional(U &&);  
    template <class U> optional(const optional<U> &);  
    template <class U> optional(optional<U> &&);  
  
    [...]  
  
    // 20.6.3.3, Assignment  
    optional &operator=(nullopt_t) noexcept;  
    optional &operator=(const optional &);  
    optional &operator=(optional &&) noexcept(see below);  
    template <class U> optional &operator=(U &&);  
    template <class U> optional& operator=(const optional<U> &);  
    template <class U> optional& operator=(optional<U> &&);  
    template <class... Args> void emplace(Args &&...);  
    template <class U, class... Args>  
        void emplace(initializer_list<U>, Args &&...);  
  
    [...]  
};
```

2. In 20.6.3.1 [optional.object.ctor], insert new signature specifications after p31:

[Note: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — end note]

```
template <class U>  
constexpr optional(U&& v);
```

[-? Effects: Initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::forward<U>(v)`.

[-? Postconditions: `*this` contains a value.

-?- Throws: Any exception thrown by the selected constructor of τ .

-?- Remarks: If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless $\text{is_constructible } v<\tau, U\&\>$ is true and U is not the same type as τ . The constructor is explicit if and only if $\text{is_convertible } v<U\&\&, \tau>$ is false.

```
template <class U>
optional(const optional<U>& rhs);
```

-?- Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type τ with the expression $*rhs$.

-?- Postconditions: $\text{bool}(rhs) == \text{bool}(*this)$.

-?- Throws: Any exception thrown by the selected constructor of τ .

-?- Remarks: This constructor shall not participate in overload resolution unless $\text{is_constructible } v<\tau, \text{const } U\>$ is true, $\text{is_same}<\text{decay } t<U>, \tau>$ is false, $\text{is_constructible } v<\tau, \text{const optional}<U>\&\>$ is false and $\text{is_convertible } v<\text{const optional}<U>\&, \tau>$ is false. The constructor is explicit if and only if $\text{is_convertible } v<\text{const } U\&, \tau>$ is false.

```
template <class U>
optional(optional<U>&& rhs);
```

-?- Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type τ with the expression $\text{std::move}(*rhs)$. $\text{bool}(rhs)$ is unchanged.

-?- Postconditions: $\text{bool}(rhs) == \text{bool}(*this)$.

-?- Throws: Any exception thrown by the selected constructor of τ .

-?- Remarks: This constructor shall not participate in overload resolution unless $\text{is_constructible } v<\tau, U\&\>$ is true, $\text{is_same}<\text{decay } t<U>, \tau>$ is false, $\text{is_constructible } v<\tau, \text{optional}<U>\&\&$ is false and $\text{is_convertible } v<\text{optional}<U>\&\&, \tau>$ is false and U is not the same type as τ . The constructor is explicit if and only if $\text{is_convertible } v<U\&\&, \tau>$ is false.

3. In 20.6.3.3 [optional.object.assign], change as indicated:

```
template <class U> optional<T>& operator=(U&& v);
```

-22- Remarks: If an exception is thrown, the result of the expression $\text{bool}(*this)$ remains unchanged. If an exception is thrown during the call to τ 's constructor, the state of v is determined by the exception safety guarantee of τ 's constructor. If an exception is thrown during the call to τ 's assignment, the state of $*val$ and v is determined by the exception safety guarantee of τ 's assignment. The function shall not participate in overload resolution unless $\text{decay } t<U>$ is not $\text{nullopt } t$ and $\text{decay } t<U>$ is not a specialization of optional . $\text{is_same}_v<\text{decay } t<U>, \tau>$ is true.

-23- Notes: The reason for providing such generic assignment and then constraining it so that effectively $\tau == U$ is to guarantee that assignment of the form $o = \{ \}$ is unambiguous.

```
template <class U> optional<T>& operator=(const optional<U>& rhs);
```

-?- Requires: $\text{is_constructible } v<\tau, \text{const } U\>$ is true and $\text{is_assignable } v<\tau\&, \text{const } U\>$ is true.

-?- Effects:

Table ? — $\text{optional::operator}=(\text{const optional}<U>\&)$ effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns $*rhs$ to the contained value</u>	<u>initializes the contained value as if direct-non-list-initializing an object of type τ with $*rhs$</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling $val->\tau::\sim\tau()$</u>	<u>no effect</u>

-?- Returns: *this.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` is false.

```
template <class U> optional<T>& operator=(optional<U>&& rhs);
```

-?- Requires: `is_constructible_v<T, U>` is true and `is_assignable_v<T&, U>` is true.

-?- Effects: The result of the expression `bool(rhs)` remains unchanged.

Table ? — `optional::operator=(optional<U>&&)` effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value as if direct-non-list-initializing an object of type <code>T</code> with <code>std::move(*rhs)</code>
rhs does not contain a value	destroys the contained value by calling <code>val->T::~~T()</code>	no effect

-?- Returns: *this.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` is false.

[2016-08-05 Chicago LWG]

Ville provides revised wording, that also fixes LWG [2753](#).

Rationale:

1. The resolution of LWG [2753](#) makes special member functions defined as deleted in case the desired constraints aren't met.
2. There is no decay for the converting constructor `optional(U&&)`, there is a `remove_reference` instead. The target type may hold a cv-qualified type, and the incoming type may hold a cv-qualified type, but neither can hold a reference. Thus, `remove_reference` is what we need, `remove_cv` would be wrong, and `decay` would be wrong.
3. There is no decay or `remove_reference` for converting constructors like `optional(optional<U>)`, because none is needed.
4. For `optional(U&&)`, an added constraint is that `U` is not a specialization of `optional`

[2016-08, Chicago]

Fri PM: Move to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.6.3 [optional.object] as indicated:

```
template <class T> class optional
{
public:
    using value_type = T;

    // 20.6.3.1, Constructors
```

```
constexpr optional() noexcept;
constexpr optional(nullopt_t) noexcept;
optional(const optional &);
optional(optional &&) noexcept(see below);
constexpr optional(const T &);
constexpr optional(T &&);
template <class... Args> constexpr explicit optional(in_place_t, Args &&...);
template <class U, class... Args>
    constexpr explicit optional(in_place_t, initializer_list<U>, Args &&...);
template <class U> EXPLICIT constexpr optional(U &&);
template <class U> EXPLICIT optional(const optional<U> &);
template <class U> EXPLICIT optional(optional<U> &&);

[...]
```

// 20.6.3.3, Assignment

```
optional &operator=(nullopt_t) noexcept;
optional &operator=(const optional &);
optional &operator=(optional &&) noexcept(see below);
template <class U> optional &operator=(U &&);
template <class U> optional& operator=(const optional<U> &);
template <class U> optional& operator=(optional<U> &&);
template <class... Args> void emplace(Args &&...);
template <class U, class... Args>
    void emplace(initializer_list<U>, Args &&...);

[...]
```

};

2. Change 20.6.3.1 [optional.object.ctor] as indicated:

```
optional(const optional<T>& rhs);

-3- Requires: is_copy_constructible_v<T> is true.

[...]
```

~~?- Remarks: This constructor shall be defined as deleted unless is_copy_constructible_v<T> is true.~~

```
optional(optional<T>&& rhs) noexcept(see below);

-7- Requires: is_move_constructible_v<T> is true.

[...]
```

~~-11- Remarks: The expression inside noexcept is equivalent to is_nothrow_move_constructible_v<T>. This constructor shall be defined as deleted unless is_move_constructible_v<T> is true.~~

```
constexpr optional(const T& v);

-12- Requires: is_copy_constructible_v<T> is true.

[...]
```

~~-16- Remarks: If T's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless is_copy_constructible_v<T> is true.~~

```
constexpr optional(T&& v);

-17- Requires: is_move_constructible_v<T> is true.

[...]
```

~~-21- Remarks: If T's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless is_move_constructible_v<T> is true.~~

```
template <class... Args> constexpr explicit optional(in_place_t, Args&&... args);

-22- Requires: is_constructible_v<T, Args&&...> is true.

[...]
```

-26- *Remarks:* If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless $\text{is_constructible_v}<\tau, \text{Args}...\text{>}$ is true.

```
template <class U, class... Args>
constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

-27- *Requires:* $\text{is_constructible_v}<\tau, \text{initializer_list}<U>\&, \text{Args}\&\&...\text{>}$ is true.

[...]

-31- *Remarks:* The function constructor shall not participate in overload resolution unless $\text{is_constructible_v}<\tau, \text{initializer_list}<U>\&, \text{Args}\&\&...\text{>}$ is true. If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor.

[Note: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — end note]

```
template <class U>
EXPLICIT constexpr optional(U&& v);
```

-?- *Effects:* Initializes the contained value as if direct-non-list-initializing an object of type τ with the expression $\text{std::forward}<U>(v)$.

-?- *Postconditions:* $*\text{this}$ contains a value.

-?- *Throws:* Any exception thrown by the selected constructor of τ .

-?- *Remarks:* If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless $\text{is_constructible_v}<\tau, U\&\&\text{>}$ is true, $\text{remove_reference}<\tau>$ is not the same type as τ , and U is not a specialization of optional . The constructor is explicit if and only if $\text{is_convertible_v}<U\&\&, \tau>$ is false.

```
template <class U>
EXPLICIT optional(const optional<U>& rhs);
```

-?- *Effects:* If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type τ with the expression $*\text{rhs}$.

-?- *Postconditions:* $\text{bool}(\text{rhs}) == \text{bool}(*\text{this})$.

-?- *Throws:* Any exception thrown by the selected constructor of τ .

-?- *Remarks:* This constructor shall not participate in overload resolution unless $\text{is_constructible_v}<\tau, \text{const } U\&\&\text{>}$ is true and $\text{is_same_v}<U, \tau>$ is false. The constructor is explicit if and only if $\text{is_convertible_v}<\text{const } U\&, \tau>$ is false.

```
template <class U>
EXPLICIT optional(optional<U>&& rhs);
```

-?- *Effects:* If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type τ with the expression $\text{std::move}(*\text{rhs})$. $\text{bool}(\text{rhs})$ is unchanged.

-?- *Postconditions:* $\text{bool}(\text{rhs}) == \text{bool}(*\text{this})$.

-?- *Throws:* Any exception thrown by the selected constructor of τ .

-?- *Remarks:* This constructor shall not participate in overload resolution unless $\text{is_constructible_v}<\tau, U\&\&\text{>}$ is true and $\text{is_same_v}<U, \tau>$ is false. The constructor is explicit if and only if $\text{is_convertible_v}<U\&\&, \tau>$ is false.

3. Change 20.6.3.3 [optional.object.assign] as indicated:

```
optional<T>& operator=(const optional<T>& rhs);
```

-4- *Requires:* $\text{is_copy_constructible_v}<\tau>$ is true and $\text{is_copy_assignable_v}<\tau>$ is true.

[...]

-8- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. This operator shall be defined as deleted unless `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true.

```
optional<T>& operator=(optional<T>&& rhs) noexcept(see below);
```

~~-9- *Requires:* `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.~~

[...]

-13- *Remarks:* The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>
```

-14- If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment. This operator shall be defined as deleted unless `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.

```
template <class U> optional<T>& operator=(U&& v);
```

~~-15- *Requires:* `is_constructible_v<T, U>` is true and `is_assignable_v<T&, U>` is true.~~

[...]

-19- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment. ~~The~~This function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` `decay_t<U>` is not `nullptr_t`, `decay_t<U>` is not a specialization of `optional`, `is_constructible_v<T, U>` is true and `is_assignable_v<T&, U>` is true.

~~-20- *Notes:* The reason for providing such generic assignment and then constraining it so that effectively `T == U` is to guarantee that assignment of the form `o = {}` is unambiguous.~~

```
template <class U> optional<T>& operator=(const optional<U>& rhs);
```

-?- *Effects:*

Table ? — `optional::operator=(const optional<U>&)` effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	assigns <code>*rhs</code> to the contained value	initializes the contained value as if direct-non-list-initializing an object of type <code>T</code> with <code>*rhs</code>
<u>rhs does not contain a value</u>	destroys the contained value by calling <code>val->T::~~T()</code>	no effect

-?- *Returns:* `*this`.

-?- *Postconditions:* `bool(rhs) == bool(*this)`.

-?- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `is_constructible_v<T, const U&>` is true and `is_assignable_v<T&, const U&>` is true and `is_same_v<U, T>` is false.

```
template <class U> optional<T>& operator=(optional<U>&& rhs);
```

-?- Effects: The result of the expression `bool(rhs)` remains unchanged.

Table ? — `optional::operator=(optional<U>&&)` effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value as if direct-non-list-initializing an object of type <code>T</code> with <code>std::move(*rhs)</code>
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code>	no effect

-?- Returns: `*this`.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment. The function shall not participate in overload resolution unless `is_constructible v<T, U>` is true and `is_assignable v<T&, U>` is true and `is_same v<U, T>` is false.

[2016-08-08 Ville reopens and provides improved wording]

This alternative proposed wording also resolves [2753](#).

The constructors that take a `const T&` or `T&&` are replaced by a constructor template that takes a `U&&` and defaults `U = T`. This allows copy-list-initialization with empty braces to still work:

```
optional<whatever> o = {}; // equivalent to initializing optional with nullopt
```

This resolution makes converting constructors and assignments have the same capabilities, including using arguments that can't be deduced. That is achieved by using a perfect-forwarding constructor and an assignment operator that default their argument to `T`. We don't need separate overloads for `T`, the overload for `U` does the job:

```
optional<vector<int>> ovi{{1, 2, 3}}; // still works
ovi = {4, 5, 6, 7}; // now works, didn't work before
```

Furthermore, this proposed wording makes `optional` "always unwrap". That is, the result of the following initializations is the same:

```
optional<optional<int>> oi = optional<int>();
optional<optional<int>> oi = optional<short>();
```

Both of those initializations initialize the `optional` wrapping another `optional` as if initializing with `nullopt`. Assignments do the same. These changes solve the issues pointed out by Tomasz Kamiński.

This P/R has been implemented and tested as a modification on top of `libstdc++`'s `optional`.

[2016-08-08 Ville and Tomasz collaborate and improve wording]

The suggested wording retains `optional`'s converting constructors and assignment operators, but provides sane results for the types Tomasz Kaminski depicts in previous discussions.

As opposed to the current P/R of this issue, which does "always unwrap", this P/R does "always value-forward unless the incoming type is exactly a type that a special member function takes by reference, and don't unwrap if a value-forwarder can take an `optional` by any kind of reference".

I and Tomasz believe this is the best compromise between the different desires, and thus the best outcome so far.

This P/R has been implemented and tested on `libstdc++`.

[2016-09-08 Casey Carter finetunes existing resolution for move members]

[2016-09-09 Issues Resolution Telecom]

Move to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. Modify 20.6.3 [optional.object] as indicated:

```
template <class T> class optional
{
public:
    using value_type = T;

    // 20.6.3.1, Constructors
    constexpr optional() noexcept;
    constexpr optional(nullopt_t) noexcept;
    optional(const optional &);
    optional(optional &&) noexcept(see below);
constexpr optional(const T &);
constexpr optional(T &&);
    template <class... Args> constexpr explicit optional(in_place_t, Args &&...);
    template <class U, class... Args>
        constexpr explicit optional(in_place_t, initializer_list<U>, Args &&...);
template <class U = T> EXPLICIT constexpr optional(U &&);
template <class U> EXPLICIT optional(const optional<U> &);
template <class U> EXPLICIT optional(optional<U> &&);

    [...]

    // 20.6.3.3, Assignment
    optional &operator=(nullopt_t) noexcept;
    optional &operator=(const optional &);
    optional &operator=(optional &&) noexcept(see below);
    template <class U = T> optional &operator=(U &&);
template <class U> optional& operator=(const optional<U> &);
template <class U> optional& operator=(optional<U> &&);
    template <class... Args> void emplace(Args &&...);
    template <class U, class... Args>
        void emplace(initializer_list<U>, Args &&...);

    [...]

};
```

2. Change 20.6.3.1 [optional.object.ctor] as indicated:

```
optional(const optional<T>& rhs);

-3- Requires: is_copy_constructible_v<T> is true.

[...]

-?- Remarks: This constructor shall be defined as deleted unless is copy constructible v<T> is true.

optional(optional<T>&& rhs) noexcept(see below);

-7- Requires: is_move_constructible_v<T> is true.

[...]

-11- Remarks: The expression inside noexcept is equivalent to is_nothrow_move_constructible_v<T>. This constructor shall not participate in overload resolution unless is move constructible v<T> is true.

constexpr optional(const T& v);

-12- Requires: is_copy_constructible_v<T> is true.

-13- Effects: Initializes the contained value as if direct non-list-initializing an object of type T with the expression v.

-14- Postcondition: *this contains a value.

-15- Throws: Any exception thrown by the selected constructor of T.
```


~~-16- Remarks: If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor.~~

```
constexpr optional(T&& v);
```

~~-17- Requires: `is_move_constructible_v<T>` is true.~~

~~-18- Effects: Initializes the contained value as if direct-non-list-initializing an object of type τ with the expression `std::move(v)`.~~

~~-19- Postcondition: `*this` contains a value.~~

~~-20- Throws: Any exception thrown by the selected constructor of τ .~~

~~-21- Remarks: If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor.~~

```
template <class... Args> constexpr explicit optional(in_place_t, Args&&... args);
```

~~-22- Requires: `is_constructible_v<T, Args&&...>` is true.~~

[...]

-26- Remarks: If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, Args...>` is true.

```
template <class U, class... Args>
constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);
```

~~-27- Requires: `is_constructible_v<T, initializer_list<U>&, Args&&...>` is true.~~

[...]

-31- Remarks: ~~The function~~ This constructor shall not participate in overload resolution unless `is_constructible_v<T, initializer_list<U>&, Args&&...>` is true. If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor.

[Note: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — end note]

```
template <class U = T>
    EXPLICIT constexpr optional(U&& v);
```

?- Effects: Initializes the contained value as if direct-non-list-initializing an object of type τ with the expression `std::forward<U>(v)`.

?- Postconditions: `*this` contains a value.

?- Throws: Any exception thrown by the selected constructor of τ .

?- Remarks: If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is true, `is_same_v<U, in_place_t>` is false, and `is_same_v<optional<T>, decay_t<U>>` is false. The constructor is explicit if and only if `is_convertible_v<U&&, T>` is false.

```
template <class U>
    EXPLICIT optional(const optional<U>& rhs);
```

?- Effects: If `rhs` contains a value, initializes the contained value as if direct-non-list-initializing an object of type τ with the expression `*rhs`.

?- Postconditions: `bool(rhs) == bool(*this)`.

?- Throws: Any exception thrown by the selected constructor of τ .

?- Remarks: This constructor shall not participate in overload resolution unless `is_constructible_v<T, const U&>` is true, `is_constructible_v<T, optional<U>&>` is false, `is_constructible_v<T, const optional<U>&>` is false, `is_constructible_v<T, const`

`optional<U>&&> is false, is constructible v<T, optional<U>&&> is false, is convertible v<optional<U>&, T> is false, is convertible v<const optional<U>&, T> is false, is convertible v<const optional<U>&&, T> is false, and is convertible v<optional<U>&&, T> is false. The constructor is explicit if and only if is convertible v<const U&, T> is false.`

```
template <class U>
    EXPLICIT optional(optional<U>&& rhs);
```

-?- Effects: If `rhs` contains a value, initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)`. `bool(rhs)` is unchanged.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Throws: Any exception thrown by the selected constructor of `T`.

-?- Remarks: This constructor shall not participate in overload resolution unless `is constructible v<T, U&&> is true, is constructible v<T, optional<U>&> is false, is constructible v<T, const optional<U>&> is false, is constructible v<T, const optional<U>&&> is false, is constructible v<T, optional<U>&&> is false, is convertible v<optional<U>&, T> is false, is convertible v<const optional<U>&, T> is false, is convertible v<const optional<U>&&, T> is false, and is convertible v<optional<U>&&, T> is false. The constructor is explicit if and only if is convertible v<U&&, T> is false.`

3. Change 20.6.3.3 [optional.object.assign] as indicated:

```
optional<T>& operator=(const optional<T>& rhs);
```

~~-4- Requires: is_copy_constructible_v<T> is true and is_copy_assignable_v<T> is true.~~

[...]

-8- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. This operator shall be defined as deleted unless `is_copy_constructible_v<T> is true and is_copy_assignable_v<T> is true.`

```
optional<T>& operator=(optional<T>&& rhs) noexcept(see below);
```

~~-9- Requires: is_move_constructible_v<T> is true and is_move_assignable_v<T> is true.~~

[...]

-13- Remarks: The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>
```

-14- If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment. This operator shall not participate in overload resolution unless `is_move_constructible_v<T> is true and is_move_assignable_v<T> is true.`

```
template <class U = T> optional<T>& operator=(U&& v);
```

~~-15- Requires: is_constructible_v<T, U> is true and is_assignable_v<T&, U> is true.~~

[...]

-19- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment. ~~The~~This function shall not participate in overload

resolution unless `is_same_v<decay_t<U>, T>` is same `v<optional<T>, decay_t<U>>` is false, is constructible `v<T, U>` is true, and is assignable `v<T&, U>` is true.

-20- Notes: The reason for providing such generic assignment and then constraining it so that effectively `τ == U` is to guarantee that assignment of the form `o = {}` is unambiguous.

```
template <class U> optional<T>& operator=(const optional<U>& rhs);
```

-?- Effects: See Table ?.

Table ? — `optional::operator=(const optional<U>&)` effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>*rhs</code> to the contained value	initializes the contained value as if direct-non-list-initializing an object of type <code>T</code> with <code>*rhs</code>
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code>	no effect

-?- Returns: `*this`.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `τ`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `τ`'s constructor. If an exception is thrown during the call to `τ`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `τ`'s assignment. This function shall not participate in overload resolution unless `is_constructible_v<T, const U&>` is true, is assignable `v<T&, const U&>` is true, is constructible `v<T, optional<U>&>` is false, is constructible `v<T, const optional<U>&>` is false, is constructible `v<T, optional<U>&&>` is false, is convertible `v<optional<U>&, T>` is false, is convertible `v<const optional<U>&, T>` is false, is convertible `v<const optional<U>&&, T>` is false, is assignable `v<T&, optional<U>&>` is false, is assignable `v<T&, const optional<U>&&>` is false, and is assignable `v<T&, optional<U>&&>` is false.

```
template <class U> optional<T>& operator=(optional<U>&& rhs);
```

-?- Effects: See Table ?. The result of the expression `bool(rhs)` remains unchanged.

Table ? — `optional::operator=(optional<U>&&)` effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value as if direct-non-list-initializing an object of type <code>T</code> with <code>std::move(*rhs)</code>
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code>	no effect

-?- Returns: `*this`.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `τ`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `τ`'s constructor. If an exception is thrown during the call to `τ`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `τ`'s assignment. This function shall not participate in overload resolution unless `is_constructible_v<T, U>` is true, is assignable `v<T&, U>` is true, is constructible `v<T, optional<U>&>` is false, is constructible `v<T, const optional<U>&>` is false, is constructible `v<T, const optional<U>&&>` is false, is convertible `v<optional<U>&, T>` is false, is convertible `v<const optional<U>&, T>` is false, is convertible `v<const optional<U>&&, T>` is false, is assignable `v<T&, optional<U>&>` is false, is assignable `v<T&, const optional<U>&&>` is false, and is assignable `v<T&, optional<U>&&>` is false.

is false, is assignable v<T&, const optional<U>&&> is false, and is assignable v<T&, optional<U>&&> is false.

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.6.3 [optional.object] as indicated:

```
template <class T> class optional
{
public:
    using value_type = T;

    // 20.6.3.1, Constructors
    constexpr optional() noexcept;
    constexpr optional(nullopt_t) noexcept;
    optional(const optional &);
    optional(optional &&) noexcept(see below);
constexpr optional(const T &);
constexpr optional(T &&);
    template <class... Args> constexpr explicit optional(in_place_t, Args &&...);
    template <class U, class... Args>
        constexpr explicit optional(in_place_t, initializer_list<U>, Args &&...);
template <class U = T> EXPLICIT constexpr optional(U &&);
template <class U> EXPLICIT optional(const optional<U> &);
template <class U> EXPLICIT optional(optional<U> &&);

    [...]

    // 20.6.3.3, Assignment
    optional &operator=(nullopt_t) noexcept;
    optional &operator=(const optional &);
    optional &operator=(optional &&) noexcept(see below);
    template <class U = T> optional &operator=(U &&);
template <class U> optional& operator=(const optional<U> &);
template <class U> optional& operator=(optional<U> &&);
    template <class... Args> void emplace(Args &&...);
    template <class U, class... Args>
        void emplace(initializer_list<U>, Args &&...);

    [...]
};
```

2. Change 20.6.3.1 [optional.object.ctor] as indicated:

```
optional(const optional<T>& rhs);
```

~~-3- Requires: is_copy_constructible_v<T> is true.~~

[...]

-?- Remarks: This constructor shall be defined as deleted unless is_copy_constructible_v<T> is true.

```
optional(optional<T>&& rhs) noexcept(see below);
```

~~-7- Requires: is_move_constructible_v<T> is true.~~

[...]

~~-11- Remarks: The expression inside noexcept is equivalent to is_nothrow_move_constructible_v<T>. This constructor shall not participate in overload resolution unless is_move_constructible_v<T> is true.~~

```
constexpr optional(const T& v);
```

~~-12- Requires: is_copy_constructible_v<T> is true.~~

~~-13- Effects: Initializes the contained value as if direct-non-list-initializing an object of type T with the expression v.~~

~~-14- Postcondition: *this contains a value.~~

~~-15- Throws: Any exception thrown by the selected constructor of T.~~

~~-16- Remarks: If T's selected constructor is a constexpr constructor, this constructor shall be a constexpr~~

~~constructor.~~

~~constexpr optional(T&& v);~~

~~-17- Requires: is_move_constructible_v<T> is true.~~

~~-18- Effects: Initializes the contained value as if direct-non-list-initializing an object of type τ with the expression `std::move(v)`.~~

~~-19- Postcondition: *this contains a value.~~

~~-20- Throws: Any exception thrown by the selected constructor of τ .~~

~~-21- Remarks: If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor.~~

template <class... Args> constexpr explicit optional(in_place_t, Args&&... args);

~~-22- Requires: is_constructible_v<T, Args&&...> is true.~~

[...]

-26- Remarks: If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless is_constructible_v<T, Args&&...> is true.

template <class U, class... Args>
constexpr explicit optional(in_place_t, initializer_list<U> il, Args&&... args);

~~-27- Requires: is_constructible_v<T, initializer_list<U>&, Args&&...> is true.~~

[...]

-31- Remarks: ~~The function~~ This constructor shall not participate in overload resolution unless is_constructible_v<T, initializer_list<U>&, Args&&...> is true. If τ 's constructor selected for the initialization is a constexpr constructor, this constructor shall be a constexpr constructor.

[Note: The following constructors are conditionally specified as explicit. This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — end note]

template <class U = T>
EXPLICIT constexpr optional(U&& v);

-?- Effects: Initializes the contained value as if direct-non-list-initializing an object of type τ with the expression `std::forward<U>(v)`.

-?- Postconditions: *this contains a value.

-?- Throws: Any exception thrown by the selected constructor of τ .

-?- Remarks: If τ 's selected constructor is a constexpr constructor, this constructor shall be a constexpr constructor. This constructor shall not participate in overload resolution unless is_constructible_v<T, U&&> is true, is_same_v<U, in_place_t> is false, and is_same_v<optional<T>, decay_t<U>> is false. The constructor is explicit if and only if is_convertible_v<U&&, T> is false.

template <class U>
EXPLICIT optional(const optional<U>& rhs);

-?- Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type τ with the expression `*rhs`.

-?- Postconditions: `bool(rhs) == bool(*this)`.

-?- Throws: Any exception thrown by the selected constructor of τ .

-?- Remarks: This constructor shall not participate in overload resolution unless is_constructible_v<T, const U&> is true, is_constructible_v<T, optional<U>&> is false, is_constructible_v<T, const optional<U>&> is false, is_constructible_v<T, const optional<U>&&> is false, is_constructible_v<T, optional<U>&&> is false, is_convertible_v<optional<U>&, T> is false, is_convertible_v<const optional<U>&, T> is false, is_convertible_v<const optional<U>&&, T> is false, and is_convertible_v<optional<U>&&, T> is false. The constructor is explicit if and only if is_convertible_v<const U&, T> is false.

```
template <class U>
    EXPLICIT optional(optional<U>&& rhs);
```

-?- *Effects:* If `rhs` contains a value, initializes the contained value as if direct-non-list-initializing an object of type `T` with the expression `std::move(*rhs)`. `bool(rhs)` is unchanged.

-?- *Postconditions:* `bool(rhs) == bool(*this)`.

-?- *Throws:* Any exception thrown by the selected constructor of `T`.

-?- *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible_v<T, U&&>` is true, `is_constructible_v<T, optional<U>&>` is false, `is_constructible_v<T, const optional<U>&>` is false, `is_constructible_v<T, const optional<U>&&>` is false, `is_constructible_v<T, optional<U>&&>` is false, `is_convertible_v<optional<U>&, T>` is false, `is_convertible_v<const optional<U>&, T>` is false, `is_convertible_v<const optional<U>&&, T>` is false, and `is_convertible_v<optional<U>&&, T>` is false. The constructor is explicit if and only if `is_convertible_v<U&&, T>` is false.

3. Change 20.6.3.3 [optional.object.assign] as indicated:

```
optional<T>& operator=(const optional<T>& rhs);
```

~~4- *Requires:* `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true.~~

[...]

-8- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. This operator shall be defined as deleted unless `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true.

```
optional<T>& operator=(optional<T>&& rhs) noexcept(see below);
```

~~9- *Requires:* `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.~~

[...]

-13- *Remarks:* The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_assignable_v<T> && is_nothrow_move_constructible_v<T>
```

-14- If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment. This operator shall not participate in overload resolution unless `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.

```
template <class U = T> optional<T>& operator=(U&& v);
```

~~15- *Requires:* `is_constructible_v<T, U>` is true and `is_assignable_v<T&, U>` is true.~~

[...]

-19- *Remarks:* If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment. The `This` function shall not participate in overload resolution unless `is_same_v<decay_t<U>, T>` is same `v<optional<T>, decay_t<U>>` is false, conjunction `v<is scalar<T>, is same<T, decay_t<U>>>` is false, `is_constructible_v<T, U>` is true, and `is_assignable_v<T&, U>` is true.

~~20- *Notes:* The reason for providing such generic assignment and then constraining it so that effectively `T == U` is to guarantee that assignment of the form `o = {}` is unambiguous.~~

```
template <class U> optional<T>& operator=(const optional<U>& rhs);
```

-?- *Effects:* See Table ?.

Table ? — `optional::operator=(const optional<U>&)` effects

*this contains a value	*this does not contain a value

rhs contains a value	assigns *rhs to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with *rhs
rhs does not contain a value	destroys the contained value by calling val->T::~T()	no effect

-?- Returns: *this.

-?- Postconditions: bool(rhs) == bool(*this).

-?- Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment. This function shall not participate in overload resolution unless is_constructible v<T, const U&> is true, is_assignable v<T&, const U&> is true, is_constructible v<T, optional<U>&> is false, is_constructible v<T, const optional<U>&> is false, is_constructible v<T, const optional<U>&&> is false, is_constructible v<T, optional<U>&&> is false, is_convertible v<optional<U>&, T> is false, is_convertible v<const optional<U>&, T> is false, is_convertible v<const optional<U>&&, T> is false, is_convertible v<optional<U>&&, T> is false, is_assignable v<T&, optional<U>&> is false, is_assignable v<T&, const optional<U>&> is false, is_assignable v<T&, const optional<U>&&> is false, and is_assignable v<T&, optional<U>&&> is false.

```
template <class U> optional<T>& operator=(optional<U>&& rhs):
```

-?- Effects: See Table ?. The result of the expression bool(rhs) remains unchanged.

Table ? — optional::operator=(optional<U>&&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns std::move(*rhs) to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with std::move(*rhs)
rhs does not contain a value	destroys the contained value by calling val->T::~T()	no effect

-?- Returns: *this.

-?- Postconditions: bool(rhs) == bool(*this).

-?- Remarks: If any exception is thrown, the result of the expression bool(*this) remains unchanged. If an exception is thrown during the call to T's constructor, the state of *rhs.val is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of *val and *rhs.val is determined by the exception safety guarantee of T's assignment. This function shall not participate in overload resolution unless is_constructible v<T, U> is true, is_assignable v<T&, U> is true, is_constructible v<T, optional<U>&> is false, is_constructible v<T, const optional<U>&> is false, is_constructible v<T, const optional<U>&&> is false, is_constructible v<T, optional<U>&&> is false, is_convertible v<optional<U>&, T> is false, is_convertible v<const optional<U>&, T> is false, is_convertible<const optional<U>&&, T> is false, is_convertible<optional<U>&&, T> is false, is_assignable v<T&, optional<U>&> is false, is_assignable v<T&, const optional<U>&> is false, is_assignable v<T&, const optional<U>&&> is false, and is_assignable v<T&, optional<U>&&> is false.

2758. std::string{}.assign("ABCDE", 0, 1) is ambiguous

Section: 21.3.1.6.3 [string::assign] Status: [Tentatively Ready](#) Submitter: Marshall Clow Opened: 2016-07-30 Last modified: 2016-10-16

Priority: 1

View all other [issues](#) in [string::assign].

View all issues with [Tentatively Ready](#) status.

Discussion:

Before C++17, we had the following signature to std::basic_string:

```
basic_string&
assign(const basic_string& str, size_type pos, size_type n = npos);
```

Unlike most of the other member functions on std::basic_string, there were not corresponding versions that take a charT* or

(charT *, size).

In [p0254r2](#), we (I) added:

```
basic_string&
    assign(basic_string_view<charT, traits> sv, size_type pos, size_type n = npos);
```

which made the code above ambiguous. There are two conversions from "const charT*", one to basic_string, and the other to basic_string_view, and they're both equally good (in the view of the compiler).

This ambiguity also occurs with the calls

```
insert(size_type pos1, const basic_string& str, size_type pos2, size_type n = npos);
insert(size_type pos1, basic_string_view<charT, traits> sv, size_type pos2, size_type n = npos);
```

but I will file a separate issue ([2757](#)) for that.

A solution is to add *even more* overloads to assign, to make it match all the other member functions of basic_string, which come in fours (string, pointer, pointer + size, string_view).

[2016-08-03, Chicago, Robert Douglas provides wording]

[2016-08-05, Tim Song comments]

On the assumption that the basic_string version is untouchable, I like the updated P/R, with a couple comments:

1. If it's constraining on is_convertible to basic_string_view, then I think it should take by reference to avoid copying T, which can be arbitrarily expensive. Both const T& and T&& should work; the question is whether to accommodate non-const operator basic_string_view()s (which arguably shouldn't exist).
2. Minor issue: compare tests is_convertible and then uses direct-initialization syntax (which is is_constructible). They should match, because it's possible to have basic_string_view sv = t; succeed yet basic_string_view sv(t); fail.

[2016-08-05, Chicago LWG]

1. breaking ABI was a non-starter
2. we want to convert char const* to basic_string_view if possible
3. we want to take the original type by reference, to avoid new user string types from being copied
4. it is unfortunate that each length of string literal will generate a new entry in the symbol table

Given feedback from discussion, we wish to go with the is_convertible_v method, but change:

1. the function parameter type to T const& for each overload
2. the parameter type in is_convertible_v to T const&
3. the initialization of sv(t) to sv = t

[2016-08, Chicago]

Fri PM: Move to Tentatively Ready

[2016-08-16, Jonathan Wakeley reopens]

The P/R is not correct, the new overloads get chosen in preference to the overloads taking const char* when passed a char*:

```
#include <string>

int main () {
    std::string str("a");
    char c = 'b';
    str.replace(0, 1, &c, 1);
    if (str[0] != 'b')
        __builtin_abort();
}
```

With the resolution of 2758 this is now equivalent to:

```
str.replace(0, 1, string_view{"b", 1}.substr(1, npos) i.e. an empty string.
```

which replaces the character with string_view{"b", 1}.substr(1, npos) i.e. an empty string.

The SFINAE constraints need to disable the new overloads for (at least) the case of non-const value_type* arguments.

I've implemented an alternative resolution, which would be specified as:

Remarks: This function shall not participate in overload resolution unless `is_convertible_v<const T&, basic_string_view<charT, traits>>` is true **and `is_convertible_v<const T&, const charT*>` is false**.

All the overloads have the same problem.

This program prints no output in C++14, and we should make sure it prints no output in C++17 too:

```
#include <string>

int main()
{
    std::string str("a");
    char c[1] = { 'b' };
    str.append(c, 1);
    if (str != "ab")
        puts("bad append");
    str.assign(c, 1);
    if (str != "b")
        puts("bad assign");
    str.insert(0, c, 1);
    if (str != "bb")
        puts("bad insert");
    str.replace(0, 2, c, 1);
    if (str != "b")
        puts("bad replace");
    if (str.compare(0, 1, c, 1))
        puts("bad compare");
}
```

Ville and I considered "`is_same_v<decay_t<T>, char*>` is false" but that would still select the wrong overload for an array of const char, because the new function template would be preferred to doing an array-to-pointer conversion to call the old overload.

[2016-09-09 Issues Resolution Telecom]

Ville to provide updated wording; Marshall to implement

[2016-10-05]

Ville provides revised wording.

[2016-10 Telecom]

Ville's wording has been implemented in libstdc++ and libc++. Move this to Tentatively Ready and [2757](#) to Tentatively Resolved

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. In 21.3.1 [basic.string] modify the synopsis for `basic_string` as follows:

```
namespace std {
    template<class charT, class traits = char_traits<charT>,
            class Allocator = allocator<charT>>
    class basic_string {
    public:
        [...]
        template<class T>
        basic_string& append(basic_string_view<charT, traits> svconst T& t,
                           size_type pos, size_type n = npos);

        [...]
        template<class T>
        basic_string& assign(basic_string_view<charT, traits> svconst T& t,
                             size_type pos, size_type n = npos);

        [...]
        template<class T>
        basic_string& insert(size_type pos1, basic_string_view<charT, traits> svconst T& t,
                             size_type pos2, size_type n = npos);

        [...]
        template<class T>
        basic_string& replace(size_type pos1, size_type n1,
                              basic_string_view<charT, traits> svconst T& t,
                              size_type pos2, size_type n2 = npos);

        [...]
        template<class T>
        int compare(size_type pos1, size_type n1,
                    basic_string_view<charT, traits> svconst T& t,
```

```

        size_type pos2, size_type n2 = npos) const;
    [...]
};
}

```

2. In 21.3.1.6.2 [string::append], modify basic_string_view overload as follows:

```

template<class T>
basic_string& append(basic_string_view<charT, traits> sv const T& t,
                    size_type pos, size_type n = npos);

```

-7- *Throws*: out_of_range if pos > sv.size().

-8- *Effects*: Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t. Determines the effective length rlen of the string to append as the smaller of n and sv.size() - pos and calls append(sv.data() + pos, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible v<const T&, basic_string_view<charT, traits>> is true.

-9- *Returns*: *this.

3. In 21.3.1.6.3 [string::assign], modify basic_string_view overload as follows:

```

template<class T>
basic_string& assign(basic_string_view<charT, traits> sv const T& t,
                    size_type pos, size_type n = npos);

```

-9- *Throws*: out_of_range if pos > sv.size().

-10- *Effects*: Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t. Determines the effective length rlen of the string to assign as the smaller of n and sv.size() - pos and calls assign(sv.data() + pos, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible v<const T&, basic_string_view<charT, traits>> is true.

-11- *Returns*: *this.

4. In 21.3.1.6.4 [string::insert], modify basic_string_view overload as follows:

```

template<class T>
basic_string& insert(size_type pos1, basic_string_view<charT, traits> sv const T& t,
                    size_type pos2, size_type n = npos);

```

-6- *Throws*: out_of_range if pos1 > size() OR pos2 > sv.size().

-7- *Effects*: Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t. Determines the effective length rlen of the string to insert as the smaller of n and sv.size() - pos2 and calls insert(pos1, sv.data() + pos2, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible v<const T&, basic_string_view<charT, traits>> is true.

-8- *Returns*: *this.

5. In 21.3.1.6.6 [string::replace], modify basic_string_view overload as follows:

```

template<class T>
basic_string& replace(size_type pos1, size_type n1,
                    basic_string_view<charT, traits> sv const T& t,
                    size_type pos2, size_type n2 = npos);

```

-6- *Throws*: out_of_range if pos1 > size() OR pos2 > sv.size().

-7- *Effects*: Creates a variable, sv, as if by basic_string_view<charT, traits> sv = t. Determines the effective length rlen of the string to be inserted as the smaller of n2 and sv.size() - pos2 and calls replace(pos1, n1, sv.data() + pos2, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible v<const T&, basic_string_view<charT, traits>> is true.

-8- *Returns*: *this.

-9- *Throws*: out_of_range if pos > sv.size().

-10- *Effects*: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`. Determines the effective length rlen of the string to assign as the smaller of n and sv.size() - pos and calls assign(sv.data() + pos, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible `v<const T&, basic_string_view<charT, traits>>` is true and is convertible `v<const T&, const charT*>` is false.

-11- *Returns*: *this.

4. In 21.3.1.6.4 [string::insert], modify basic_string_view overload as follows:

```
template<class T>
basic_string& insert(size_type pos1, basic_string_view<charT, traits> svconst T& t,
                    size_type pos2, size_type n = npos);
```

-6- *Throws*: out_of_range if pos1 > size() OR pos2 > sv.size().

-7- *Effects*: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`. Determines the effective length rlen of the string to assign as the smaller of n and sv.size() - pos2 and calls insert(pos1, sv.data() + pos2, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible `v<const T&, basic_string_view<charT, traits>>` is true and is convertible `v<const T&, const charT*>` is false.

-8- *Returns*: *this.

5. In 21.3.1.6.6 [string::replace], modify basic_string_view overload as follows:

```
template<class T>
basic_string& replace(size_type pos1, size_type n1,
                     basic_string_view<charT, traits> svconst T& t,
                     size_type pos2, size_type n2 = npos);
```

-6- *Throws*: out_of_range if pos1 > size() OR pos2 > sv.size().

-7- *Effects*: Creates a variable, sv, as if by `basic_string_view<charT, traits> sv = t`. Determines the effective length rlen of the string to be inserted as the smaller of n2 and sv.size() - pos2 and calls replace(pos1, n1, sv.data() + pos2, rlen).

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible `v<const T&, basic_string_view<charT, traits>>` is true and is convertible `v<const T&, const charT*>` is false.

-8- *Returns*: *this.

[Drafting note: The wording changes below are for the same part of the standard as [2771](#). However, they do not conflict. This one changes the definition of the routine, while the other changes the "Effects". — end drafting note]

6. In 21.3.1.7.9 [string::compare], modify basic_string_view overload as follows:

```
template<class T>
int compare(size_type pos1, size_type n1,
            basic_string_view<charT, traits> svconst T& t,
            size_type pos2, size_type n2 = npos) const;
```

-4- *Effects*: Equivalent to:

```
{
    basic_string_view<charT, traits> sv = t;
    return basic_string_view<charT, traits>(this.data(), pos1, n1).compare(sv, pos2, n2);
}
```

-?- *Remarks*: This function shall not participate in overload resolution unless is convertible `v<const T&, basic_string_view<charT, traits>>` is true and is convertible `v<const T&, const charT*>` is false.

08-01 **Last modified:** 2016-08-02

Priority: Not Prioritized

View all issues with [Tentatively Ready](#) status.

Discussion:

With the acceptance of gcd and lcm in the working draft, the same problem as pointed out by LWG [2733](#) exists here as well and should be fixed accordingly.

[2016-08, Chicago]

Monday PM: Moved to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Adjust 26.8.13 [numeric.ops.gcd] p2 as indicated:

```
template<class M, class N>
constexpr common_type_t<M, N> gcd(M m, N n);
```

[...]

-2- *Remarks:* If either M or N is not an integer type, **or if either is (possibly cv-qualified) bool**, the program is ill-formed.

2. Adjust 26.8.14 [numeric.ops.lcm] p2 as indicated:

```
template<class M, class N>
constexpr common_type_t<M, N> lcm(M m, N n);
```

[...]

-2- *Remarks:* If either M or N is not an integer type, **or if either is (possibly cv-qualified) bool**, the program is ill-formed.

[2760](#). non-const basic_string::data should not invalidate iterators

Section: 21.3.1.1 [string.require] **Status:** [Tentatively Ready](#) **Submitter:** Billy Baker **Opened:** 2016-08-03 **Last modified:** 2016-08-08

Priority: Not Prioritized

View other [active issues](#) in [string.require].

View all other [issues](#) in [string.require].

View all issues with [Tentatively Ready](#) status.

Discussion:

21.3.1.1 [string.require]/4 does not list non-const basic_string::data() as being a function that may be called with the guarantee that it will not invalidate references, pointers, and iterators to elements of a basic_string object.

[2016-08 Chicago]

Wed PM: Move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Change 21.3.1.1 [string.require]/4 as indicated:

-4- References, pointers, and iterators referring to the elements of a basic_string sequence may be invalidated by the following uses of that basic_string object:

- as an argument to any standard library function taking a reference to non-const basic_string as an argument. (footnote 230)

- Calling non-const member functions, except operator[], at, **data**, front, back, begin, rbegin, end, and rend.

2765. Did LWG 1123 go too far?

Section: 27.5.3.1.6 [ios::Init] **Status:** [Tentatively Ready](#) **Submitter:** Richard Smith **Opened:** 2016-08-13 **Last modified:** 2016-09-15

Priority: 0

View all other [issues](#) in [ios::Init].

View all issues with [Tentatively Ready](#) status.

Discussion:

[1123](#) fixed a bug where users of `<iostream>` were not guaranteed to have their streams flushed on program shutdown. However, it also added this rule:

"Similarly, the entire program shall behave as if there were at least one instance of `ios_base::Init` with static lifetime."

This seems pointless: it only affects the behavior of programs that never include `<iostream>` (because programs that do include it are already guaranteed at least one such instance), and those programs do not need an implicit flush because they cannot have written to the relevant streams.

It's also actively harmful, because it requires the iostreams component to be linked into programs that do not use it, apparently even including freestanding implementations! Fortunately, C++ implementations appear to uniformly ignore this rule.

Can it be removed?

[2016-09-09 Issues Resolution Telecom]

P0; move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 27.5.3.1.6 [ios::Init] p3 as indicated:

-3- The objects are constructed and the associations are established at some time prior to or during the first time an object of class `ios_base::Init` is constructed, and in any case before the body of `main` begins execution.(footnote 293) The objects are not destroyed during program execution.(footnote 294) The results of including `<iostream>` in a translation unit shall be as if `<iostream>` defined an instance of `ios_base::Init` with static storage duration. ~~Similarly, the entire program shall behave as if there were at least one instance of `ios_base::Init` with static storage duration.~~

2767. `not_fn` `call_wrapper` can form invalid types

Section: 20.14.9 [func.not_fn] **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2016-08-19 **Last modified:** 2016-09-11

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

The definition of the `call_wrapper` type in the C++17 CD means this fails to compile:

```
#include <functional>

struct abc { virtual void f() const = 0; };
struct derived : abc{ void f() const { } };
struct F { bool operator()(abc&) { return false; } };
bool b = std::not_fn(F{})(derived{});
```

The problem is that the return types use `result_of_t<F(abc)>` and `F(abc)` is not a valid function type, because it takes an abstract class by value.

The return types should use `result_of_t<F(Args&&...)>` instead.

P0; move to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.14.9 [func.not_fn], class *call_wrapper* synopsis, as indicated:

```
class call_wrapper
{
    [...]
    template<class... Args>
        auto operator()(Args&&...) &
            -> decltype(!declval<result_of_t<FD&(Args&&...)>>());
    template<class... Args>
        auto operator()(Args&&...) const&
            -> decltype(!declval<result_of_t<FD const&(Args&&...)>>());
    template<class... Args>
        auto operator()(Args&&...) &&
            -> decltype(!declval<result_of_t<FD(Args&&...)>>());
    template<class... Args>
        auto operator()(Args&&...) const&&
            -> decltype(!declval<result_of_t<FD const(Args&&...)>>());
    [...]
};
```

2. Modify the prototype declarations of 20.14.9 [func.not_fn] as indicated:

```
template<class... Args>
    auto operator()(Args&&... args) &
        -> decltype(!declval<result_of_t<FD&(Args&&...)>>());
template<class... Args>
    auto operator()(Args&&... args) const&
        -> decltype(!declval<result_of_t<FD const&(Args&&...)>>());

    [...]

template<class... Args>
    auto operator()(Args&&... args) &&
        -> decltype(!declval<result_of_t<FD(Args&&...)>>());
template<class... Args>
    auto operator()(Args&&... args) const&&
        -> decltype(!declval<result_of_t<FD const(Args&&...)>>());
```

2768. any_cast and move semantics

Section: 20.8.4 [any.nonmembers] **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2016-08-27 **Last modified:** 2016-10-05

Priority: 0

View other [active issues](#) in [any.nonmembers].

View all other [issues](#) in [any.nonmembers].

View all issues with [Tentatively Ready](#) status.

Discussion:

LWG [2509](#) made two changes to the specification of *any* in v2 of the library fundamentals TS:

1. It altered the effects of the *any_cast*(*any*&&) overload to enable moving the value out of the *any* object and/or obtaining an rvalue reference to the contained value.
2. It made changes to support pathological copyable-but-not-movable contained values, which is madness.

Change 1 has very desirable effects; I propose that we apply the sane part of LWG [2509](#) to *any* in the C++17 WP, for all of the reasons cited in the discussion of LWG [2509](#).

P0; move to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. In 20.8.4 [any.nonmembers] p5, edit as follows:

```
template<class ValueType>
    ValueType any_cast(const any& operand);
template<class ValueType>
    ValueType any_cast(any& operand);
template<class ValueType>
    ValueType any_cast(any&& operand);
```

-4- *Requires*: `is_reference_v<ValueType>` is true OR `is_copy_constructible_v<ValueType>` is true. Otherwise the program is ill-formed.

-5- *Returns*: For the first form, `*any_cast<add_const_t<remove_reference_t<ValueType>>>(&operand)`. For the second and third forms, `*any_cast<remove_reference_t<ValueType>>(&operand)`. For the third form, `std::forward<ValueType>(*any_cast<remove_reference_t<ValueType>>(&operand))`.

[...]

Proposed resolution:

Resolved by the wording provided by LWG [2769](#).

[2771](#). Broken *Effects* of some `basic_string::compare` functions in terms of `basic_string_view`

Section: 21.3.1.7.9 [string::compare] **Status:** [Tentatively Ready](#) **Submitter:** Daniel Krügler **Opened:** 2016-09-05 **Last modified:** 2016-10-16

Priority: 1

View all issues with [Tentatively Ready](#) status.

Discussion:

Some `basic_string::compare` functions are specified in terms of a non-existing `basic_string_view` constructor, namely 21.3.1.7.9 [string::compare] p3,

```
return basic_string_view<charT, traits>(this.data(), pos1, n1).compare(sv);
```

and 21.3.1.7.9 [string::compare] p4:

```
return basic_string_view<charT, traits>(this.data(), pos1, n1).compare(sv, pos2, n2);
```

because there doesn't exist a `basic_string_view` constructor with three arguments.

Albeit this can be easily fixed by a proper combination of the existing constructor

```
constexpr basic_string_view(const charT* str, size_type len);
```

with the additional member function

```
constexpr basic_string_view substr(size_type pos = 0, size_type n = npos) const;
```

it should be decided whether adding the seemingly natural constructor

```
constexpr basic_string_view(const charT* str, size_type pos, size_type n);
```

could simplify matters. A counter argument for this addition might be, that `basic_string` doesn't provide this constructor either.

Another problem is related to the specification of 21.3.1.7.9 [string::compare] p4, which attempts to call a non-existing `basic_string_view::compare` overload that would match the signature:

```
constexpr int compare(basic_string_view str, size_type pos1, size_type n1) const;
```

[2016-09-09 Issues Resolution Telecom]

Marshall to investigate using P/R vs. adding the missing constructor.

Marshall reports that P/R is better. Status to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

[Drafting note: The wording changes below are for the same part of the standard as [2758](#). However, they do not conflict. This one changes the "Effects" of the routine, while the other changes the definition. — end drafting note]

1. Change 21.3.1.7.9 [string::compare] as indicated:

```
int compare(size_type pos1, size_type n1,
            basic_string_view<charT, traits> sv) const;
```

-3- *Effects*: Equivalent to:

```
return basic_string_view<charT, traits>(&this->data(), &size().substr(pos1, n1).compare(sv);
```

```
int compare(size_type pos1, size_type n1,
            basic_string_view<charT, traits> sv,
            size_type pos2, size_type n2 = npos) const;
```

-4- *Effects*: Equivalent to:

```
return basic_string_view<charT, traits>(&this->data(), &size().substr(pos1, n1).compare(sv, &size().substr(pos2, n2));
```

[2773](#). Making `std::ignore` constexpr

Section: 20.5.1 [tuple.general] **Status:** [Tentatively Ready](#) **Submitter:** Vincent Reverdy **Opened:** 2016-09-10 **Last modified:** 2016-10-07

Priority: 0

View other [active issues](#) in [tuple.general].

View all other [issues](#) in [tuple.general].

View all issues with [Tentatively Ready](#) status.

Discussion:

Currently `std::ignore` is not specified constexpr according to the C++ draft N4606 in the paragraph 20.5.1 [tuple.general]. It prevents some use in constexpr context: for example declaring a constexpr variable equals to the result of a function to which `std::ignore` has been passed as a parameter:

```
constexpr int i = f(std::ignore); // Won't compile
```

If there is no fundamental reason preventing `std::ignore` to be constexpr, then we propose to declare it as constexpr instead of as const.

[Issues processing Telecom 2016-10-7]

P0; set to Tentatively Ready

Proposed resolution:

This wording is relative to N4606.

1. Modify 20.5.1 [tuple.general] as indicated:

```
// 20.5.2.4, tuple creation functions:
constexpr unspecified ignore;
```

[2777](#). `basic_string_view::copy` should use `char_traits::copy`

Section: 21.4.2.6 [string.view.ops], 21.3.1.6.7 [string::copy] **Status:** [Tentatively Ready](#) **Submitter:** Billy Robert O'Neal III **Opened:** 2016-09-27 **Last modified:** 2016-10-07

Priority: 0

View all issues with [Tentatively Ready](#) status.

Discussion:

`basic_string_view::copy` is inconsistent with `basic_string::copy`, in that the former uses `copy_n` and the latter uses `traits::copy`. We should have this handling be consistent.

Moreover, the `basic_string::copy` description is excessively roundabout due to copy-on-write era wording.

[Issues processing Telecom 2016-10-7]

P0; set to Tentatively Ready

Removed "Note to project editor", since the issue there has been fixed in the current draft.

Proposed resolution:

This wording is relative to N4606.

1. Change 21.3.1.6.7 [string::copy] as indicated:

```
size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

-?- Let `rlen` be the smaller of `n` and `size() - pos`.

-2- Throws: `out_of_range` if `pos > size()`.

-?- Requires: `[s, s + rlen)` is a valid range.

-3- Effects: Determines the effective length `rlen` of the string to copy as the smaller of `n` and `size() - pos`. `s` shall designate an array of at least `rlen` elements. Equivalent to: `traits::copy(s, data() + pos, rlen)`. [Note: This does not terminate `s` with a null object. — end note]

The function then replaces the string designated by `s` with a string of length `rlen` whose elements are a copy of the string controlled by `*this` beginning at position `pos`.

The function does not append a null object to the string designated by `s`.

-4- Returns: `rlen`.

2. Change 21.4.2.6 [string.view.ops] as indicated:

```
size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

-1- Let `rlen` be the smaller of `n` and `size() - pos`.

-2- Throws: `out_of_range` if `pos > size()`.

-3- Requires: `[s, s + rlen)` is a valid range.

-4- Effects: Equivalent to: `copy_n(begin() + pos, rlen, s)` `traits::copy(s, data() + pos, rlen)`

-5- Returns: `rlen`.

-6- Complexity: $O(rlen)$.

2778. basic_string_view is missing constexpr

Section: 21.4 [string.view] Status: [Tentatively Ready](#) Submitter: Billy Robert O'Neal III Opened: 2016-09-30 Last modified: 2016-10-07

Priority: 0

View other [active issues](#) in [string.view].

View all other [issues](#) in [string.view].

View all issues with [Tentatively Ready](#) status.

Discussion:

basic_string_view was not updated to account for other library machinery made constexpr in Oulu. Now that reverse_iterator is constexpr there's no reason the reverse range functions can't be. Also, now that we have C++14 relaxed constexpr, we can also take care of the assignment operator and copy.

[Issues processing Telecom 2016-10-7]

split off the copy into its' own issue [2780](#)

P0; set what's left to Tentatively Ready

Previous resolution [SUPERSEDED]:

This wording is relative to N4606.

1. In 21.4.2 [string.view.template], add constexpr to the assignment operator:

```
constexpr basic_string_view& operator=(const basic_string_view&) noexcept = default;
```

2. In 21.4.2 [string.view.template], add constexpr to the reverse range functions:

```
constexpr const_reverse_iterator rbegin() const noexcept;  
constexpr const_reverse_iterator rend() const noexcept;  
constexpr const_reverse_iterator crbegin() const noexcept;  
constexpr const_reverse_iterator crend() const noexcept;
```

3. In 21.4.2.2 [string.view.iterators], add constexpr to the reverse range functions:

```
constexpr const_reverse_iterator rbegin() const noexcept;  
constexpr const_reverse_iterator crbegin() const noexcept;
```

-6- Returns: const_reverse_iterator(end())

```
constexpr const_reverse_iterator rend() const noexcept;  
constexpr const_reverse_iterator crend() const noexcept;
```

-7- Returns: const_reverse_iterator(begin())

4. In 21.4.2 [string.view.template], add constexpr to copy:

```
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

5. In 21.4.2.6 [string.view.ops], add constexpr to copy:

```
constexpr size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

Proposed resolution:

This wording is relative to N4606.

1. In 21.4.2 [string.view.template], add constexpr to the assignment operator:

```
constexpr basic_string_view& operator=(const basic_string_view&) noexcept = default;
```

2. In 21.4.2 [string.view.template], add constexpr to the reverse range functions:

```
constexpr const_reverse_iterator rbegin() const noexcept;  
constexpr const_reverse_iterator rend() const noexcept;  
constexpr const_reverse_iterator crbegin() const noexcept;  
constexpr const_reverse_iterator crend() const noexcept;
```

3. In 21.4.2.2 [string.view.iterators], add constexpr to the reverse range functions:

```
constexpr const_reverse_iterator rbegin() const noexcept;  
constexpr const_reverse_iterator crbegin() const noexcept;
```

-6- Returns: const_reverse_iterator(end())

```
constexpr const_reverse_iterator rend() const noexcept;  
constexpr const_reverse_iterator crend() const noexcept;
```

-7- Returns: const_reverse_iterator(begin())